UNIVERSIDADE TECNOLÓGICA FEDERAL DO PARANÁ
DIRETORIA DE PESQUISA E PÓS-GRADUAÇAO
PROGRAMA DE PÓS-GRADUAÇÃO EM CIÊNCIA DA COMPUTAÇÃO

VICTOR PEDROSO AMBIEL BARROS

AVALIAÇÃO DO DESEMPENHO DE ALGORITMOS DE RETROPROPAGAÇÃO COM REDES NEURAIS ARTIFICIAIS PARA A RESOLUÇÃO DE PROBLEMAS NÃO-LINEARES

DISSERTAÇÃO

PONTA GROSSA

2018

VICTOR PEDROSO AMBIEL BARROS

AVALIAÇÃO DO DESEMPENHO DE ALGORITMOS DE RETROPROPAGAÇÃO COM REDES NEURAIS ARTIFICIAIS PARA A RESOLUÇÃO DE PROBLEMAS NÃO-LINEARES

Dissertação apresentada como requisito parcial à obtenção do título de Mestre em Ciência da Computação do programa de Pós-Graduação em Ciência da Computação da Universidade Tecnológica Federal do Paraná – Campus Ponta Grossa.

Área de Concentração: Sistemas E Métodos De Computação

Orientador: Prof. Dr. Max Mauro Dias Santos

PONTA GROSSA

2018

Ficha catalográfica elaborada pelo Departamento de Biblioteca
da Universidade Tecnológica Federal do Paraná, Câmpus Ponta Grossa
n.40/18

B277 Barros, Victor Pedroso Ambiel

Avaliação do desempenho de algoritmos de retropropagação com redes neurais artificiais para a resolução de problemas não-lineares. / Victor Pedroso Ambiel Barros. 2018.

135 f.; il. 30 cm

Orientador: Prof. Dr. Max Mauro Dias Santos

Dissertação (Mestrado em Ciência da Computação) - Programa de Pós-Graduação em Ciência da Computação. Universidade Tecnológica Federal do Paraná, Ponta Grossa, 2018.

1. Redes neurais (Computação). 2. Avaliação. 3. Desempenho. 4. Algoritmos. I. Santos, Max Mauro Dias. II. Universidade Tecnológica Federal do Paraná. III. Título.

CDD 004

Elson Heraldo Ribeiro Junior. CRB-9/1413. 24/07/2018.

	Ministério da Educação **UNIVERSIDADE TECNOLÓGICA FEDERAL DO PARANÁ** **Câmpus Ponta Grossa** Diretoria de Pesquisa e Pós-Graduação Programa de Pós-Graduação em Ciência da Computação	

FOLHA DE APROVAÇÃO

Título de Dissertação Nº **3/2018**

AVALIAÇÃO DO DESEMPENHO DE ALGORITMOS DE RETROPROPAGAÇÃO COM REDES NEURAIS ARTIFICIAIS PARA A RESOLUÇÃO DE PROBLEMAS NÃO-LINEARES

Por

Victor Pedroso Ambiel Barros

Esta dissertação foi apresentada às **8 horas 30 minutos** de **05 de julho de 2018**, na **Miniauditório do Bloco V**, como requisito parcial para a obtenção do título de MESTRE EM CIÊNCIA DA COMPUTAÇÃO, Programa de Pós-Graduação em Ciência da Computação. O candidato foi arguido pela Banca Examinadora, composta pelos professores abaixo assinados. Após deliberação, a Banca Examinadora considerou o trabalho APROVADO.

Prof. Dr. Leopoldo Rideki Yoshioka
(USP)

Prof. Dr. Maurício Zadra Pacheco
(UEPG)

Profª. Drª. Simone Nasser Matos
(UTFPR)

Prof. Dr. Max Mauro Dias Santos
(UTFPR) – *Orientadora e presidente da banca*

Universidade Tecnológica Federal do Paraná
Campus Ponta Grossa

Visto da Coordenadora:

Profª. Drª. Sheila Morais de Almeida
Coordenadora do PPGCC
UTFPR – Câmpus Ponta Grossa

Dedico este trabalho à minha vó Maria de Lourdes Brancalioni Pedroso que sempre brilhou os olhos ao falar sobre meus estudos e infelizmente nos deixou no decorrer do desenvolvimento desse trabalho.

AGRADECIMENTOS

Gratidão é a palavra que fica marcada e melhor expressa esses anos de pesquisa e desenvolvimento do trabalho.

Agradeço primeiramente as duas pessoas com que tenho o maior amor possível, meus pais Marcos Ambiel Barros e Benedita Raquel Pedroso Ambiel Barros por sempre terem apoiado minhas escolhas e sempre terem me incentivado a buscar em primeiro lugar o conhecimento.

Agradeço ao meu orientador Prof. Dr. Max Mauro Dias Santos, o qual tem um conhecimento imensurável e um coração gigante, por todo o apoio, amizade e oportunidades oferecidas nesses anos de pesquisa.

Agradeço também a professora Dra. Simone Nasser Matos, que me iniciou na pesquisa ainda na graduação e acompanhou minha trajetória no mestrado como coordenadora do programa de Pós-Graduação em Ciência da Computação, todas orientações e conselhos durante esses anos que trabalhamos juntos.

Agradecimento especial aos meus colegas de trabalho da Renault do Brasil, os quais participaram ativamente no projeto de pesquisa, proporcionando um enorme crescimento profissional e pessoal, sem dúvida foram essenciais para que todos os resultados aqui apresentados fossem possíveis.

Aos meus familiares e amigos que sempre estiveram comigo nessa caminhada, me apoiando e amparando em cada pensamento de desistência que tive ao longo desses anos de pesquisa.

Enfim, a todos os que por algum motivo contribuíram direta e indiretamente para a realização desta pesquisa.

RESUMO

BARROS, Victor Pedroso Ambiel. **Avaliação do desempenho de algoritmos de retropropagação com redes neurais artificiais para a resolução de problemas não-lineares**. 2018. 136 f. Dissertação (Mestrado em Ciência da Computação) - Universidade Tecnológica Federal do Paraná. Ponta Grossa, 2018.

As redes neurais artificiais possibilitam trabalhar com modelagem e resolução de problemas não-lineares, treinando, testando e validando a rede neural com um conjunto de dados na entrada e um objetivo de saída. Porém a construção de uma rede neural artificial é algo complexo e trabalhoso, pois não existe um modelo de rede neural pronto que solucione qualquer problema, cada rede neural deve ser construída com base no problema que se quer solucionar. Um dos principais pontos na construção de uma rede neural é a escolha correta do algoritmo de treinamento para a rede convergir corretamente, produzir bons resultados e solucionar corretamente o problema abordado. Cada algoritmo de treinamento contém seus prós e contras que devem ser levados em consideração. O presente trabalho apresenta a comparação de desempenho entre os algoritmos Levenberg-Marquardt, Bayesian Regularization, Scaled Conjugate Gradient e Resilient Backpropagation aplicado a um estudo de caso não-linear. A aplicação dos algoritmos no estudo de caso foi realizada em três arquiteturas de redes neurais diferentes, possibilitando avaliar em diferentes arquiteturas o desempenho dos algoritmos. O estudo de caso abordado é a aprendizagem do valor da riqueza estequiométrica em motores a combustão interna ciclo Otto, que se caracteriza como um problema não-linear e de alta complexidade. Os resultados obtidos com os treinamentos dos algoritmos nas diferentes arquiteturas mostraram a importância da arquitetura da rede neural utilizada, sendo que uma das três arquiteturas desenvolvida obteve o melhor resultado e dois algoritmos conseguiram atingir ótimas taxas de desempenho, enquanto outros dois algoritmos não obtiveram resultados satisfatórios.

Palavras-chave: Redes Neurais. Problemas Não-Lineares. Avaliação. Desempenho. Algoritmos.

ABSTRACT

BARROS, Victor Pedroso Ambiel. **Evaluation of the performance of backpropagation algorithms with artificial neural networks to solve non-linear problems**. 2018. 136 f. Dissertation (Master in Computer Science) – Federal University of Technology - Paraná. Ponta Grossa, 2018.

Artificial neural networks make possible to work with modeling and resolution of nonlinear problems by training, testing and validating the neural network with a set of input data and an output goal. However, the construction of an artificial neural network is complex and hard-working because there is no neural network model ready to solve any problem, each neural network must be built based on the problem that needs to be solved. One of the main points in the construction of a neural network is the correct choice of the training algorithm for the network to converge correctly, produce good results and correctly solve the problem addressed. Each training algorithm contains its pros and cons that should be taken into consideration. The present work presents the performance comparison between the Levenberg-Marquardt, Bayesian Regularization, Scaled Conjugate Gradient and Resilient Backpropagation algorithms applied to a non-linear case study. The application of the algorithms in the case study was carried out in three different neural network architectures, allowing to evaluate the performance of the algorithms in different architectures. The case study is the learning of the value of the stoichiometric richness in spark ignition engine, which is characterized as a non-linear and high complexity problem. The results obtained with the training of the algorithms in the different architectures showed the importance of the architecture of the neural network used, being that one of the three architectures developed the best result and two algorithms were able to achieve excellent performance rates, whereas other two algorithms did not obtain satisfactory results.

Keywords: Neural Networks. Non-Linear Problems. Evaluation. Performance. Algorithms.

LISTA DE FIGURAS

Figura 1 – Representação do neurônio biológico e o neurônio artificial 18
Figura 2 - Modelo de neurônio apresentado por McCulloch e Pitts 19
Figura 3 - Funções de ativação 20
Figura 4 - Rede neural de camada única 21
Figura 5 - Rede neural de múltiplas camadas 21
Figura 6 - Rede neural de múltiplas camadas com realimentação 22
Figura 7 - Representação por Diagrama da Aprendizagem Supervisionada 23
Figura 8 - Representação do processo de Classificação e Regressão 24
Figura 9 - Representação por Diagrama da Aprendizagem Não-Supervisionada 25
Figura 10 - Arquitetura da Rede Neural Artificial com uma camada oculta e sete neurônios 49
Figura 11 – Os tempos de funcionamento do Motor a Combustão Interna 51
Figura 12 - Motor Renault HR16 1.6 16V SCe 55
Figura 13 - Arquitetura da RNA com 24 neurônios na camada oculta 58
Figura 14 - Arquitetura da RNA com 48 neurônios na camada oculta 59
Figura 15 - Arquitetura da RNA com 64 neurônios na camada oculta 60
Figura 16 - Etapas para realização dos treinamentos da rede neural 61
Figura 17 - Fluxograma da aquisição dos dados do veículo 62
Figura 18 - Fluxograma do treinamento da RNA 63
Figura 19 - Fluxograma para validação da RNA 65
Figura 20 - Interação entre a ferramenta desenvolvida com o MATLAB e a Neural Network Toolbox 123
Figura 21 - Interface de Treinamento 124
Figura 22 - Seleção dos dados para treinamento 125
Figura 23 - Opção para abrir a ferramenta de manipulação dos dados 126
Figura 24 - Interface *Configuration Data* 128
Figura 25 - Opção para abrir a ferramenta de configuração do objetivo de treinamento da rede neural 129
Figura 26 - Configuração dos parâmetros da Rede Neural Artificial 129
Figura 27 - Resultados do treinamento da RNA 130
Figura 28 - Seção para visualização de gráficos 131
Figura 29 - Interface de Adaptação de RNA 132
Figura 30 - Configuração da RNA para Adaptação 133
Figura 31 - Interface de Validação 134
Figura 32 - Configuração da Validação 134
Figura 33 - Interface para Controle da RNA 135

LISTA DE GRÁFICOS

Gráfico 1 - Valor Real (Experiment) e o Valor Produzido pela Rede Neural (Neural network) no Treinamento....45
Gráfico 2 - Valor Real (Experiment) e o Valor Produzido pela Rede Neural (Neural network) em Dados de Teste....46
Gráfico 3 - Divisão entre os sinais no treinamento da rede neural....47
Gráfico 4 - Taxa de Regressão para o LM com 24 neurônios....67
Gráfico 5 - Valor real (laranja) e o valor do treinamento (azul) no algoritmo LM com 24 neurônios....68
Gráfico 6 - Valor real (laranja) e o valor simulado (azul) no algoritmo LM com 24 neurônios....69
Gráfico 7 - Taxa de Regressão para o LM com 48 neurônios....71
Gráfico 8 - Valor real (laranja) e o valor do treinamento (azul) no algoritmo LM com 48 neurônios....72
Gráfico 9 - Valor real (laranja) e o valor da simulação (azul) no algoritmo LM com 48 neurônios....73
Gráfico 10 - Taxa de Regressão para o LM com 64 neurônios....74
Gráfico 11 - Valor real (laranja) e o valor do treinamento (azul) no algoritmo LM com 64 neurônios....75
Gráfico 12 - Valor real (laranja) e o valor da simulação (azul) no algoritmo LM com 64 neurônios....76
Gráfico 13 - Taxa de Regressão para o algoritmo BR com 24 neurônios....77
Gráfico 14 - Valor real (laranja) e o valor do treinamento (azul) no algoritmo BR com 24 neurônios....78
Gráfico 15 - Valor real (laranja) e o valor da simulação (azul) no algoritmo BR com 24 neurônios....79
Gráfico 16 - Taxa de Regressão para o algoritmo BR com 48 neurônios....81
Gráfico 17 - Valor real (laranja) e o valor do treinamento (azul) no algoritmo BR com 48 neurônios....82
Gráfico 18 - Valor real (laranja) e o valor da simulação (azul) no algoritmo BR com 48 neurônios....83
Gráfico 19 - Taxa de Regressão para o algoritmo BR com 64 neurônios....84
Gráfico 20 - Valor real (laranja) e o valor do treinamento (azul) no algoritmo BR com 64 neurônios....85
Gráfico 21 - Valor real (laranja) e o valor da simulação (azul) no algoritmo BR com 64 neurônios....86
Gráfico 22 - Taxa de Regressão para o algoritmo SCG com 24 neurônios....87
Gráfico 23 - Valor real (laranja) e o valor do treinamento (azul) no algoritmo SCG com 24 neurônios....88
Gráfico 24 - Valor real (laranja) e o valor da simulação (azul) no algoritmo SCG com 24 neurônios....89
Gráfico 25 - Taxa de Regressão para o algoritmo SCG com 48 neurônios....91
Gráfico 26 - Valor real (laranja) e o valor do treinamento (azul) no algoritmo SCG com 48 neurônios....92
Gráfico 27 - Valor real (laranja) e o valor da simulação (azul) no algoritmo SCG com 48 neurônios....93
Gráfico 28 - Taxa de Regressão para o algoritmo SCG com 64 neurônios....94

Gráfico 29 - Valor real (laranja) e o valor do treinamento (azul) no algoritmo SCG com 64 neurônios.....95
Gráfico 30 - Valor real (laranja) e o valor da simulação (azul) no algoritmo SCG com 64 neurônios.....96
Gráfico 31 - Taxa de Regressão para o algoritmo Rprop com 24 neurônios.....97
Gráfico 32 - Valor real (laranja) e o valor do treinamento (azul) no algoritmo Rprop com 24 neurônios.....98
Gráfico 33 - Valor real (laranja) e o valor da simulação (azul) no algoritmo Rprop com 24 neurônios.....99
Gráfico 34 - Taxa de Regressão para o algoritmo Rprop com 48 neurônios.....100
Gráfico 35 - Valor real (laranja) e o valor do treinamento (azul) no algoritmo Rprop com 48 neurônios.....101
Gráfico 36 - Valor real (laranja) e o valor da simulação (azul) no algoritmo Rprop com 48 neurônios.....102
Gráfico 37 - Taxa de Regressão para o algoritmo Rprop com 64 neurônios.....103
Gráfico 38 - Valor real (laranja) e o valor do treinamento (azul) no algoritmo Rprop com 64 neurônios.....104
Gráfico 39 - Valor real (laranja) e o valor da simulação (azul) no algoritmo Rprop com 64 neurônios.....105

LISTA DE TABELAS

Tabela 1 - Indicação de performance do treinamento e validação da rede neural 48
Tabela 2 - Relação estequiométrica ar-combustível 53
Tabela 3 - Informações sobre o treinamento com 24 neurônios 66
Tabela 4 - Taxas de erro e taxa de acerto do algoritmo LM no treinamento com 24 neurônios.....67
Tabela 5 - Taxas de erro e taxa de acerto do algoritmo LM na simulação com 24 neurônios.....69
Tabela 6 - Informações sobre o treinamento com 48 neurônios 70
Tabela 7 - Taxas de erro e taxa de acerto do algoritmo LM no treinamento com 48 neurônios.....71
Tabela 8 - Taxas de erro e taxa de acerto do algoritmo LM na simulação com 48 neurônios.....72
Tabela 9 - Informações sobre o treinamento com 64 neurônios 74
Tabela 10 - Taxas de erro e taxa de acerto do algoritmo LM no treinamento com 64 neurônios.....75
Tabela 11 - Taxas de erro e taxa de acerto do algoritmo LM na simulação com 64 neurônios.....76
Tabela 12 - Informações sobre o treinamento com 24 neurônios 77
Tabela 13 - Taxas de erro e taxa de acerto do algoritmo BR no treinamento com 24 neurônios.....78
Tabela 14 - Taxas de erro e taxa de acerto do algoritmo BR na simulação com 24 neurônios.....79
Tabela 15 - Informações sobre o treinamento com 48 neurônios 80
Tabela 16 - Taxas de erro e taxa de acerto do algoritmo BR no treinamento com 48 neurônios.....81
Tabela 17 - Taxas de erro e taxa de acerto do algoritmo BR na simulação com 48 neurônios.....82
Tabela 18 - Informações sobre o treinamento com 64 neurônios 83
Tabela 19 - Taxas de erro e taxa de acerto do algoritmo BR no treinamento com 64 neurônios.....84
Tabela 20 - Taxas de erro e taxa de acerto do algoritmo BR na simulação com 64 neurônios.....85
Tabela 21 - Informações sobre o treinamento com 24 neurônios 87
Tabela 22 - Taxas de erro e taxa de acerto do algoritmo SCG no treinamento com 24 neurônios.....88
Tabela 23 - Taxas de erro e taxa de acerto do algoritmo SCG na simulação com 24 neurônios.....89
Tabela 24 - Informações sobre o treinamento com 48 neurônios 90
Tabela 25 - Taxas de erro e taxa de acerto do algoritmo SCG no treinamento com 48 neurônios.....91
Tabela 26 - Taxas de erro e taxa de acerto do algoritmo SCG na simulação com 48 neurônios.....92
Tabela 27 - Informações sobre o treinamento com 64 neurônios 93
Tabela 28 - Taxas de erro e taxa de acerto do algoritmo SCG no treinamento com 64 neurônios.....94
Tabela 29 - Taxas de erro e taxa de acerto do algoritmo SCG na simulação com 64 neurônios.....95

Tabela 30 - Informações sobre o treinamento com 24 neurônios 97
Tabela 31 - Taxas de erro e taxa de acerto do algoritmo Rprop no treinamento com 24 neurônios.....98
Tabela 32 - Taxas de erro e taxa de acerto do algoritmo Rprop na simulação com 24 neurônios.....99
Tabela 33 - Informações sobre o treinamento com 48 neurônios 100
Tabela 34 - Taxas de erro e taxa de acerto do algoritmo Rprop no treinamento com 48 neurônios.....101
Tabela 35 - Taxas de erro e taxa de acerto do algoritmo Rprop na simulação com 48 neurônios.....102
Tabela 36 - Informações sobre o treinamento com 64 neurônios 103
Tabela 37 - Taxas de erro e taxa de acerto do algoritmo Rprop no treinamento com 64 neurônios.....104
Tabela 38 - Taxas de erro e taxa de acerto do algoritmo Rprop na simulação com 64 neurônios.....105
Tabela 39 – Pontuação para as Taxas de Acerto dos Algoritmos na Simulação 106
Tabela 40 - Pontuação para as Taxas de Acerto dos Algoritmos no Treinamento..107
Tabela 41 - Pontuação para as taxas de Regressão 107
Tabela 42 - Pontuação para as taxas de MSE 108
Tabela 43 - Pontuação para os Tempos de Treinamento 108
Tabela 44 - Informações sobre o treinamento dos algoritmos com a primeira arquitetura (24 neurônios) 110
Tabela 45 - Informações sobre o treinamento dos algoritmos com a segunda arquitetura (48 neurônios) 111
Tabela 46 - Informações sobre o treinamento dos algoritmos com a segunda arquitetura (64 neurônios) 113
Tabela 47 - Pontuação final do desempenho de cada algoritmo em ordem decrescente.....115

SUMÁRIO

1 INTRODUÇÃO 14
1.1 OBJETIVOS 15
1.2 ORGANIZAÇÃO DO TRABALHO 16
2 FUNDAMENTAÇÃO TEÓRICA 17
2.1 REDES NEURAIS ARTIFICIAIS 17
2.1.1 Modelo de Neurônio Artificial 19
2.1.2 Arquiteturas de Redes Neurais 20
2.1.3 Aprendizagem Supervisionada 22
2.1.4 Aprendizagem Não-Supervisionada 24
2.2 PERCEPTRON DE MÚLTIPLAS CAMADAS 25
2.2.1 Algoritmo de Treinamento por Retropropagação (*Backpropagation*) 27
2.2.2 Treinamento Local e em Lote 30
2.2.3 Critérios de Parada 32
2.3 MÉTODOS DE AVALIAÇÃO DE ALGORITMOS COM REDES NEURAIS ARTIFICIAIS 33
2.4 LEVENBERG-MARQUARDT BACKPROPAGATION 34
2.5 BAYESIAN REGULARIZATION BACKPROPAGATION 37
2.6 SCALED CONJUGATE GRADIENT BACKPROPAGATION 39
2.7 RESILIENT BACKPROPAGATION 40
3 TRABALHOS RELACIONADOS 43
3.1 COMPARAÇÃO DE TRÊS ALGORITMOS DE TREINAMENTO DE RETRO-PROPAGAÇÃO PARA DOIS ESTUDOS DE CASO 43
3.2 APLICAÇÃO DE REDES NEURAIS ARTIFICIAIS PARA PREVISÃO DE CONSUMO ESPECÍFICO DE COMBUSTÍVEL E TEMPERATURA DE ESCAPE PARA UM MOTOR DIESEL 44
3.3 AS APLICAÇÕES DE REDES NEURAIS ARTIFICIAIS EM MOTORES 46
3.4 PREVISÃO DO DESEMPENHO DO MOTOR DE GASOLINA COM REDE NEURAL ARTIFICIAL 48
4 ESTUDO DE CASO 50
4.1 OS QUATRO TEMPOS DO MOTOR 50
4.2 FORMAÇÃO DA MISTURA AR COMBUSTÍVEL 52
4.3 ESTUDO DE CASO: MOTOR A COMBUSTÃO INTERNA CICLO OTTO 54
5 RESULTADOS EXPERIMENTAIS 56
5.1 ARQUITETURA DESENVOLVIDA PARA A REDE NEURAL ARTIFICIAL 56
5.2 PROCESSO PARA A REALIZAÇÃO DOS TREINAMENTOS 61
5.3 DESEMPENHO DO ALGORITMO LEVENBERG-MARQUARDT NO ESTUDO DE CASO 65
5.3.1 Levenberg-Marquardt com 24 Neurônios na Camada Oculta 66

5.3.2 Levenberg-Marquardt com 48 Neurônios na Camada Oculta 70
5.3.3 Levenberg-Marquardt com 64 Neurônios na Camada Oculta 73
5.4 DESEMPENHO DO ALGORITMO BAYESIAN REGULARIZATION NO ESTUDO DE CASO 76
5.4.1 Bayesian Regularization com 24 Neurônios na Camada Oculta 77
5.4.2 Bayesian Regularization com 48 Neurônios na Camada Oculta 80
5.4.3 Bayesian Regularization com 64 Neurônios na Camada Oculta 83
5.5 DESEMPENHO DO ALGORITMO SCALED CONJUGATE GRADIENT NO ESTUDO DE CASO 86
5.5.1 Scaled Conjugate Gradient com 24 Neurônios na Camada Oculta..... 86
5.5.2 Scaled Conjugate Gradient com 48 Neurônios na Camada Oculta..... 89
5.5.3 Scaled Conjugate Gradient com 64 Neurônios na Camada Oculta..... 93
5.6 DESEMPENHO DO ALGORITMO RESILIENT BACKPROPAGATION NO ESTUDO DE CASO 96
5.6.1 Resilient Backpropagation com 24 Neurônios na Camada Oculta 96
5.6.2 Resilient Backpropagation com 48 Neurônios na Camada Oculta 100
5.6.3 Resilient Backpropagation com 64 Neurônios na Camada Oculta 102
5.7 CRITÉRIOS DE COMPARAÇÃO DO DESEMPENHO ENTRE OS ALGORITMOS 105
5.8 COMPARAÇÃO DO DESEMPENHO ENTRE OS ALGORITMOS..... 109
6 CONCLUSÃO 117
6.1 TRABALHOS FUTUROS 118
REFERÊNCIAS..... 119
APÊNDICE A – Ferramenta Desenvolvida para o Treinamento das Redes Neurais Artificiais..... 122

1 INTRODUÇÃO

As redes neurais artificiais são técnicas computacionais que visam imitar o funcionamento dos neurônios de organismos inteligentes através de modelos matemáticos. Com os modelos, é possível treinar, testar e validar a rede neural artificial e utilizar a rede treinada para simulações, reconhecimento de padrões, previsão, entre outras aplicações (HAYKIN, 1994).

Existe uma grande gama de problemas que podem ser estudados e resolvidos com as redes neurais artificiais, porém não existe um modelo pronto de rede que se encaixe na resolução de todos os problemas, cada rede neural é construída com base no problema que se deseja resolver, o que em muitos casos torna a construção da rede neural um processo complexo que requer tempo e experiência para a construção de uma rede que consiga atingir bons resultados. Na construção de uma rede neural artificial são muitas as configurações possíveis e que podem variar de problema para problema, tais como: variáveis de entrada, se a rede vai ter ou não camadas ocultas, se tiver camadas ocultas qual é quantidade de camadas ocultas, a quantidade de neurônios em cada camada oculta, o bias, funções de ativação, realimentação da rede, entre outros diversos parâmetros.

Um dos critérios essenciais na construção de uma rede neural artificial é a escolha do algoritmo de treinamento. Existem diversos algoritmos de treinamento que se encaixam nas diversas categorias de problemas, utilizar o algoritmo correto para a categoria de problemas faz com que a rede consiga solucionar o problema desejado e seja confiável. Na mesma categoria de problemas, existem diversos algoritmos de treinamento, cada um com seus prós e contras.

O presente trabalho apresenta um estudo comparativo entre quatro algoritmos para o treinamento supervisionado de redes neurais aplicado a um problema não-linear, com o objetivo de avaliar o desempenho de cada algoritmo. Para avaliar os algoritmos foram criados critérios de avaliação que pontuam os desempenhos dos algoritmos, possibilitando classificar os algoritmos em suas ordens de pontuação de visualizar qual algoritmo obteve melhor desempenho no estudo de caso aplicado. O trabalho também contempla a construção de três arquiteturas de redes neurais para o treinamento dos quatro algoritmos, além de a construção de uma ferramenta que

facilita a manipulação dos dados para treinamento e a apresentação de um processo para a realização dos treinamentos no estudo de caso.

O estudo de caso utilizado é um motor a combustão interna ciclo Otto, com o objetivo de treinar a rede neural para aprender a formação do valor da riqueza estequiométrica. Todo o processo de funcionamento do motor depende da melhor proporção possível da mistura de ar e combustível, pois a falta ou excesso de qualquer um dos dois elementos químicos causa o mal funcionamento do motor, além de emitir mais poluentes e elevar o consumo de combustível. A condição de mistura ideal entre o ar e combustível é chamada de mistura estequiométrica (PUJATTI, 2007) e denominada valor de riqueza. Manter a condição da riqueza próxima do ideal é uma tarefa complexa na calibração dos motores a combustão interna ciclo Otto, caracterizando-se como um problema não-linear e de alta complexidade, ideal para ser utilizado como estudo de caso.

1.1 OBJETIVOS

O objetivo geral deste trabalho é realizar a avaliação do desempenho dos algoritmos Levenberg-Marquardt, Bayesian Regularization, Scaled Conjugated Gradient e Resilient Backpropgation na resolução de problemas não-lineares, definindo critérios de avaliação e pontuando os algoritmos por cada critério, podendo assim classificar os algoritmos pelo seu desempenho no estudo de caso aplicado, mostrando quais os pontos fortes e fracos de cada algoritmo.

Os objetivos específicos do trabalho são:

- Buscar na literatura trabalhos relacionados com o tema da pesquisa.
- Identificar os elementos que influenciam no valor da riqueza estequiométrica, com o objetivo de aplicar corretamente os algoritmos no estudo de caso.
- Desenvolver as arquiteturas de redes neurais artificiais para treinar os algoritmos com os dados do estudo de caso.
- Desenvolver um processo de treinamento da rede neural artificial.
- Aplicar os algoritmos com as arquiteturas desenvolvidos no estudo de caso.
- Definir critérios de comparação do desempenho entre os algoritmos.

- Realizar a comparação do desempenho entre os diferentes algoritmos com as diferentes arquiteturas.

1.2 ORGANIZAÇÃO DO TRABALHO

O presente trabalho está organizado em 6 capítulos. O capítulo 2 apresenta as referências bibliográficas utilizadas para o desenvolvimento do projeto. Dentro desse capítulo, foi abordado o conceito sobre redes neurais artificiais, mostrando seu funcionamento, as arquiteturas de redes neurais, tipos de treinamento, algoritmos de treinamento, entre outros conceitos que abrangem o tema de redes neurais artificiais.

O capítulo 3 apresenta os trabalhos relacionados que motivam a dissertação.

O capítulo 4 descreve o estudo de caso da dissertação, abordando o conceito de motores a combustão interna e da mistura ar-combustível, que é o estudo de caso da dissertação, e também apresenta qual o motor utilizado e suas características.

O capítulo 5 apresenta os resultados obtidos com os algoritmos aplicado no estudo de caso, mostrando as arquiteturas de redes neurais desenvolvidas, qual o processo para a realização do treinamento das redes neurais, os critérios de comparação entre os algoritmos e a comparação do desempenho de cada algoritmo.

O capítulo 6 faz a conclusão do trabalho, apresentando quais os ganhos e possíveis trabalhos futuros.

2 FUNDAMENTAÇÃO TEÓRICA

Este capítulo apresenta uma visão geral sobre os temas que são relevantes para a construção da dissertação, tais como redes neurais artificiais e motores a combustão interna.

A seção 2.1 apresenta o conceito sobre redes neurais artificiais, mostrando seu surgimento, quais os modelos, principais arquiteturas e tipos de aprendizagem. A seção 2.2 apresenta mais profundamente a arquitetura de rede neural utilizada no trabalho, juntamente com um algoritmo de treinamento padrão, as formas de treinamento e critérios de parada do algoritmo. Da seção 2.3 até a seção 2.6 são apresentados a fundamentação teórica e o funcionamento dos quatro algoritmos que serão analisados, o Levenberg-Marquardt Backpropagation, Bayesian Regularization Backpropagation, Scaled Conjugate Gradient Backpropagation e o Resilient Backpropagation.

2.1 REDES NEURAIS ARTIFICIAIS

O cérebro humano possui uma capacidade incrível de absorção de conhecimento para desempenhar funções como percepção, intuição, inferência, reconhecimento de padrões, controle motor, entre outros.

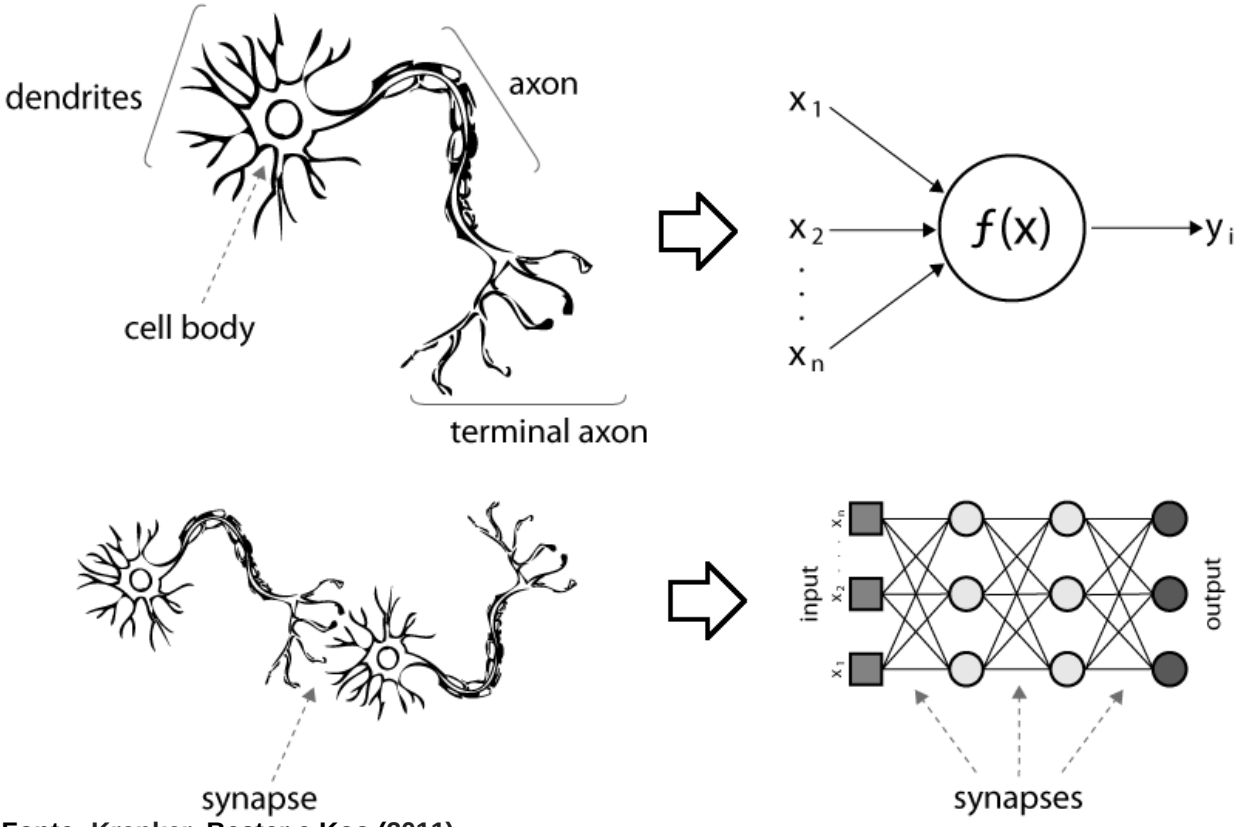
Qualquer que seja o organismo multicelular, independente da sua complexidade e organização, possui algum tipo de sistema nervoso que é responsável por alimentar o organismo através de entradas sensoriais sobre informações do ambiente em que o organismo vive. As informações coletadas são processadas e comparadas com experiências passadas e posteriormente são transformadas em ações apropriadas para a situação ou absorvidas em forma de conhecimento (HAYKIN, 1994).

O sistema nervoso humano é extremamente complexo, formado por um conjunto de células neurais, que desempenham um papel essencial no funcionamento do corpo humano e do raciocínio. Os neurônios podem receber e enviar sinais a vários outros neurônios e se comunicam através de sinapses, que é a região onde dois

neurônios entram em contato e é através de sinapses que os impulsos nervosos são transmitidos.

As Redes Neurais Artificiais (RNA) se inspiram no funcionamento da rede neural desses organismos inteligentes, sendo que as RNA são técnicas computacionais construídas a partir de modelos matemáticos que buscam adquirir conhecimento através de experiências, assim como uma rede neural biológica. Mesmo tentando imitar um sistema neural, a RNA não consegue o mesmo desempenho que um cérebro humano, pois enquanto uma RNA consegue ter milhares de unidades de processamento, o cérebro consegue ter bilhões de neurônios (HAYKIN, 1994). A Figura 1 faz uma representação entre o neurônio biológico e o neurônio artificial, sendo que o neurônio artificial busca ter as mesmas características do biológico, como as sinapses de ligação, o corpo do neurônio, a axioma terminal, entre outras características.

Figura 1 – Representação do neurônio biológico e o neurônio artificial



Fonte: Krenker, Bester e Kos (2011)

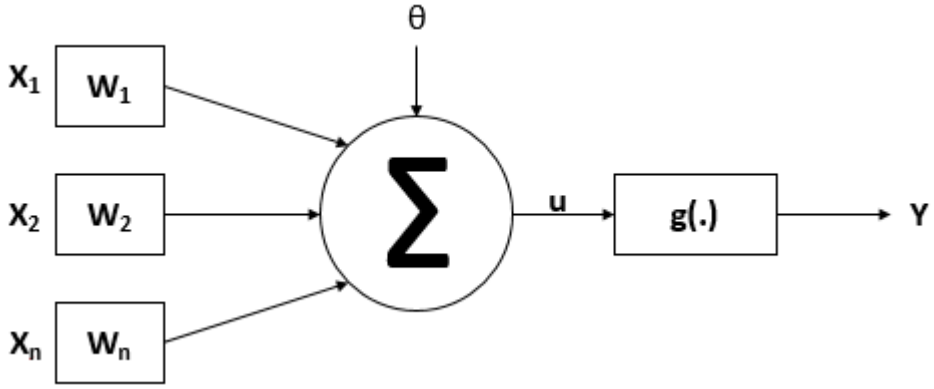
Existem muitas vertentes sobre o surgimento das Redes Neurais Artificiais. O primeiro registro da construção de uma máquina construída sendo inspirada no cérebro humano foi em 1943, por Warren McCulloch e Walter Pitts (MCCULLOCH,

PITTS, 1943), que desenvolveram um modelo matemático de neurônio biológico artificial inspirado no cérebro humano, porém o modelo desenvolvido não conseguiu desempenhar a tarefa de aprendizado. Somente 15 anos mais tarde, em 1958, que houve o primeiro desenvolvimento bem-sucedido de um neurocomputador. O desenvolvimento foi realizado por Frank Rosenblatt (ROSENBLATT, 1958), conhecido como os fundadores da neurocomputação, pois seu trabalho sustentou modelos que são utilizados atualmente, o *Perceptron* (redes neurais de um nível) e o MLP (*perceptron* de múltiplas camadas).

2.1.1 Modelo de Neurônio Artificial

A base de um projeto de redes neurais artificiais é o seu modelo de neurônio, sendo que um neurônio é uma unidade de processamento de informações. Uma rede neural artificial é composta por várias unidades de processamento. O modelo de neurônio apresentado será o de McCulloch e Pitts, ilustrado na Figura 2.

Figura 2 - Modelo de neurônio apresentado por McCulloch e Pitts



Fonte: Haykin (1994)

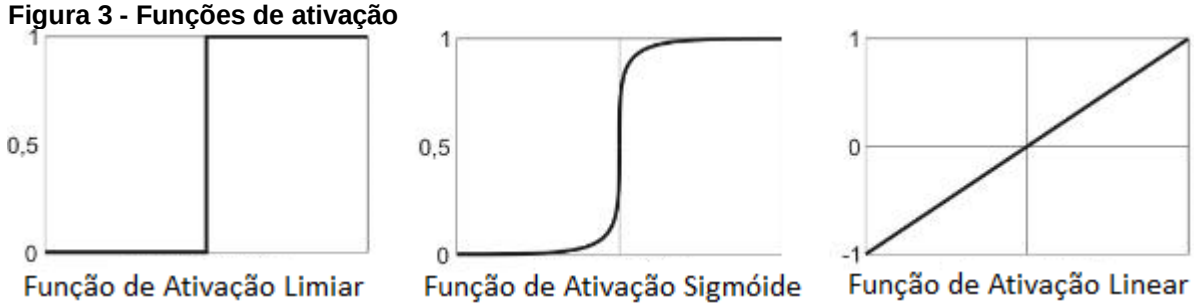
O modelo é composto por três partes principais, as entradas (ou sinais de entrada, sinapses, conexões), o somador e a função de ativação.

Cada sinal de entrada é composto por um índice e um peso, sendo que cada sinal de entrada é multiplicado pelo seu peso e indica a influência da entrada no valor da saída. A soma dos pesos de cada entrada não indica nenhuma característica do modelo, os pesos são variados conforme cada entrada e diferentemente de uma sinapse do cérebro, podem ser tanto positivos quanto negativos.

O somador tem a função de somar ponderadamente todas as entradas multiplicadas pelos seus respectivos pesos e produzir uma entrada para a função de ativação.

A função de ativação tem como objetivo determinar um limite mínimo e máximo para a amplitude da saída do neurônio, para tal, é determinado um intervalo de ativação, descrito como intervalo unitário fechado, variando de 0 a 1 ou de -1 a 1, e posteriormente a função de ativação produz uma resposta de saída. Outra característica da função de ativação é como ela é ativada, podendo ser uma função limiar (degrau), linear por partes ou sigmoide, como ilustrado na Figura 3.

Figura 3 - Funções de ativação



Fonte: Haykin (1994)

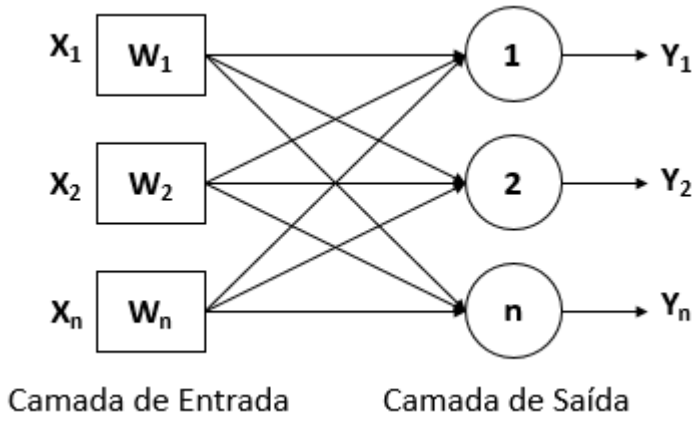
O tipo de função de ativação a ser utilizada é definido conforme a finalidade da construção da rede neural artificial. Permitir que a função de ativação assuma valores negativos pode trazer benefícios em alguns projetos e não trazer benefícios em outros.

2.1.2 Arquiteturas de Redes Neurais

Para a construção de uma rede neural artificial, existem três arquiteturas fundamentais de como a rede pode ser estruturada. Duas dessas arquiteturas são em camadas, sendo assim, os neurônios estão organizados em camadas.

A primeira arquitetura é a Rede Neural de Camada Única. Nessa arquitetura a camada de entrada está diretamente associada a um ou mais neurônios da camada de saída. Essa ligação é direcional e acíclica, sendo que somente a camada de entrada aponta para a camada de saída, o contrário nunca ocorre. A Figura 4 ilustra a arquitetura de uma rede neural de camada única.

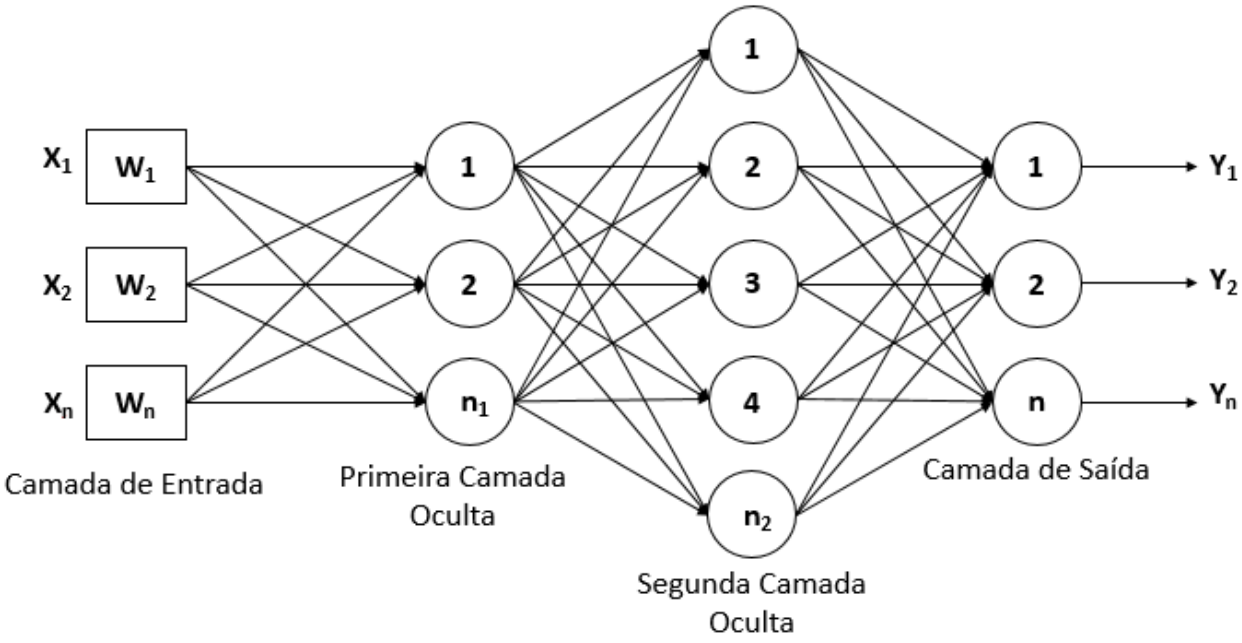
Figura 4 - Rede neural de camada única



Fonte: Haykin (1994)

A segunda arquitetura é a rede neural de múltiplas camadas, que diferentemente da rede neural de camada única, a arquitetura contém uma ou mais camadas ocultas, que também são chamadas de neurônios ocultos ou unidades ocultas. A motivação de da utilização de camadas ocultas se dá pela possibilidade da rede extrair estatísticas de ordem elevada, podendo assim a rede adquirir uma perspectiva global devido ao conjunto extra de camadas ocultas que formam novas conexões sinápticas e da dimensão extra de interações neurais (HAYKIN, 1994). A Figura 5 ilustra uma rede neural de múltiplas camadas.

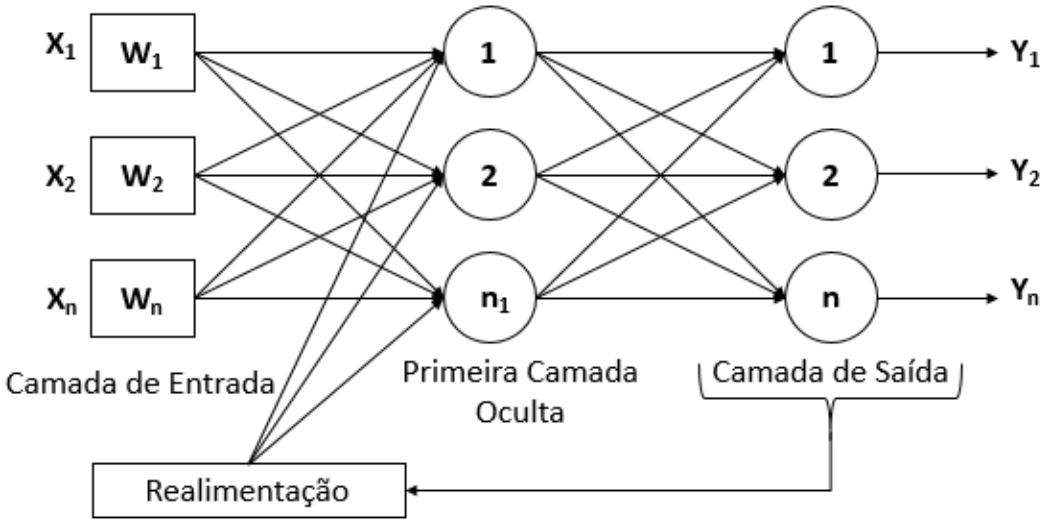
Figura 5 - Rede neural de múltiplas camadas



Fonte: Haykin (1994)

A terceira arquitetura é a de rede neural com realimentação, ilustrada na Figura 6. O conceito dessa arquitetura é conter pelo menos um laço de realimentação, podendo ela ser uma rede de camada única ou de múltiplas camadas, desde que tenha uma realimentação na entrada com dados de saída produzidos pela própria rede neural.

Figura 6 - Rede neural de múltiplas camadas com realimentação



Fonte: Haykin (1994)

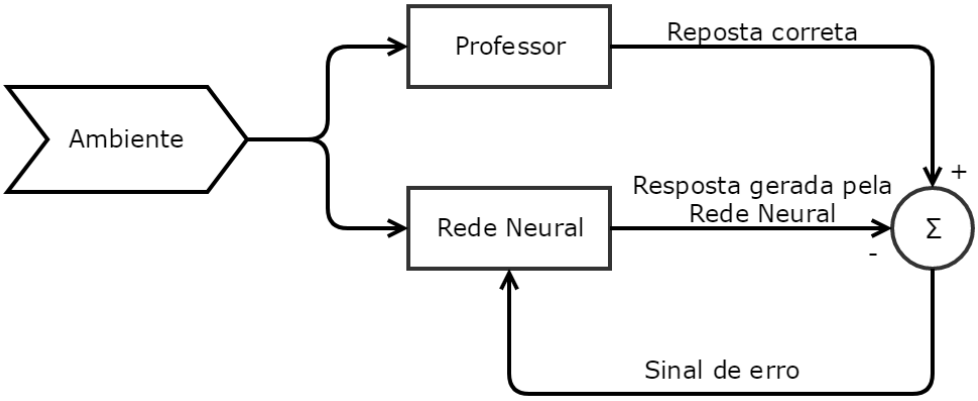
A utilização da realimentação na arquitetura da rede neural tem um grande impacto na aprendizagem da rede e no seu desempenho, além de resultar em um comportamento dinâmico não-linear, devido a utilização dos ramos particulares compostos de elementos de atraso unitário.

2.1.3 Aprendizagem Supervisionada

A aprendizagem supervisionada, também conhecida como aprendizagem com um professor, supõe a existência de um “professor” que conhece previamente o ambiente em que a rede será treinada e ensina o comportamento que deve ser adotado em cada situação, através de um conjunto de exemplos de entradas e saídas. Sendo assim, para cada entrada, existe uma saída que é a resposta correta desejada, informada pelo “professor”. A diferença entre a resposta produzida pela rede e a saída informada pelo professor é caracterizada como *sinal de erro*. O sinal de erro é ajustado passo a passo em cada iteração do treinamento, com o objetivo de transferir o

conhecimento do professor para a rede neural, minimizando o máximo possível o valor do erro até que a rede esteja autônoma o suficiente para continuar a aprender com o ambiente por si própria (HAYKIN, 1994). A Figura 7 apresenta através de um diagrama a representação do funcionamento do processo da aprendizagem supervisionada.

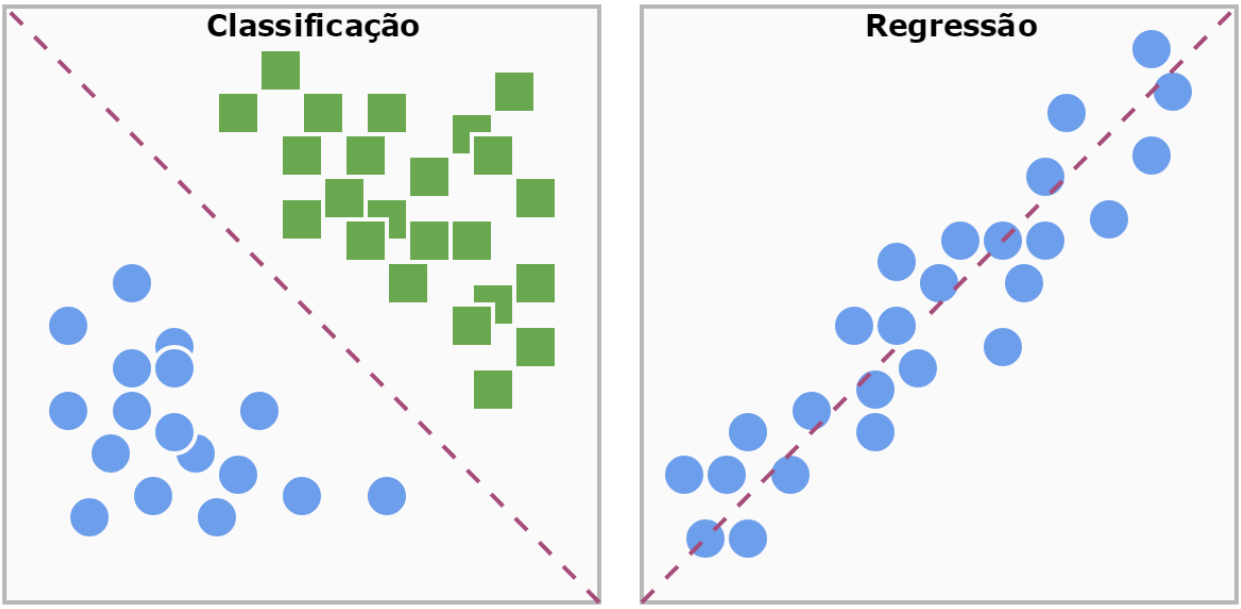
Figura 7 - Representação por Diagrama da Aprendizagem Supervisionada



Fonte: Autoria própria (HAYKIN, 1994)

Os problemas que abrangem o aprendizado supervisionado são classificados como problemas de regressão e classificação, ilustrado na Figura 8. Os problemas de regressão têm como objetivo gerar uma saída que tenta prever resultados para valores contínuos, ou seja, mapear as variáveis de entrada para uma função contínua. Exemplos clássicos da regressão são, com base em um conjunto de dados, tentar prever qual será o índice de inflação de um mês, ou tentar prever qual será a quantidade de produtos vendidos por uma determinada loja em um período de tempo.

Figura 8 - Representação do processo de Classificação e Regressão



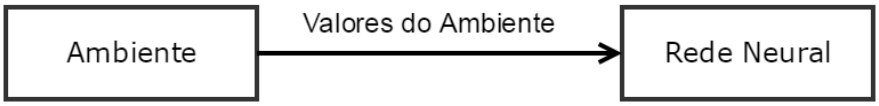
Fonte: Autoria própria

Os problemas de classificação tentam prever o resultado para uma saída com valores discretos. Exemplos clássicos para esta categoria de problemas são, dado um conjunto de dados sobre um tumor cancerígeno, tentar prever se ele é benigno ou maligno, através de algumas variáveis, como o tamanho do tumor e a idade do paciente. Outro exemplo é o mapeamento de uma imagem de uma pessoa para classificar qual o sexo, masculino ou feminino.

2.1.4 Aprendizagem Não-Supervisionada

Na aprendizagem não-supervisionada, aprendizagem sem professor ou aprendizagem auto-organizada, não existe o papel do “professor” que conhece previamente o ambiente em que a rede neural será treinada, sendo assim, ao apresentar os dados para a rede, ela tem que descobrir sozinha os padrões, relações, categorias ou regularidades que existem e produzir uma saída com o seu resultado. A Figura 9 ilustra a interação entre o ambiente e a rede neural, onde o ambiente interage com a rede neural e ela tenta aprender sozinha como o ambiente funciona, encontrando seus padrões e relações.

Figura 9 - Representação por Diagrama da Aprendizagem Não-Supervisionada



Fonte: Autoria própria (HAYKIN, 1994)

O processo de aprendizagem não-supervisionada possibilita a abordagem de problemas com pouca ou nenhuma informação, podendo encontrar padrões e informações nos dados onde dificilmente conseguiríamos manualmente. Um exemplo da utilização da aprendizagem não-supervisionada é dado uma coleção de 1000 pesquisas de uma universidade, encontrar uma maneira de agrupar automaticamente estas pesquisas em grupos que são de alguma forma semelhantes ou relacionadas por diferentes variáveis, tais como a frequência das palavras, frases, contagem de páginas, entre outros.

Existem diferentes algoritmos de aprendizagem utilizados na aprendizagem não-supervisionada, entre os mais conhecidos temos a aprendizagem de Hebbian, o aprendizado competitivo, aprendizado *winner-take-all* entre outros.

Outro paradigma da aprendizagem não-supervisionada é a aprendizagem por reforço. Nessa aprendizagem é realizada um mapeamento de entrada e saída através de uma iteração contínua com o ambiente, com o objetivo de minimizar um índice escalar de desempenho (HAYKIN, 1994). No aprendizado por reforço, o algoritmo escolhe uma ação em resposta a cada ponto de dados. O algoritmo de aprendizado também recebe um sinal de recompensa pouco tempo depois, indicando se a decisão foi boa. Com base nisso, o algoritmo modifica sua estratégia para alcançar a recompensa mais alta.

2.2 PERCEPTRON DE MÚLTIPLAS CAMADAS

A arquitetura do Perceptron de Múltiplas Camadas (*Multilayer Perceptron* - MLP) é uma das principais, mais utilizada e importante classe de redes neurais. A arquitetura da MLP consiste de três conjuntos principais. O primeiro conjunto são as unidades sensoriais que constituem a camada de entrada. O segundo conjunto é a camada ou as camadas ocultas (pode-se ter uma ou várias camadas, depende da finalidade da aplicação), também conhecidas como camadas intermediárias ou

escondidas. O terceiro conjunto é a camada de saída. Os sinais de entrada são propagados camada a camada pela rede em uma direção positiva, ou seja, da entrada, passando pela camada oculta até a camada de saída.

As redes MLP são utilizadas com sucesso para a solução de vários problemas envolvendo altos graus de não-linearidade. Seu treinamento é supervisionado e utiliza um algoritmo muito popular chamado retropropagação do erro (*error backpropagation*). Este algoritmo é baseado numa regra de aprendizagem que "corrige" o erro durante o treinamento (HAYKIN, 1994).

O processo de retropropagação do erro é constituído de duas fases: uma fase de propagação do sinal funcional (*feedforward*) e uma de retropropagação do erro (*backpropagation*). Na fase *feedforward*, os vetores de dados são aplicados às unidades de entrada, e seu efeito se propaga pela rede, camada a camada. Finalmente, um conjunto de saídas é produzido como resposta da rede. Durante a fase *feedforward*, os pesos das conexões são mantidos fixos. Na retropropagação do erro, por outro lado, os pesos são ajustados de acordo com uma regra de correção do erro. Especificamente, a resposta da rede em um instante de tempo é subtraída da saída desejada (target) para produzir um sinal de erro. Este sinal de erro é propagado da saída para a entrada, camada a camada, originando o nome "retropropagação do erro". Os pesos são ajustados de forma que o valor do erro entre a resposta da rede e a resposta desejada seja reduzido.

Uma rede perceptron de múltiplas camadas possui três características distintas:

1. Função de ativação: o modelo de cada unidade de neurônio da rede inclui uma função de ativação não-linear. A função de ativação não-linear deve ser suave (diferenciável em qualquer ponto). A função de ativação representa o efeito que a entrada interna e o estado atual de ativação exercem na definição do próximo estado de ativação da unidade. Quando propriedades dinâmicas estão envolvidas na definição do estado de ativação, equações diferenciais (caso contínuo) ou a diferenças (caso discreto) são empregadas. Tendo em vista a simplicidade desejada para as unidades processadoras, geralmente define-se seu estado de ativação como uma função algébrica da entrada interna atual, independente de valores passados do estado de ativação ou mesmo da entrada interna. Geralmente, esta função é

monotonicamente não-decrescente e apresenta um tipo de não-linearidade associada ao efeito da saturação.

2. Número de camadas e unidades intermediárias: a rede deve possuir uma ou mais camadas de neurônios ocultos, também conhecidas como camadas intermediárias, pois não são parte da camada de entrada ou da camada de saída da rede. A camada oculta possibilita a rede aprender tarefas complexas extraindo progressivamente as características mais significativas dos padrões da entrada.
3. Conexões: a rede é conectada através de sinapses e possui um alto grau de conectividade. Modificações na conectividade da rede requerem uma mudança na população das conexões sinápticas ou de seus pesos.

O poder computacional do perceptron de múltiplas camadas ocorre da combinação dessas três características, juntamente com a habilidade de aprender através de treinamento. Porém, essas três características também são responsáveis por algumas deficiências sobre o conhecimento da rede. A primeira deficiência é que a forma distribuída de não-linearidade e alta conectividade da rede ocasiona na dificuldade de análise teórica de um perceptron de múltiplas camadas. A segunda deficiência é que a utilização de camadas ocultas faz com que o processo de aprendizagem seja mais difícil de ser visualizado, pois o processo de aprendizagem deve decidir quais características da entrada devem ser representadas pelos neurônios ocultos. Consequentemente, o processo de aprendizagem se torna mais difícil pois a busca deve ser conduzida em um espaço muito maior de funções possíveis e deve ser feita uma escolha entre representações alternativas do padrão de entrada (RUMELHART; HINTON; WILLIAMS, 1985).

2.2.1 Algoritmo de Treinamento por Retropropagação (*Backpropagation*)

A rede neural perceptron de múltiplas camadas com retropropagação geralmente utilizam função sigmoide para a ativação dos neurônios. A função sigmoide utilizada é definida da seguinte forma:

$$\sigma(x) = \frac{1}{1 + e^{-x}} \tag{1}$$

Diferenciando a função:

$$\frac{d\sigma(x)}{dx} = \sigma(x) \cdot (1 - \sigma(x)) \tag{2}$$

A função sigmoide não consegue realmente chegar a 0 ou 1, sendo assim, é comum aceitar valores como 0,9 representado como 1 e 0,1 representado como 0.

O algoritmo de retropropagação inicia atribuindo valores aleatórios aos pesos da rede e geralmente os valores são pequenos, entre -0,5 e 0,5. Os pesos também podem ser normalmente distribuídos em função do número de entradas da camada de entrada, sendo -2,4/n e 2,4/n, onde n representa o número de entradas.

O algoritmo de retropropagação segue como descrito na seção 2.2. Cada iteração do algoritmo envolvendo desde a entrada, a(s) camada(s) oculta(s), até a saída, fazendo o processo de *feedforward* e *backpropagation,* até que as saídas produzidas para os dados de treinamento aproximem-se suficientemente dos valores desejados (até que os valores de erro sejam suficientemente pequenos). Nesta seção, iremos focar como o ajuste dos pesos é realizado pelo algoritmo de retropropagação.

Para o ajuste dos pesos no algoritmo, vamos considerar uma rede de três camadas, a camada de entrada, uma camada oculta e a camada de saída. A variável *i* será utilizada para representar a camada de entrada, a variável *j* para representar a camada oculta e a variável *k* para representar a camada de saída.

Para gerar o valor da saída de um nó *j* da rede, é utilizada a seguinte função:

$$X_j = \sum_{i=1}^{n} X_i \cdot W_{ij} - \theta_j \tag{3}$$

$$Y_j = \frac{1}{1 + e^{-X_j}} \tag{4}$$

O valor de *n* representa o número de entradas para o nó *j*, w_{ij} representa o peso da conexão entre cada nó *i* e cada nó *j*, θ_j representa o valor estabelecido entre

0 e 1 que é o valor de limiar em uso para o nó *j*, x_i é o valor de entrada para o nó de entrada *i* e Y_j representa o valor de saída produzido pelo nó *j*.

Após a rede produzir uma saída, é calculado o gradiente de erro para cada um dos nós *k* na camada de saída. O sinal de erro produzido para k é calculado pela diferença entre o valor desejado (d_k) e o valor gerado pela rede (y_k), como apresentado na equação:

$$e_{k=\, d_k - y_k} \tag{5}$$

O valor do gradiente de erro para o nó de saída *k* é definido como o valor de erro do nó, multiplicado pela derivada da função de ativação, apresentado na equação:

$$\delta_k = \frac{\delta_{yk}}{\delta_{xk}} \cdot e_k \tag{6}$$

Onde x_k é a soma ponderada dos valores de entrada do nó *k*.

Devido *y* ser definido como uma função sigmoide de x, é possível utilizar a fórmula anterior da derivada da função sigmoide para o gradiente de erro:

$$\delta_k = y_k \cdot (1 - y_k) \cdot e_k \tag{7}$$

O valor do gradiente de erro para cada nó *j* da camada oculta é calculado de maneira semelhante, onde *n* representa o número de nós na camada de saída:

$$\delta_j = y_j (1 - y_j) \sum_{k=1}^{n} W_{jk}\, \partial_k \tag{8}$$

Cada peso da rede w_{ij} ou w_{jk} é atualizado de acordo com as seguintes formulas:

$$\begin{aligned} W_{ij} &\leftarrow W_{ij} + \alpha \cdot x_i \cdot \delta_j \\ W_{jk} &\leftarrow W_{jk} + \alpha \cdot y_j \cdot \delta_k \end{aligned} \tag{9}$$

O valor de entrada é representado por x_i para o nó *i* e α é a taxa de aprendizado, sendo um número positivo abaixo de 1 e que não deve ser muito alto.

Este método é conhecido como gradiente descendente, pois ele envolve descer a superfície que representa a função de erro pelo caminho mais íngreme, para tentar achar o mínimo no espaço de erro, o que representará o conjunto de pesos que fornece o melhor desempenho da rede.

Na verdade, a iteração do algoritmo de retropropagação é geralmente concluída quando a soma dos quadrados dos erros dos valores de saída, para todos os dados de treinamento em uma época, for menor do que um limiar, tal como 0,001.

Note que este método atribui responsabilidade a nós individuais dentro da rede comparando os pesos de cada nó com o erro associado aquele nó. No caso dos nós ocultos, não há valor de erro por não haver um valor de saída específico para estes nós. Neste caso, o peso de cada conexão entre um nó da camada oculta e um nó de saída é multiplicado pelo erro daquele nó de saída para tentar distribuir a responsabilidade entre os nós na camada oculta de acordo com quanto cada um contribui para o erro.

2.2.2 Treinamento Local e em Lote

O aprendizado do algoritmo de retropropagação resulta das múltiplas apresentações (entradas) de um conjunto de exemplos de treinamento para o perceptron de múltiplas camadas. No treinamento do algoritmo é utilizado o conceito de época, sendo que cada época é determinada pela apresentação completa do conjunto de treinamento. O processo de aprendizagem é mantido em uma base de época em época até os pesos sinápticos e os níveis de bias se estabilizarem e o erro médio quadrado sobre todo o conjunto de treinamento convergir para um valor mínimo (HAYKIN, 1994). Para garantir um melhor treinamento, é recomendado que a ordem de apresentação dos exemplos de treinamento seja aleatória de uma época para a outra, pois a aleatoriedade tende a tornar a busca no espaço de pesos estocástica sobre os ciclos de aprendizagem, o que evita a possibilidade de ciclos limitados.

Existem duas formas básicas em que o aprendizado por retropropagação pode ocorrer através de um dado conjunto de dados para treinamento, que são:

- Treinamento Local (Sequencial): além dos nomes de treinamento local ou treinamento sequencial, esse modo de treinamento também é conhecido frequentemente como treinamento online, estocástico ou padrão. No treinamento sequencial, cada exemplo de treinamento é apresentado ao algoritmo e atualização dos pesos é realizada logo após cada apresentação de exemplo.
- Treinamento em Lote: diferente do treinamento sequencial, no treinamento em lote, como o próprio nome já sugere, o ajuste dos pesos somente é realizado após a apresentação de todos os exemplos de treinamento que constituem uma época.

Comparando os dois modos de treinamento, o modo sequencial é mais indicado que o modo por lote, pois tem a vantagem de necessitar menos espaço de armazenamento local para cada conexão sináptica. Outra vantagem é que como os dados são apresentados à rede aleatoriamente, o uso de ajuste de pesos de passo a passo torna a busca no espaço de pesos de natureza estocástica, tornando menos provável que o algoritmo de retropropagação se prenda em um mínimo local. Em contrapartida, a natureza estocástica do modo sequencial faz com que seja mais difícil estabelecer as condições teóricas para a convergência do algoritmo. A apresentação sequencial dos dados também é vantajosa quando no conjunto de dados de treinamento contém cópias exatas dos mesmos padrões, ou seja, são dados redundantes, principalmente quando o conjunto de dados é grande e altamente redundante.

A vantagem do modo em lote em comparação com o modo sequencial, é que o uso do treinamento por lote oferece uma estimativa precisa do vetor de gradiente, sendo assim a convergência para um mínimo local é garantida sob condições simples. Outra vantagem é que o modo por lote torna mais fácil de ser paralelizado em comparação com o modo sequencial.

Dentre os dois modos de treinamento, o modo de treinamento sequencial é o mais popular e utilizado, devido a duas grandes vantagens, a facilidade na implementação do algoritmo e a capacidade de fornecer soluções efetivas a problemas grandes e difíceis.

2.2.3 Critérios de Parada

No processo de treinamento não há um critério de parada da execução do algoritmo bem definido e não há também uma convergência garantida. Como não há uma convergência garantida, não é recomendado definir um valor fixo de quantidade de iterações para interromper o treinamento. Para suprir esse problema, há alguns critérios razoáveis que podem ser utilizados para encerrar o treinamento, cada critério com seu mérito.

Para formular um critério de parada, deve-se considerar a possibilidade de existência de mínimos locais. Seja θ^* o vetor de parâmetros (pesos) que denota um ponto de mínimo, seja ele local ou global. Uma condição necessária para que θ^* seja um mínimo, é que o vetor gradiente $\nabla J(\theta)$ (ou seja, a derivada parcial de primeira ordem) da superfície de erro em relação ao vetor de pesos θ seja zero em $\theta = \theta^*$. Podemos assim formular um critério de convergência sensível para a aprendizagem por retropropagação:

- Considera-se que o algoritmo de retropropagação tenha convergido quando a norma euclidiana do vetor gradiente alcançar um limiar suficientemente pequeno.

O problema desse critério de parada é que para obter simulações bem-sucedidas, o tempo de treinamento pode ser muito longo. Este critério também requer o cálculo da norma do vetor gradiente.

Outra propriedade única de um mínimo, e que pode ser utilizada como critério de parada, é o fato de que a função custo, ou medida de erro, $J_{med}(\theta)$ é estacionária no ponto $\theta = \theta^*$. Consequentemente é possível sugerir um novo critério de parada:

- Considera-se que o algoritmo de retropropagação tenha convergido quando a taxa absoluta de variação do erro médio quadrado por época for suficientemente pequena.

O treinamento deve ser interrompido quando a rede apresentar uma boa capacidade de generalização e quando a taxa de erro for suficientemente pequena, menor que um erro admissível. É necessário encontrar um ponto ótimo de parada com erro mínimo e capacidade de generalização máxima.

2.3 MÉTODOS DE AVALIAÇÃO DE ALGORITMOS COM REDES NEURAIS ARTIFICIAIS

A avaliação de algoritmos pode ser realizada de diferentes maneiras, o que depende da finalidade da avalição do desempenho. É possível abstrair o hardware em que o algoritmo é executado, realizando a avaliação dos algoritmos através de cada instrução executada, como apresentado por (CORMEN, 2009), isso garante que todos os algoritmos sejam avaliados na mesma base e seu resultado de desempenho seja indiferente do hardware de execução.

Os algoritmos de treinamento com redes neurais podem também ser avaliados por instrução, porém quando focamos no resultado final que a rede neural produz, a avalição de maior importância é o que se conseguiu obter com o treinamento do algoritmo, como valores de erro da rede, taxa de regressão (dependendo do problema), entre outros.

Um exemplo de trabalho que realizada avaliação do desempenho dos algoritmos com redes neurais é apresentado por (DHAR, V. K. et al, 2010), aplicando alguns algoritmos de redes neurais em problemas *benchmark* e funções de avaliação. O trabalho utiliza como parâmetro de comparação entre os algoritmos apenas o *Root Mean Squared Error* (RMS), que é uma medida frequentemente utilizada para encontrar o erro entre diferenças dos valores preditos por um modelo ou um estimador e os valores experimentais.

Diferente do trabalho apresentado por (DHAR, V. K. et al, 2010), o presente trabalho utiliza diversos parâmetros para a comparação entre os algoritmos, são eles: *Mean Squared Error* (MSE), Regressão, Taxa de Acerto no Treinamento, Taxa de Acerto na Simulação e Tempo de Treinamento.

O MSE é um estimador que mede o quão perto a convergência da rede neural conseguiu chegar perto do objetivo. É a média dos quadrados dos erros, a diferença entre o objetivo e o que é estimado. A diferença ocorre por causa da aleatoriedade ou porque a rede não considera informações que poderiam produzir uma estimativa mais precisa. O MSE é considerado uma medida de qualidade, os valores nunca são negativos e quanto mais próximo de zero melhor o resultado.

A Regressão é interpretada como a relação entre as variáveis de entrada e a variável de saída, o quanto a rede consegue realizar essa relação para conseguir

convergir. O valor da regressão vai de 0 até 1, sendo que quanto mais próximo de 1, melhor o valor da regressão.

O erro é calculado ponto a ponto a partir da diferença da riqueza produzida pela rede neural e a riqueza real aquisitada no veículo (objetivo). Quando a diferença entre os pontos da riqueza real e da riqueza da rede neural é menor ou igual a 1%, então o erro é $\varepsilon \leq 1\%$. Se a diferença entre os pontos é maior que 1% e menor ou igual de 3%, então o erro é $\varepsilon \leq 3\%$. Se a diferença entre os pontos é maior que 3% e menor ou igual a 5%, então o erro é $\varepsilon \leq 5\%$. Se a diferença entre os pontos é maior que 5% e menor ou igual a 10%, então o erro é $\varepsilon \leq 10\%$ e se a diferença entre os pontos é maior que 10%, então o erro é $\varepsilon \leq 10\%$. A taxa de acerto é a soma dos erros $\varepsilon \leq 1\%$ e $\varepsilon \leq 3\%$. Consideramos que erros até 3% são aceitáveis para quando tratamos essa categoria de problema.

A taxa de acerto no treinamento é importante para saber o quão preciso a rede está conseguindo convergir em comparação com o valor real (objetivo), porém nem sempre boas taxas de treinamento refletem em boas taxas de simulação. A taxa de acerto no treinamento e na simulação são calculadas da mesma maneira, o que difere entre as duas são os dados utilizados para calcular cada uma. No treinamento os dados utilizados são os mesmos que treinaram a rede, na simulação são dados diferentes de uma nova coleta realizada no veículo.

Assim, criamos nosso próprio índice para comparação entre os algoritmos, pois em nossos experimentos nem sempre boas taxas de MSE e regressão resultavam em boas taxas de acerto no treinamento e na simulação. A tabela de pontuação de cada critério comparado é explicada detalhadamente na seção 5.7.

2.4 LEVENBERG-MARQUARDT BACKPROPAGATION

O algoritmo *Levenberg-Marquardt* (LM) é utilizado para resolver problemas de mínimos quadrados não lineares. O método de ajuste do algoritmo é a combinação de outros dois métodos, o método Gradiente Descendente e o método de Gauss-Newton.

Ambos os métodos, do Gradiente Descendente e de Gauss-Newton são algoritmos iterativos, ou seja, realizam uma série de cálculos (baseado em palpites para um determinado valor x) para encontrar a melhor solução. Em cada iteração do

método Gradiente Descendente, a solução é atualizada escolhendo valores que tornam o valor da função menor, sendo assim a soma dos erros quadrados é reduzida movendo-se na direção da descida mais acentuada. O algoritmo *Levenberg-Marquardt* em cada iteração escolhe o método do Gradiente Descendente ou Gauss-Newton e atualiza o valor da solução (MORÉ, 1978).

A atualização iterativa do algoritmo depende do valor de λ, que define qual o método será utilizado. O valor de λ representa um fator de amortecimento não negativo de suavização. Os valores serão atualizados pelo método de Gauss-Newton se o valor de λ for pequeno, próximo ao valor ótimo. E serão atualizados por Gradiente Descendente quando o valor de λ for grande (GAVIN, 2007). O método de Gauss-Newton é mais preciso e rápido que o método Gradiente Descente quando próximo ao erro mínimo. Sendo assim, o algoritmo *Levenberg-Marquardt* migrará para o método de Gauss-Newton o mais rápido possível.

O algoritmo *Levenberg-Marquardt* tende a ser muito rápido para o treinamento de redes neurais artificiais, porém requer uma grande quantidade de memória. Entre outras vantagens e desvantagens do algoritmo, é possível citar como vantagem a existência de duas opções possíveis para a direção do algoritmo em cada iteração, sendo assim o algoritmo é mais robusto que o de Gauss-Newton (NELLES, 2013). É possível lidar com modelos com vários parâmetros que não são conhecidos com precisão (observe que, para conjuntos muito grandes, o algoritmo pode ser lento) e se o seu palpite inicial está longe do objetivo, o algoritmo ainda pode encontrar uma solução ideal. Entre as desvantagens, é possível citar que em alguns casos o algoritmo pode ser muito lento para convergir.

Os problemas de aproximação que utilizam um número finito de dados amostrados e definido um modelo de aproximação ĝ(.,θ), A função do vetor de parâmetros $\in \Re^{P}$ é a distância entre a função a ser aproximada e sua aproximação dist(g(.), ĝ(.,θ)). Se tomarmos a norma euclidiana como medida de distância, a expressão produzida é:

$$J(\theta) = \frac{1}{N}\sum_{l=1}^{N}(\mathrm{g}(\mathrm{x}) - \hat{\mathrm{g}}(\mathrm{x},\theta))^2 \qquad (10)$$

A equação anterior representa o erro quadrático médio (MSE). Considerando que a função seja a soma dos erros quadráticos (SSE) e pode haver múltiplas saídas para o problema, a equação para o erro é:

$$J(\theta) = \sum_{i=l}^{N}\sum_{j=l}^{M}(g_{ij}(x) - \hat{g}_{ij}(x,\theta))^2 = \sum_{k=l}^{q} {r_l}^2 \tag{11}$$

sendo *N* o número de amostras, *l* o número de camadas neurônios, *r* o erro residual, *m* o número de saídas, e *q* o produto *N* x *m*.

A matriz Jacobiana da equação anterior pode ser escrita da seguinte forma:

$$\mathbf{J} \equiv \begin{bmatrix} \nabla {r_1}^t \\ \vdots \\ \nabla {r_q}^t \end{bmatrix} \tag{12}$$

sendo o erro residual denominado por *r*.

Diferenciando a equação 12, temos:

$$\nabla J = 2\mathbf{J}^T\mathbf{r} = 2\sum_{k=1}^{q} r_k \nabla \mathbf{r}_k \tag{13}$$

$$\nabla^2 J = 2\left(\mathbf{J}^T\mathbf{J} + \sum_{k=1}^{q} r_k\, \nabla^2 \mathbf{r}_k\right) \tag{14}$$

Quando os erros residuais são suficientemente pequenos, a matriz hessiana pode ser aproximada pelo primeiro termo da equação acima, resultando:

$$\nabla^2 J \approx 2\mathbf{J}^T\mathbf{J} \tag{15}$$

Esta aproximação geralmente é válida em um mínimo de J para a maioria dos propósitos, e é a base para o método de Gauss-Newton (HAGAN, 1994). A lei de atualização torna-se então:

$$\Delta\theta = [\mathbf{J}^T\mathbf{J}]^{-1}\mathbf{J}^T\mathbf{r} \tag{16}$$

A modificação de Levenberg-Marquardt para o método de Gauss-Newton é:

$$\Delta\theta = [\mathbf{J}^T\mathbf{J} + \mu\mathbf{I}]^{-1}\mathbf{J}^T\mathbf{r} \quad (17)$$

O efeito da matriz adicional $\mu\mathbf{I}$ é adicionar μ a cada autovalor de $\mathbf{J}^T\mathbf{J}$. Uma vez que a matriz $\mathbf{J}^T\mathbf{J}$ é semi-definida positiva e, portanto, o autovalor mínimo possível é zero, qualquer valor positivo, pequeno, mas numericamente significativo, de μ será suficiente para restaurar a matriz aumentada e produzir uma direção descendente de busca.

2.5 BAYESIAN REGULARIZATION BACKPROPAGATION

O método *Levenberg-Marquardt* foi desenvolvido especialmente para uma convergência mais rápida em algoritmos de retropropagação. Essencialmente, o *Bayesian Regularization Backpropagation* (BR) em comparação com o algoritmo de *Levenberg-Marquardt,* tem uma função objetiva que inclui uma soma residual de quadrados e a soma de pesos quadrados para minimizar os erros de estimativa e para alcançar um bom modelo generalizado (KAYRI, 2016).

Redes neurais artificiais que utilizam o método de *Bayesian Regularization Backpropagation* são mais robustas do que as redes de *backpropagation* padrão e podem reduzir ou eliminar a necessidade de uma longa validação cruzada.

O método *Bayesian Regularization* é um processo matemático que converte uma regressão não linear em um problema estatístico bem posicionado na forma de uma regressão em crista. A vantagem do modelo é sua robustez e o seu processo de validação é desnecessário, sendo que o processo de validação em *backpropagation* padrão é escalável como $O(n^2)$. O método também dificulta que ocorra *overtrain* (quando o algoritmo treina mais do que o necessário e prejudica o aprendizado), uma vez que os procedimentos de evidência fornecem um critério bayesiano objetivo para interromper o treinamento. O método calcula e treina em um número de parâmetros de rede ou pesos efetivos, efetivamente desligando aqueles que não são relevantes. Este número efetivo é geralmente consideravelmente menor que o número de pesos em uma rede neural padrão de *backpropagation* totalmente conectada.

No treinamento o método introduz peso na rede detonado por (YUE; SONGZHENG; TIANSHI, 2011):

$$\mathrm{F}(\omega) = \alpha E\omega + \beta E_{\mathrm{D}} \tag{18}$$

As variáveis $\boldsymbol{\alpha}$ e $\boldsymbol{\beta}$ são parâmetros das funções objetivas. A variável $E\omega$ é a soma quadrada dos pesos da rede e a variável E_D é a soma dos erros da rede. Os pesos da rede são variáveis randômicas e a distribuição dos pesos da rede e o treinamento são considerados distribuição Gaussiana. Os valores de $\boldsymbol{\alpha}$ e $\boldsymbol{\beta}$ são definidos pelo Teorema de Bayes, que relaciona duas variáveis A e B com base em suas probabilidades de variáveis anteriores e probabilidades posteriores, tal como apresentado na equação (LI; SHI, 2012):

$$\mathrm{P}(\mathrm{A}|\mathrm{B}) = \frac{\mathrm{P}(\mathrm{B}|\mathrm{A})\mathrm{P}(\mathrm{A})}{\mathrm{P}(\mathrm{B})} \tag{19}$$

Onde P(A|B) é a probabilidade posterior de A condicional em B, P(B|A) o anterior de B condicional em A e P(B) a probabilidade anterior diferente de zero do evento B, que funciona como uma constante de normalização. Com o objetivo de encontrar o espaço de peso ideal, a equação 19 precisa ser minimizada, o que equivale a maximizar a posterior função de probabilidade:

$$\mathrm{P}(\alpha, \beta|D, M) = \frac{P(D|\alpha, B, M)P(\alpha, \beta|M)}{P(D|M)} \tag{20}$$

As variáveis $\boldsymbol{\alpha}$ e $\boldsymbol{\beta}$ são fatores necessários para otimizar, D é o peso da distribuição, M é a arquitetura da rede neural, $P(D|M)$ é o fator de normalização, $P(\boldsymbol{\alpha},\boldsymbol{\beta}|M)$ é a densidade prévia uniforme para os parâmetros de regularização e $P(D|\boldsymbol{\alpha},\boldsymbol{\beta},M)$ é a função de probabilidade de D para o dado $\boldsymbol{\alpha},\boldsymbol{\beta},M$. Maximizando a função posterior $P(\boldsymbol{\alpha},\boldsymbol{\beta}\ |D,M)$ é equivalente e maximizar a probabilidade da função $P(D|\boldsymbol{\alpha},\boldsymbol{\beta},M)$.

Como resultado deste processo, os valores ótimos para $\boldsymbol{\alpha}$ e $\boldsymbol{\beta}$ para um dado espaço de peso são encontrados. Depois, o algoritmo move-se para a fase LM, onde os cálculos da matriz de Hessiana ocorrem e atualiza o espaço de peso, a fim de

minimizar a função objetivo. Então, se a convergência não for atendida, o algoritmo estima novos valores para $\boldsymbol{\alpha}$ e $\boldsymbol{\beta}$ e o todo o processo se repete até que a convergência seja alcançada (YUE; SONGZHENG; TIANSHI, 2011).

2.6 SCALED CONJUGATE GRADIENT BACKPROPAGATION

O algoritmo básico de retropropagação ajusta os pesos na direção de descida mais íngreme, ou seja, a mais negativa do gradiente. Esta é a direção na qual a função de desempenho está diminuindo mais rapidamente. Acontece que, embora a função diminua mais rapidamente ao longo do negativo do gradiente, isso não produz necessariamente a convergência mais rápida (HAGAN et al., 1996).

Scaled Conjugate Gradient Backpropagation (SCG) é um algoritmo de aprendizado supervisionado e o membro da classe de métodos de *Conjugate Gradient*. O SCG usa conceitos semelhantes da estratégia de otimização geral, mas escolhe a direção da pesquisa e o tamanho do passo de forma mais eficaz usando informações da segunda ordem. No método *Scaled Conjugate Gradient Backpropagation*, uma busca é realizada ao longo de uma direção que produz convergência geralmente mais rápida do que a direção de descida mais íngreme, preservando a minimização de erros alcançada em todas as etapas anteriores (KIŞI; UNCUOĞLU, 2005). Essa direção é chamada de direção conjugada. Na maioria dos algoritmos SCG, o tamanho do passo é ajustado em cada iteração. Uma pesquisa é feita ao longo da direção do gradiente conjugado para determinar o tamanho do passo, o que minimizará a função de desempenho ao longo dessa linha. Todos os algoritmos de SCG começam pela busca na direção de descida mais íngreme na primeira iteração.

A minimização é um processo iterativo local no qual uma aproximação da função, em uma vizinhança do ponto atual no espaço de peso é minimizada. A maioria dos métodos de otimização usados para minimizar as funções são baseados na mesma estratégia. O algoritmo SCG (MØLLER, 1993) denota a aproximação quadrática ao erro *E* em uma vizinhança de um ponto *w* por:

$$E_{qw}(y) = E(w) + E'(w)^T y + \frac{1}{2} y^T E''(w) y \tag{21}$$

Para determinar o mínimo para $E_{qw}(y)$, os pontos críticos para E_{qw} (y) devem ser encontrados. Os pontos críticos são a solução para o sistema linear definido por (MØLLER, 1993):

$$E'_{qw}(y) = E''(w)y + E'(w) = 0 \quad (22)$$

O SCG pertence à classe de Métodos de Gradiente Conjugado. Ao usar um mecanismo de dimensionamento de tamanho de etapa, o SCG evita busca de linha demorada por iteração de aprendizado, o que torna o algoritmo mais rápido do que outros algoritmos de segunda ordem.

2.7 RESILIENT BACKPROPAGATION

O método *Resilient Backpropagation* (Rprop) é um algoritmo utilizado para treinamento supervisionado da rede neural que se assemelha ao algoritmo de retropropagação padrão. Porém, o *Resilient Backpropagation* possui duas vantagens principais. A primeira é que o treinamento com o Rprop é frequentemente mais rápido que o treinamento com a retropropagação padrão. A segunda vantagem é que o Rprop não requer que você especifique nenhum valor de parâmetro livre, ao contrário da retropropagação padrão, que precisa de valores para a taxa de aprendizado. A principal desvantagem do Rprop é que é um algoritmo mais complexo de implementar do que a retropropagação padrão.

O princípio básico do funcionamento do Rprop é eliminar a influência prejudicial do tamanho da derivada parcial na etapa de peso. Como consequência, apenas o sinal da derivada é considerado para indicar a direção da atualização de peso. O tamanho da mudança de peso é determinado exclusivamente por um valor de atualização específico do peso $\Delta_{ij}^{(t)}$:

$$\boldsymbol{\Delta w_{ij}^{(t)}} = \begin{cases} -\boldsymbol{\Delta_{ij}^{(t)}}\ , & se\ \dfrac{\partial E^{(t)}}{\partial \mathrm{w}_{ij}} > 0 \\ +\boldsymbol{\Delta_{ij}^{(t)}}\ , & se\ \dfrac{\partial E^{(t)}}{\partial \mathrm{w}_{ij}} < 0 \\ 0\ , & senão \end{cases} \tag{23}$$

Onde $\frac{\partial E^{(t)}}{\partial \mathrm{w}_{ij}}$ denota a informação gradiente somada sobre todo o padrão do conjunto de padrões.

A segunda etapa do aprendizado Rprop é determinar os novos valores de atualização $\Delta_{ij}^{(t)}$. Isso é baseado em um processo de adaptação dependente de sinais, semelhante à adaptação da taxa de aprendizado em (JACOBS, 1988) (TOLLENAERE, 1990).

$$\boldsymbol{\Delta_{ij}^{(t)}} = \begin{cases} \eta^{+} \cdot \boldsymbol{\Delta_{ij}^{(t-1)}}\ , & se\ \dfrac{\partial E^{(t-1)}}{\partial \mathrm{w}_{ij}} \cdot \dfrac{\partial \mathrm{E}^{(t)}}{\partial \mathrm{w}_{ij}} > 0 \\ \eta^{-} \cdot \boldsymbol{\Delta_{ij}^{(t-1)}}\ , & se\ \dfrac{\partial E^{(t-1)}}{\partial \mathrm{w}_{ij}} \cdot \dfrac{\partial \mathrm{E}^{(t)}}{\partial \mathrm{w}_{ij}} < 0 \\ \boldsymbol{\Delta_{ij}^{(t-1)}}\ , & senão \end{cases} \tag{24}$$

A regra de adaptação funciona da seguinte maneira: Toda vez que a derivada parcial do peso correspondente $\partial \mathrm{w}_{ij}$ muda seu sinal, o que indica que a última atualização foi muito grande e o algoritmo saltou sobre um mínimo local, o valor de atualização $\Delta_{ij}^{(t)}$ é diminuído pelo fator η^{-}. Se a derivada retém seu sinal, o valor de atualização é ligeiramente aumentado para acelerar a convergência em regiões superficiais. Além disso, no caso da mudança de sinal, não deve haver adaptação na etapa de aprendizagem subsequente. Na prática, isso pode ser alcançado definindo-se $\frac{\partial E^{(t-1)}}{\partial \mathrm{w}_{ij}} := 0$ na regra de adaptação acima.

A fim de reduzir o número de parâmetros livremente ajustáveis, muitas vezes levando a uma busca tediosa no espaço de parâmetros, o fator de aumento e diminuição são definidos para valores fixos. A escolha do fator de diminuição η^{-} foi conduzida pelas seguintes considerações. Se o salto acima de um mínimo ocorreu, o valor de atualização anterior era muito grande. Em média, será um bom palpite reduzir para metade o valor de atualização (estimador de máxima probabilidade), então

escolhemos $\eta^- := 0.5$. O fator de aumento η^+, por um lado, tem que ser grande o suficiente para permitir o rápido crescimento do valor de atualização em regiões rasas da função de erro, mas por outro lado o processo de aprendizado pode ser consideravelmente perturbado, se um fator de aumento muito grande a mudanças persistentes da direção da etapa de peso. Em vários experimentos do problema examinado, a escolha de $\eta^+ = 1.2$ deu resultados muito bons, independente do problema examinado (RIEDMILLER, 1994). Pequenas variações desse valor não melhoraram nem deterioraram o tempo de convergência. Então, a fim de obter a escolha de parâmetros mais simples, decidimos fixar constantemente o parâmetro de aumento para $\eta^+ = 1.2$ (RIEDMILLER, 1994).

Para a Rprop tentar adaptar seu processo de aprendizado à topologia da função de erro, segue o princípio da aprendizagem por época. Isso significa que a atualização de peso e a adaptação são realizadas depois que as informações de gradiente de todo o conjunto de padrões são calculadas.

3 TRABALHOS RELACIONADOS

Nesta seção são apresentados alguns trabalhos relacionados com o trabalho proposto. Para tal, foram selecionados os trabalhos de (KIŞI; UNCUOĞLU, 2005), (PARLAK et al., 2006), (DENG *et al.*, 2011) e (CAY, 2013).

3.1 COMPARAÇÃO DE TRÊS ALGORITMOS DE TREINAMENTO DE RETRO-PROPAGAÇÃO PARA DOIS ESTUDOS DE CASO

O trabalho desenvolvido por (KIŞI; UNCUOĞLU, 2005) apresenta a comparação entre três algoritmos de treinamento com *backpropagation* aplicado em dois estudos de caso. Dois dos três algoritmos comparados também são utilizados na dissertação, são eles: *Levenberg-Marquardt* e *Resilient Backpropagation*. O terceiro algoritmo comparado no artigo é o *Conjugate Gradient*. O trabalho compara os algoritmos pela velocidade de convergência de cada um no treinamento e a performance nos testes.

O primeiro estudo de caso são os dados de fluxos diário do córrego de Filyos na estação de Derecikviran na Turquia. Os dados foram divididos em treinamento e teste, contendo a mesma quantidade de dados em ambos, 2000 dados em cada conjunto, porém em tempos diferentes. Para esse estudo de caso a rede neural desenvolvida foi uma MLP com três camadas, a camada de entrada, uma camada oculta e a camada de saída. Os autores relatam a dificuldade em encontrar a quantidade de neurônios da camada oculta, a taxa de treinamento e os pesos iniciais para as entradas. Para determinar esses valores, os autores fizeram os experimentos de diversas combinações de neurônios, taxas de treinamento e pesos iniciais até encontrarem uma solução, porém a solução não é divulgada no artigo.

O segundo estudo de caso é a determinação do estresse lateral em solos em coesão. A quantidade de dados do estudo de caso é inferior ao primeiro estudo de caso, nesse são utilizados 176 dados para treinamento e 88 dados para testes. Assim como no primeiro estudo de caso, também foi desenvolvida uma MLP com uma camada oculta e para determinar a quantidade de neurônios na camada oculta foi por tentativa e erro, testando diversas combinações.

Os resultados mostraram que é possível encontrar boas soluções para os dois estudos de caso com os três algoritmos. O algoritmo de Levenberg-Marquardt leva uma pequena fração do tempo gasto menos do que os outros algoritmos para treinamento da rede. No treinamento, o algoritmo *Resilient Backpropagation* também é mais rápido que o *Conjugate Gradient*. Embora o Levenberg-Marquardt tenha melhores critérios de desempenho em treinamento, os resultados de teste do *Resilient Backpropagation* foram melhores em ambos os estudos de caso. Isso mostra a consistência do algoritmo *Resilient Backpropagation*. Os autores finalizam o trabalho concluindo que é difícil saber qual algoritmo terá uma melhor performance para um determinado estudo de caso, pois pode variar muito, dependendo de muitos fatores, como a complexidade do problema, a quantidade de dados, entre outros.

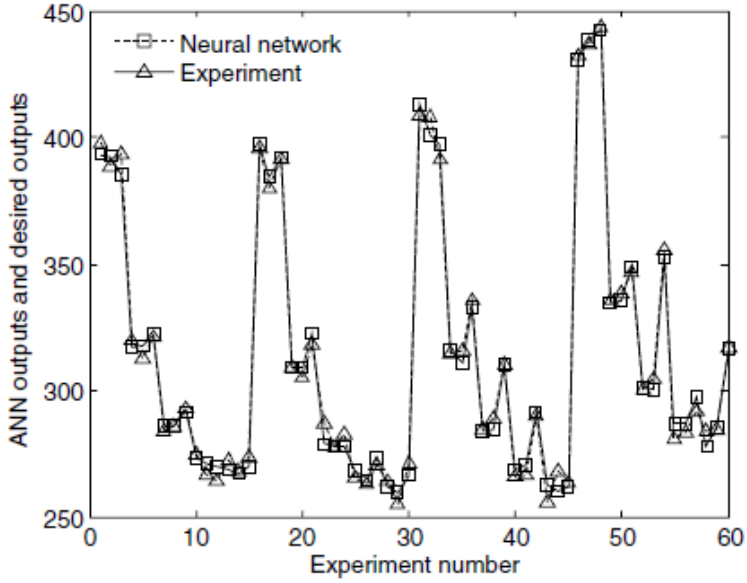
Os dois estudos de caso não se relacionam com o estudo de caso utilizado no presente trabalho, porém é realizada a comparação do desempenho de dois algoritmos que também são utilizados no trabalho. Podendo assim visualizar algumas maneiras de comparação de desempenho entre os algoritmos.

3.2 APLICAÇÃO DE REDES NEURAIS ARTIFICIAIS PARA PREVISÃO DE CONSUMO ESPECÍFICO DE COMBUSTÍVEL E TEMPERATURA DE ESCAPE PARA UM MOTOR DIESEL

O trabalho publicado por (PARLAK et al., 2006) foi um dos primeiros a utilizar redes neurais artificiais para simular o consumo de combustível e a temperatura de exaustão de motores a Diesel. Os motores a Diesel se diferem dos motores a combustão interna ciclo Otto do trabalho proposto, porém existem características na construção da rede neural que podem ser utilizadas nos dois motores (PARLAK et al., 2006).

Devido as diferenças entre os motores, tomaremos como base somente o capítulo referente as redes neurais artificiais e seus resultados. Foram utilizados um conjunto com 60 valores para treinamento da rede neural e outros 20 valores escolhidos randomicamente para testes. Ao todo, 900 ciclos de treinamento foram realizados, atingindo um nível de máximo de erro relativo satisfatório descrito pelos autores como 1,93% para o treinamento e 2,26% para testes.

Gráfico 1 - Valor Real (Experiment) e o Valor Produzido pela Rede Neural (Neural network) no Treinamento

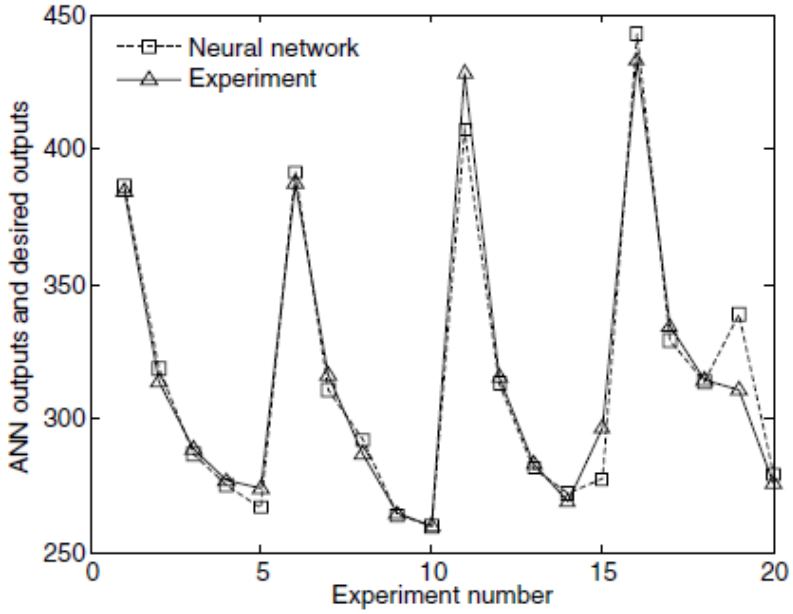


Fonte: Parlak et al. (2006)

O Gráfico 1 apresenta o gráfico entre o valor da rede neural treinada (pontos em quadrado) e o valor desejado do experimento no treinamento (pontos em triangulo). É possível observar que no treinamento a rede teve um ótimo desempenho com a rede neural conseguindo produzir pontos quase idênticos ao experimento.

A Gráfico 2 apresenta um gráfico semelhante ao apresentado na Figura 10. O gráfico também apresenta os valores entre o resultado produzido pela rede neural e o resultado do experimento, porém o experimento não é o mesmo apresentado na Figura 10, é o experimento de teste com o objetivo de validar a rede neural. A rede neural conseguiu simular com precisão os valores de teste, seguindo na maioria dos pontos o valor do experimento, apenas nos pontos 15 e próximo do 20 que houve uma maior dispersão.

Gráfico 2 - Valor Real (Experiment) e o Valor Produzido pela Rede Neural (Neural network) em Dados de Teste



Fonte: Parlak et al. (2006)

3.3 AS APLICAÇÕES DE REDES NEURAIS ARTIFICIAIS EM MOTORES

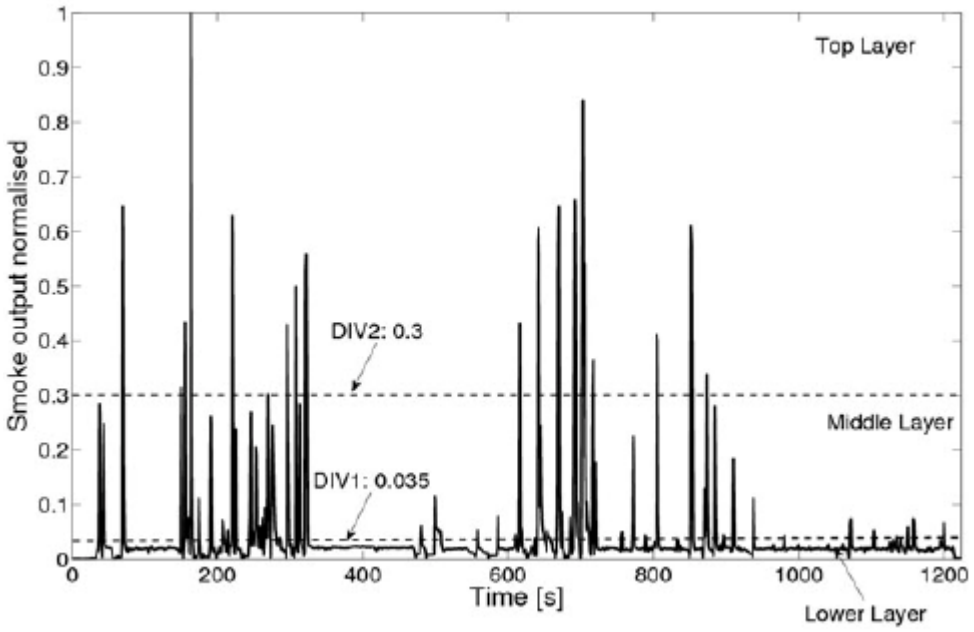
Em (DENG *et al.*, 2011), é proposta a aplicação de redes neurais artificiais em motores a combustão, como uma alternativa que oferece uma maneira tratamento dos dados de calibração do motor e ajuda na redução de emissões de poluentes do veículo e melhoria na economia de combustível. A emissão de poluentes do veículo está diretamente relacionada a estequiometria da riqueza, pois tanto uma mistura ar-combustível pobre quanto uma mistura rica fazem com que o veículo emita mais gases poluentes.

O trabalho é dividido em 3 capítulos. O primeiro capítulo aborda a construção da arquitetura da rede neural artificial através de experimentos e coleta de dados. Nesse capítulo, o trabalho reforça a importância da qualidade da coleta dos dados, sendo uma etapa crucial para o desenvolvimento da rede neural artificial, pois tem um grande impacto no desempenho da rede, especialmente por se tratar de uma modelagem de motor. O segundo capítulo do trabalho apresenta a combinação de diversas arquiteturas de redes neurais artificiais, focando em dividir as tarefas de treinamento entre as diversas redes para atingir o melhor resultado possível. O terceiro capítulo do trabalho apresenta a aplicação prática para a modelagem e simulação de emissões de NO_x (*Nitrogen Oxide*) e PM (*Particulate Matter*).

Para o treinamento e teste da rede neural, foram utilizadas 12 variáveis de entrada e somente uma de saída. As variáveis de entrada foram o torque, aumento da pressão, velocidade, massa de combustível, injeção fim de combustível, contrapressão, temperatura do coletor de admissão, temperatura de exaustão, depressão da entrada, temperatura de refrigeração dentro e fora. A única variável de saída foi o sinal da fumaça de saída.

Interessante destacar a estratégia que os autores utilizaram para treinar a rede neural. Foram divididos os conjuntos de dados em 3 partes, treinando cada uma das 3 partes separadamente, como apresentado no Gráfico 3. O conjunto de dados foi dividido em *lower layer* (LL), que contém os dados com sinais de fumaça de 0 até 0.035, *medium layer* (MD), com sinais de 0.035 até 0.3 e *top layer* (TL), com sinais de 0.3 até 1.

Gráfico 3 - Divisão entre os sinais no treinamento da rede neural



Fonte: Deng et al. (2011)

O resultado obtido com o treinamento e validação da rede neural foi caracterizado como bom pelos autores, como apresentado na Tabela 2.

Tabela 1 - Indicação de performance do treinamento e validação da rede neural

Layer	Training	Validation
TL	0.8330	0.9163
ML	0.9328	0.9006
LL	0.9743	0.9164
Total	0.9706	0.9616

Fonte: Deng et al. (2011)

Pelos valores da tabela é possível observar que no total a rede conseguiu obter bons resultados de treinamento, com 0.9706 de 1, sendo que o *low layer* obteve o melhor resultado, com um valor 0.9743 que é superior ao total, porém o médium layer e *top layer* obtiveram valores abaixo, com 0.9328 e 0.8330 respectivamente. O valor da validação também foi alto, atingindo 0.9616 e os três *layers* atingiram resultado semelhantes.

3.4 PREVISÃO DO DESEMPENHO DO MOTOR DE GASOLINA COM REDE NEURAL ARTIFICIAL

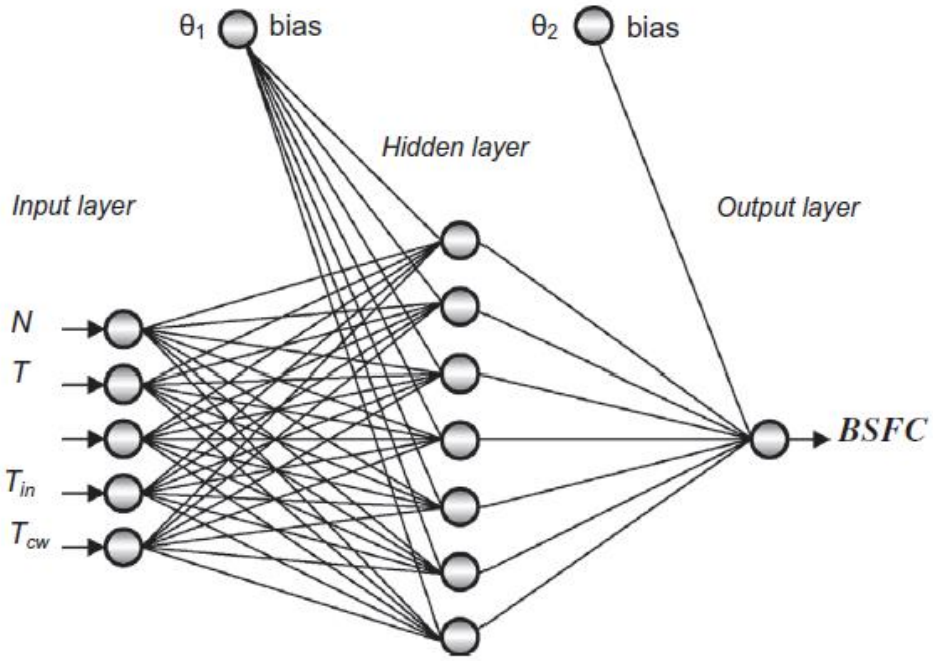
O trabalho apresentado por (CAY, 2013) realiza o estudo e criação de um modelo de rede neural artificial para motores a gasolina, com o objetivo de simular o consumo de combustível do motor, sua potência e a temperatura de exaustão do motor.

O motor utilizado para experimentos e testes foi um motor quatro cilindros de um Ford-Escort. Para o treinamento da rede neural artificial, foram utilizadas ao todo 8 variáveis do motor. A velocidade do motor, o torque do motor, temperatura média do coletor de admissão, a taxa de fluxo do combustível e a entrada de água de refrigeração foram utilizados como variáveis de entrada para o treinamento. Outras três variáveis do motor foram utilizadas como variáveis de saída separadamente, sendo assim, foram criadas três redes neurais com o mesmo modelo e as mesmas variáveis de entrada, alterando somente as variáveis de saída, que são o consumo específico de combustível, a potência efetiva e a temperatura de exaustão.

O conjunto de dados utilizado na rede neural foi composto por 107 dados, que foram divididos em duas partes, um conjunto para treinamento e outro para teste. Oitenta e um dados foram destinados para treinamento, os outros 26 dados foram

destinados para os testes e validação. A arquitetura da rede neural construída é composta por 5 nós de entradas, uma camada oculta com 7 neurônios, 2 bias, um aplicado individualmente em cada neurônio da camada oculta e um na saída. A Figura 10 ilustra a arquitetura da rede neural utilizada no trabalho. Foram também utilizados dois algoritmos para o treinamento e teste da rede neural artificial, *Levenberg-Marquardt* (LM) e *Scaled Conjugate Gradient* (SCG).

Figura 10 - Arquitetura da Rede Neural Artificial com uma camada oculta e sete neurônios



Fonte: Cay (2013)

Como resultado da pesquisa, o autor atingiu uma taxa de treinamento e validação da rede neural de 0.99 (R^2, *absolute fraction of variance*), um RMS (*Root Mean Square Error*) de 0.02 e MEP (Mean Error Percentage) menor que 2.7%, sendo valores de grande expressividade. Podendo assim concluir a efetividade da rede neural artificial como uma alternativa de modelagem para motores de combustão interna.

4 ESTUDO DE CASO

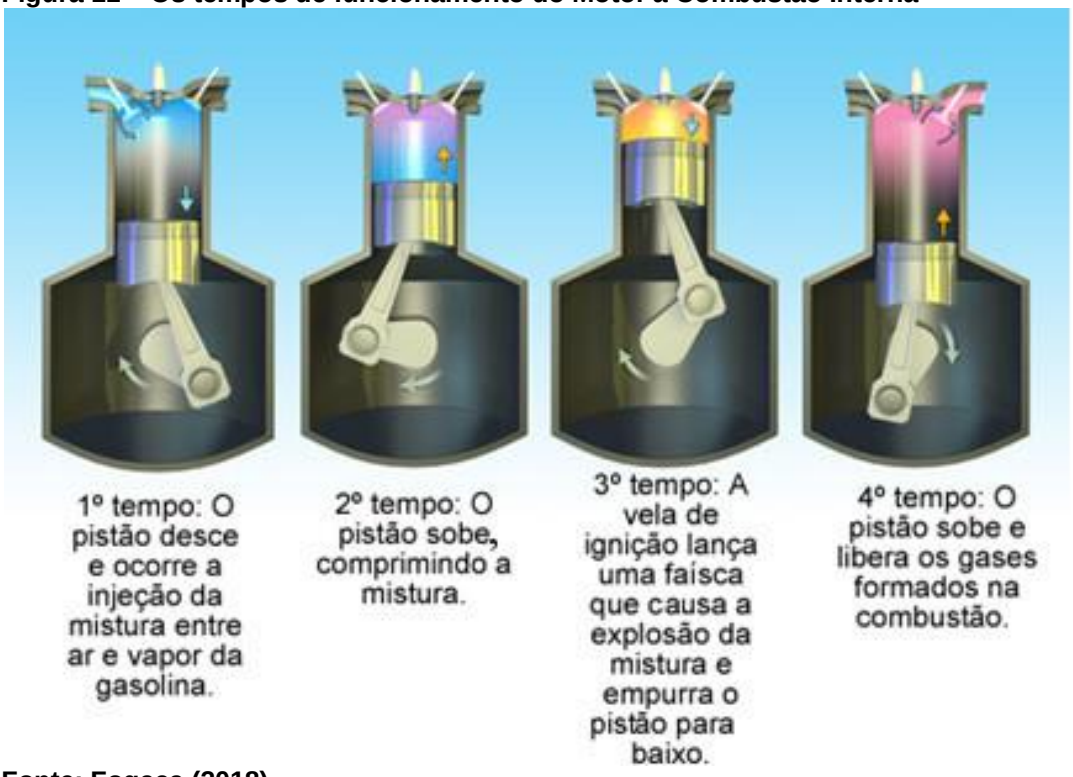
O motor a combustão interna ciclo Otto funciona com ciclo de quatro tempos e trabalha com termodinâmica e princípios de compressão e expansão para gerar força e movimento rotativo (TAYLOR, 1985). O motor foi criado em 1886 por Nikolaus August Otto e conceito de seu funcionamento continua o mesmo atualmente. O princípio do funcionamento do motor a combustão interna é a admissão de ar atmosférica, admissão de combustível do tanque e faz a mistura entre esses dois elementos proporcionalmente comprimindo dentro da câmara de combustão, que posteriormente ocorre todo o processo de inflamação da mistura, que gera explosão e o deslocamento de massa que empurra o pistão para baixo, gerando força, torque e movimento rotativo (TAYLOR, 1985).

Todo o processo de funcionamento do motor depende da melhor proporção possível da mistura de ar atmosférico e combustível, pois a falta ou excesso de qualquer um dos dois elementos químicos causa o mal funcionamento do motor, além de emitir mais poluentes e elevar o consumo de combustível. A condição de mistura ideal entre o ar e combustível é chamada de mistura estequiométrica (PUJATTI, 2007) e denominada valor de riqueza. Manter a condição da riqueza próxima do ideal é uma tarefa complexa na calibração dos motores a combustão interna ciclo Otto.

4.1 OS QUATRO TEMPOS DO MOTOR

Os termos combustão e explosão são usados no nome desse motor porque o seu princípio de funcionamento baseia-se no aproveitamento da energia liberada na reação de combustão de uma mistura de ar e combustível que ocorre dentro do cilindro do veículo. A Figura 11 apresenta uma ilustração dos quatro tempos do motor.

Figura 11 – Os tempos de funcionamento do Motor a Combustão Interna



Fonte: Fogaça (2018)

O motor de 4 tempos funciona pela repetição ordenada de quatro movimentos. Os quatro tempos do motor são:

- Admissão: A válvula de escapamento está fechada e a de admissão se abre progressivamente. O êmbolo desloca do PMS (ponto morto superior) ao PMI (ponto morto inferior), aspirando a mistura ar/combustível.
- Compressão: A válvula de admissão se fecha e a de escapamento continua fechada. O êmbolo inverte seu movimento do PMS ao PMI, comprimindo a mistura na câmara de combustão.
- Combustão: As válvulas continuam fechadas. A mistura comprimida é inflamada por uma centelha que salta da vela de ignição. Com a queima, formam-se gases que se expandem, impulsionando o êmbolo de volta para o PMI.
- Escapamento: A válvula de admissão fica fechada e a de escapamento se abre, progressivamente, à medida que o êmbolo vai do PMI ao PMS, expelindo os gases resultantes da combustão.

Um volante instalado na parte traseira da árvore de manivelas regulariza o funcionamento do motor. A combustão é provocada pela centelha na vela de ignição.

Os cilindros trabalham dentro de uma determinada ordem de combustão e o volante, por ter movimentos de inércia, transforma os impulsos que recebe em um movimento contínuo.

4.2 FORMAÇÃO DA MISTURA AR COMBUSTÍVEL

Nos motores de combustão interna ciclo Otto, o combustível é queimado internamente, transformando energia térmica em energia mecânica, através de um mecanismo construído por pistão, biela e virabrequim. A biela e o virabrequim são os responsáveis pelo movimento rotativo do pistão dentro do cilindro, produzindo energia.

Para o funcionamento do motor, o combustível não é o único elemento químico necessário, nos motores de combustão interna ciclo Otto é necessário uma quantidade de ar e uma quantidade de combustível proporcional, chamada de estequiometria, para que a queima seja realizada e transformada em energia. A admissão de ar no motor é composta por um filtro de ar e seus condutos, pelos componentes do sistema de injeção eletrônica ou pelo carburador, pela válvula borboleta e pelo coletor de admissão (PUJATTI, 2007). Para a admissão de combustível, a quantidade admitida é função direta da quantidade mássica de ar admitida.

A mistura ar-combustível é considerada estequiométrica quando o Oxigênio (O_2) presente na massa de ar admitida é suficiente para reagir com a quantidade de combustível admitida, produzindo na combustão Dióxido de Carbono (CO2), Água (H2O) e gases que são inertes a admissão de ar (PUJATTI, 2007). A proporção ar-combustível é diferente para cada tipo de combustível. A Tabela 1 ilustra a proporção estequiométrica entre os diferentes combustíveis, tomando como referência a concentração de O_2 presente no ar.

Tabela 2 - Relação estequiométrica ar-combustível

Combustível	Proporção Ar-Combustível
Metanol	6,4:1
Etanol (E100)	9,0:1
Gasolina Comum (E22)	13,2:1
Gasolina Pura (E0)	14,7:1
Diesel	15,2:1
Butano (C4H10)	15,4:1
Propano (C3H8)	15,6:1
Metano (CH4)	17,2:1
Hidrogênio	34,0:1

Fonte: Pereira (2001)

A relação da mistura ar-combustível admitida pelo motor é representada pela letra grega lambda:

$$\lambda = \frac{M_{ar}}{M_c} \tag{25}$$

A variável M_{ar} representa a massa de ar admitida pelo motor. A variável M_c representa a massa de combustível admita pelo motor e o λ representa a relação entre a massa de ar e a massa de combustível. Para a mistura da relação ar-combustível ser estequiométrica, o valor de lambda deve ser igual a 1. Quando o valor de lambda é maior que 1, então tem-se uma mistura pobre. Já quando o valor de lambda é menor que 1, então tem-se uma mistura rica.

Existe também outra denominação para a razão estequiométrica, que é chamada de Riqueza e é o inverso do conceito de lambda. Quando o valor da riqueza está acima de 1, então tem-se uma mistura rica, abaixo de 1 uma mistura pobre e assim como o lambda, quando é igual 1 tem-se o valor estequiométrico. Para o presente trabalho, será utilizado o conceito de riqueza, pois este é o conceito usado nos motores onde os ensaios para testes serão realizados.

A mistura pobre é caracterizada pela falta de combustível, dificultando a propagação da chama na câmara de combustão, podendo causar no motor situações de instabilidade e difícil controle de rotação. A mistura rica é o inverso da mistura pobre, quando há excesso de combustível, porém também é prejudicial ao motor,

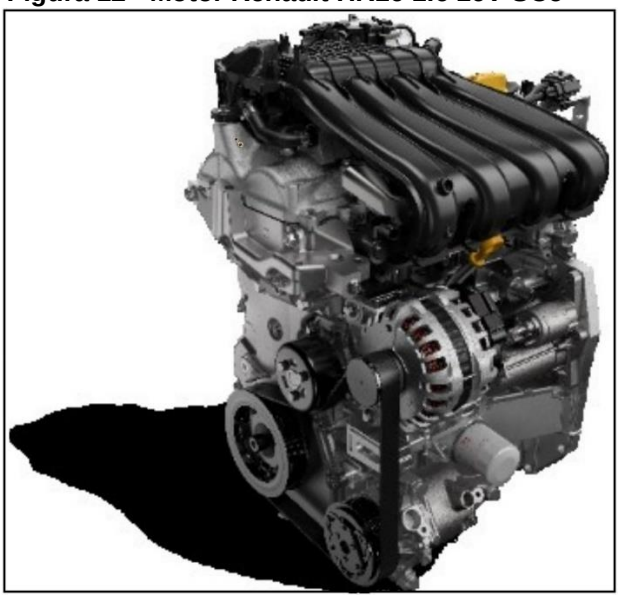
dificulta a propagação da chama e pode causar instabilidade e mal funcionamento do motor.

4.3 ESTUDO DE CASO: MOTOR A COMBUSTÃO INTERNA CICLO OTTO

O estudo de caso utilizado é um dos motores fabricados pela Renault do Brasil, possibilitando a utilização de dados reais e a aplicação do trabalho em um cenário real, porém nem todos os dados e informações do motor podem ser disponibilizados devido a propriedade intelectual, como por exemplo nomes de variáveis da calibração do motor e que são utilizados como entradas da rede neural, como será explicado detalhadamente na seção 6.1.

O motor é um HR16 1.6 16V SCe aspirado de 4 cilindros. O motor entrega 120 cv a 5500 rpm com etanol e 118 cv a 5500 rpm com gasolina e são 16.2 kgfm de torque a 4000 rpm, tanto no etanol quanto na gasolina. Quanto a admissão, é utilizado o Comando de Válvula Variável, mais conhecido como VVT (*Variable Valve Timing*), que consegue alterar o momento de abertura das válvulas ou o quanto elas se abrem para otimizar o rendimento do motor em cada circunstância. O motor HR16 produzido pela Renault possui um VVT, apenas na admissão. O VVT na admissão resulta em ganhos de potência e torque, consumo, suavidade do motor e alguma redução de emissões. Quando também há válvula de escape (que não é o caso do HR16), reduz-se drasticamente as emissões com ganhos de potência e torque, consumo e suavidade do motor (AUSTIN COMMUNITY COLLEGE, 2018).

Figura 12 - Motor Renault HR16 1.6 16V SCe



Fonte: Renault do Brasil

Em relação a injeção de combustível, que está diretamente ligada ao problema do trabalho, o aprendizado da mistura ar/combustível, o motor utiliza o sistema de injeção multiponto (MPI) que injetam combustível em cada cilindro, permitindo maior controle sobre a quantidade de combustível na queima.

No presente trabalho os dados foram restringidos em duas condições de funcionamento do motor. O único combustível utilizado foi o Etanol (E100) e somente foram realizadas aquisições no veículo com o motor quente, acima de 78ºC a temperatura da água do motor. Em relação aos locais de aquisição dos dados, foram realizadas na cidade de Curitiba e em dinamômetros dentro da Renault.

5 RESULTADOS EXPERIMENTAIS

Este capítulo apresenta os resultados do desenvolvimento da dissertação, desde a construção da rede neural, aquisição dos dados, treinamento das redes neurais com os algoritmos, critérios para avaliação do desempenho dos algoritmos e a comparação do desempenho dos algoritmos.

A seção 5.1 apresenta as três arquiteturas desenvolvidas para o treinamento da rede neural. A seção 5.2 apresenta o processo completo de como o treinamento da rede neural é realizado, desde a aquisição dos dados do veículo até a simulação com a rede neural treinada. Da seção 5.3 a seção 5.6 são apresentados os desempenhos individuais de cada algoritmo no estudo de caso. A seção 5.7 apresenta quais os critérios de comparação que serão utilizados para classificar o desempenho dos algoritmos e a seção 5.8 apresenta a comparação do desempenho dos algoritmos em cada uma das arquiteturas da rede neural.

5.1 ARQUITETURA DESENVOLVIDA PARA A REDE NEURAL ARTIFICIAL

O objetivo geral do trabalho é realizar a comparação entre os diferentes algoritmos para treinamento de uma rede neural, apresentando qual a melhor solução para o estudo de caso em que os algoritmos são aplicados. Visando obter melhores resultados, dados e cenários para comparação, foram desenvolvidas 3 arquiteturas de redes neurais diferentes para a aplicação de cada algoritmo.

As 3 arquiteturas desenvolvidas seguem o padrão da MLP, porém a diferença entre elas é a quantidade de neurônios em sua camada oculta. Além de variar a quantidade de neurônios, testes prévios foram realizados variando a quantidade de camadas ocultas, o que deixou a rede complexa e produziu resultados inferiores as redes com uma única camada oculta. Devido a isso, acreditamos que uma única camada oculta é o suficiente para a aplicação no estudo de caso e comparação do resultado entre os diferentes algoritmos de treinamento.

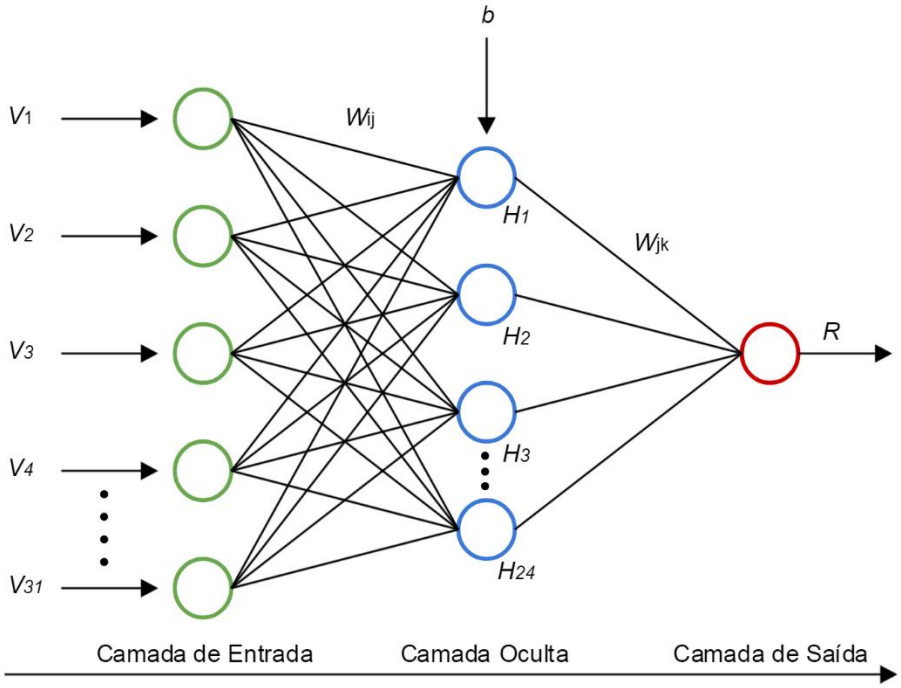
A camada de entrada é idêntica para as 3 arquiteturas, contendo 31 variáveis que se referem as variáveis do motor utilizadas. As 31 variáveis do motor são: embreagem (on/off), corte de injeção, injeção habilitada (on/off), pedal do acelerador

(on/off), fase de partida, estado do motor (se o motor está autônomo ou está desligado), estado do motor com ele autônomo, estado do enriquecimento, pedal do acelerador em porcentagem, eficiência volumétrica do motor, derivada da eficiência volumétrica, massa de combustível injetada, objetivo da massa de combustível, avanço de ignição, valor do VVT de admissão em graus, número de cilindros que estão sem injeção, tempo de injeção no primeiro injetor, valor da borboleta (porcentagem do quanto está aberta), pressão atmosférica, derivada da pressão atmosférica, temperatura do ar na admissão, rotação do motor, massa de ar entrando no cilindro, pressão atmosférica calculada, derivada da pressão atmosférica calculada, valor da sonda de oxigênio, objetivo de riqueza, temperatura da agua do motor, fator de correção para o motor frio, valor da taxa de álcool no tanque e correção da massa de combustível da parede do cilindro. Essas variáveis são utilizadas na mesma ordem em todas as arquiteturas.

A camada oculta da primeira arquitetura é composta por 24 neurônios. O que motivou essa quantidade de neurônios é ter um valor entre 70% a 90% de neurônios em função da quantidade de variáveis de entrada. Sendo assim, como são 31 variáveis de entrada, optamos por ter cerca de 75% da quantidade de entradas, resultando em 24 neurônios. A camada de saída é composta por uma única saída, que é o valor da Riqueza. A Figura 13 ilustra a arquitetura da rede neural.

A legenda das variáveis utilizadas na arquitetura é igual para todas. As variáveis V (V_1 até V_{31}) se referem as 31 variáveis de entrada. A variável W_{ij} representa o peso de cada ligaração entre as variáveis de entrada e os neurônios da camada oculta. A variável H (H_1 até H_n) representa os neurônios da camada oculta. A variável b é o *bias* aplicado a rede neural. A variável W_{jk} representa os pesos dos neurônios da camada oculta para a saída. Por fim, a variável R representa a saída, o objetivo da rede neural.

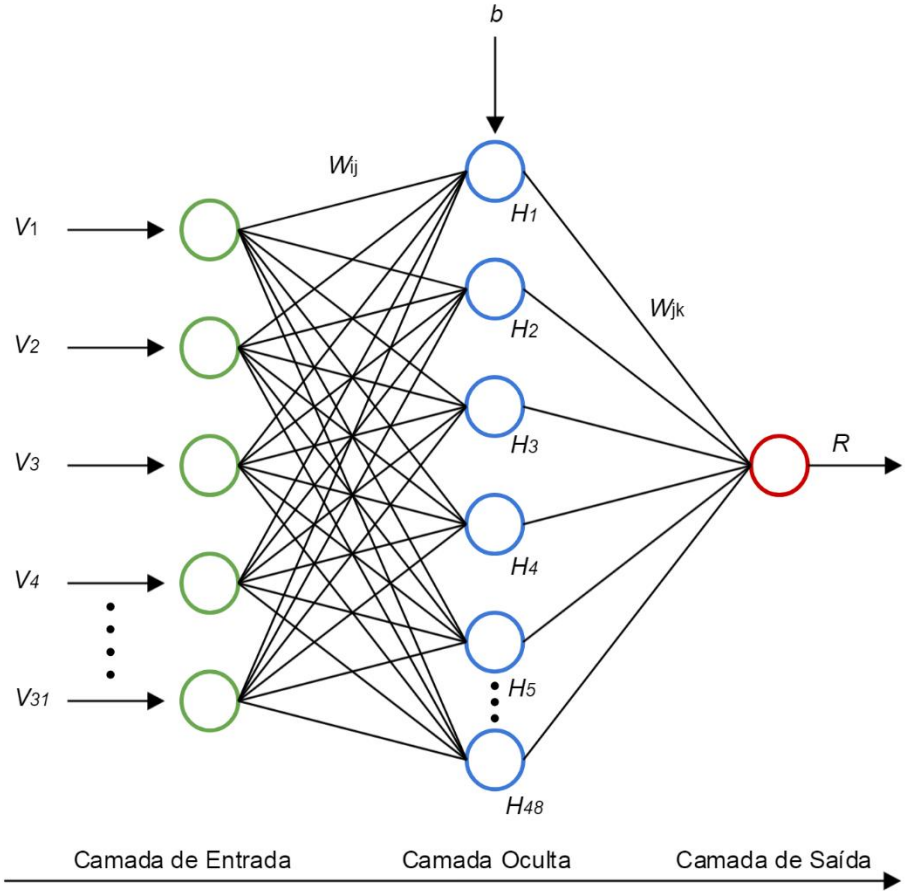
Figura 13 - Arquitetura da RNA com 24 neurônios na camada oculta



Fonte: Autoria própria

A segunda arquitetura segue o padrão da primeira. A camada de entrada é idêntica, com as mesmas 31 variáveis. A camada de saída também é idêntica, com uma única saída, a Riqueza. A diferença entre as duas arquiteturas é a quantidade de neurônios da camada oculta, nessa arquitetura são 48 neurônios na camada oculta, como apresentado na Figura 14. Os 48 neurônios também são em função da quantidade de entradas, porém nesse caso temos 50% a mais de neurônios da camada oculta em função das variáveis de entrada. Sendo assim, multiplicamos a quantidade de entrada pelo valor 1.5, resultando nos 48 neurônios da camada oculta.

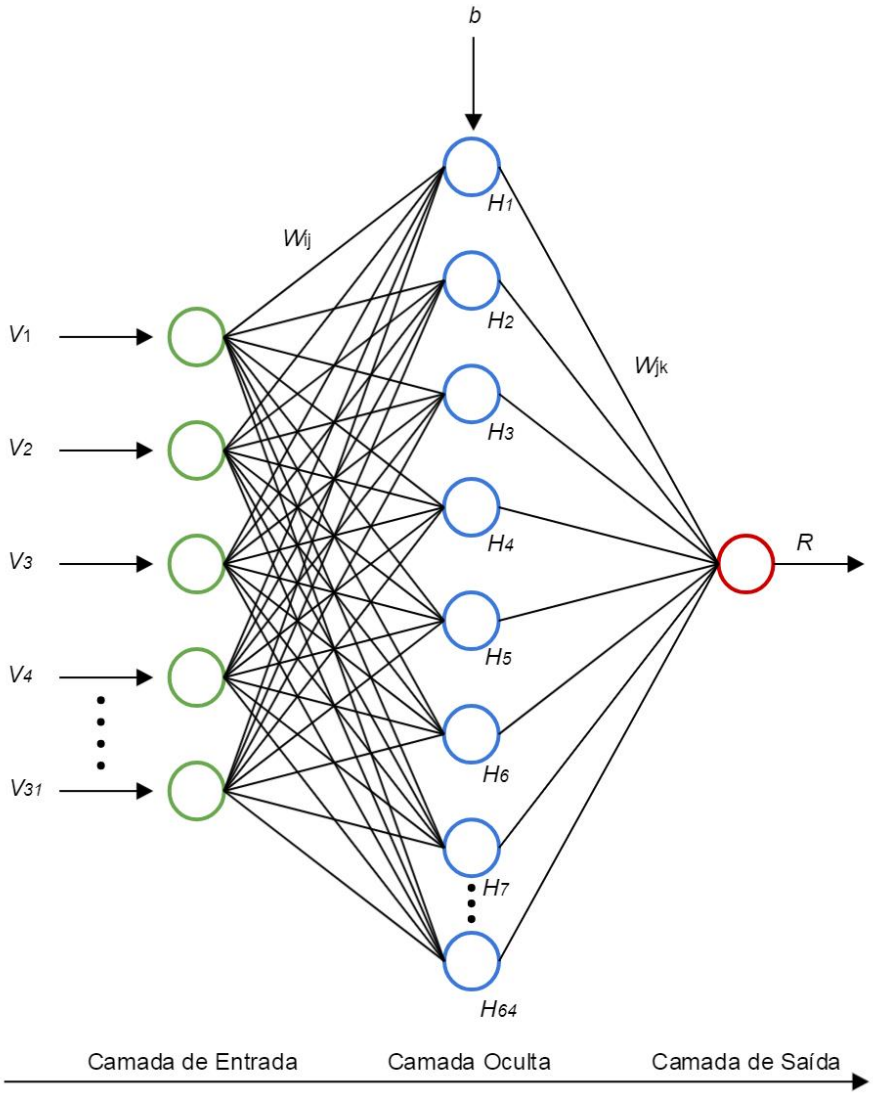
Figura 14 - Arquitetura da RNA com 48 neurônios na camada oculta



Fonte: Autoria própria

A terceira arquitetura segue o mesmo padrão das duas últimas e também varia a quantidade de neurônios da camada oculta, como apresentado na Figura 15. Nesse caso são utilizados 62 neurônios na camada oculta. Os 62 neurônios também são em função do número de entradas e nesse caso são o dobro.

Figura 15 - Arquitetura da RNA com 64 neurônios na camada oculta



Fonte: Autoria própria

Todas as arquiteturas utilizadas são em função da quantidade de entradas, pois para a aplicação do estudo de caso, é a camada da rede neural que mais pode ser modificada, incluindo ou excluindo variáveis de entrada que estejam influenciando na geração do valor da Riqueza. Portanto, como as arquiteturas são em função da quantidade de entrada, mesmo que as entradas sejam modificadas, a rede neural se mantém no mesmo padrão teoricamente.

5.2 PROCESSO PARA A REALIZAÇÃO DOS TREINAMENTOS

O processo completo para realização do treinamento da rede neural envolve 3 etapas principais, ilustrado na Figura 16. Todo o processo é demorado e varia conforme o tempo para aquisição dos dados do veículo e o tempo de treinamento necessário para a rede neural.

Figura 16 - Etapas para realização dos treinamentos da rede neural



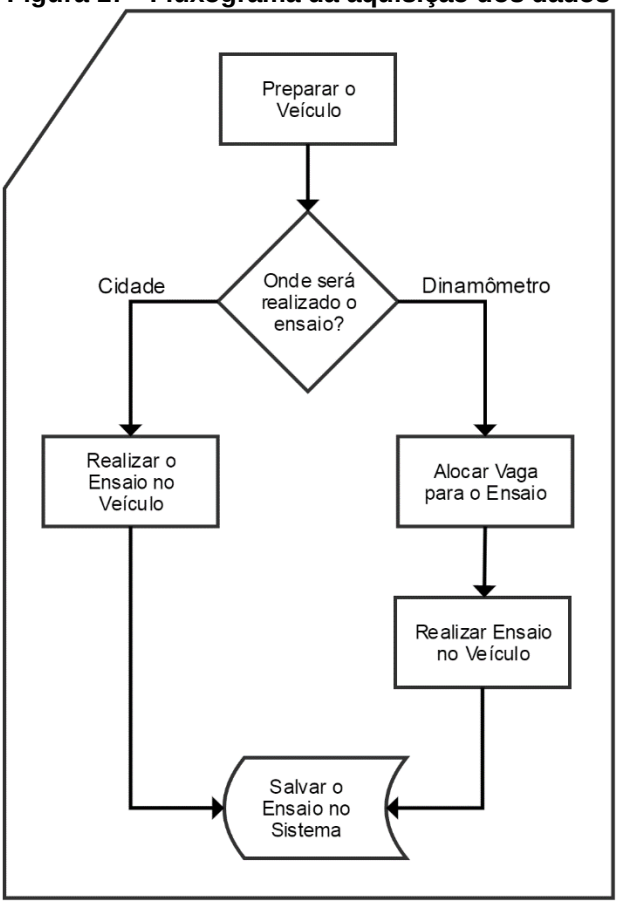
Fonte: Autoria própria

A primeira etapa para realizar o processo de treinamento é a aquisição dos dados do veículo. Este processo é ilustrado no fluxograma da Figura 17. O primeiro passo é instrumentar o veículo. A instrumentação serve para que seja possível coletar os dados do veículo e para que os dados coletados sejam confiáveis.

Os ensaios podem ser realizados em um ambiente controlado ou em um ambiente externo. No ambiente controlado os ensaios são realizados no dinamômetro, onde um motorista especializado irá simular um trajeto de percurso padrão do veículo e os dados serão aquisitados. Em nosso cenário, essa coleta de dados demora mais tempo para ser realizada, pois é necessária alocação de uma vaga do dinamômetro para que o ensaio ser realizado. No ambiente externo os ensaios são realizados na cidade e são mais fáceis e rápidos de serem realizados. Com o veículo bem instrumentado, não há diferença na qualidade dos dados entre os dois ensaios. Tanto o dinamômetro e a cidade produzem dados reais e de qualidade para o treinamento da rede neural.

Em nosso trabalho tanto os dados para treinamento quanto os dados para validação foram coletados do veículo no ambiente controlado, para garantirmos a consistência e perfil de condução do veículo. Sendo assim, o mesmo veículo, com o mesmo motor e a mesma instrumentação foram guiados pelo mesmo motorista no com um mesmo perfil de condução, a única diferença entre os dados de treinamento e de validação é a quantidade, pois para o treinamento é utilizado um conjunto de dados maior do que na validação.

Figura 17 - Fluxograma da aquisição dos dados do veículo



Fonte: Autoria própria

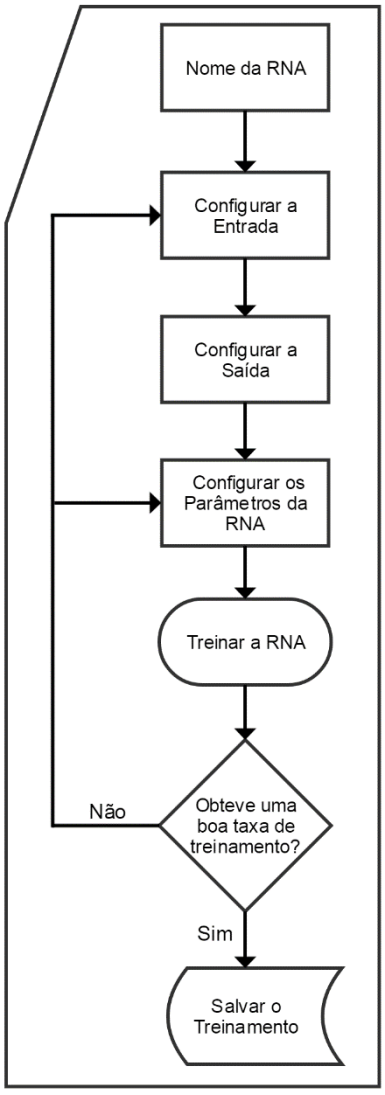
A segunda etapa do processo é o treinamento da rede neural. O treinamento é realizado com a ferramenta descrita no Apêndice A, o que facilita a manipulação e pré-tratamento dos dados, além de agilizar a configuração da rede neural. A Figura 18 apresenta o fluxograma para o treinamento da rede neural.

Todos os treinamentos dos algoritmos devem ser realizados na mesma configuração de hardware, para garantir uma comparação correta entre os algoritmos. No presente trabalho o processador utilizado foi um AMD PRO A4-8350B, com 6 *cores* (2 CPU e 4 GPU) com uma velocidade de 3.5 Gigahertz, podendo chegar até 3.9 Gigahertz. A memória utilizada foi de 8 Gigabytes com uma frequência de 1600 Megahertz. O sistema operacional foi o Windows 7 Enterprise de 64 bits. Não foi utilizado nenhuma memória dedicada de vídeo.

O primeiro passo do processo é definir o nome da rede neural. A definição do nome é importante pois é utilizada para salvar o treinamento e consequentemente distinguir as redes. O segundo passo é configurar o valor das variáveis de entrada da rede. A configuração dos treinamentos envolve o cálculo da derivada de algumas variáveis e a aplicação de *delay* de 150ms, pois na coleta dos dados no veículo existe

um atraso entre os valores das variáveis e o valor da riqueza. Outros tratamentos de dados foram aplicados em testes anteriores da rede, removendo alguns valores da aquisição do veículo, porém esses tratamentos de dados não resultaram em um bom resultado e não serão utilizados no treinamento das redes neurais.

Figura 18 - Fluxograma do treinamento da RNA



Fonte: Autoria própria

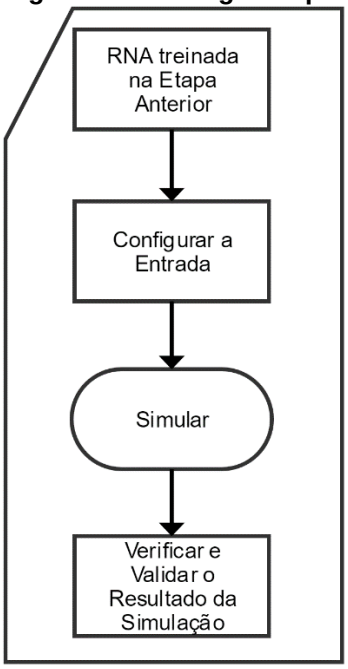
O terceiro passo do treinamento é configurar os dados de saída. A saída é composta somente por uma variável. As aquisições realizadas no veículo não são com o valor da riqueza, são com o valor do lambda, explicado na seção 4.2. Como utilizamos o valor da riqueza, então é necessário preparar a saída transformando o valor de lambda em riqueza. O quarto passo é a configuração dos parâmetros da rede neural. A configuração da rede não é única e varia para as diferentes arquiteturas e

algoritmos de treinamento utilizados no trabalho. As configurações utilizadas foram apresentadas na seção 5.1.

Após realizar os treinamentos é necessário analisar as taxas de treinamento. É possível analisar se obteve uma boa taxa de treinamento de diferentes maneiras, como pelo valor da regressão, pela taxa de erro, pela taxa de erro entre o valor real e a saída, entre outros. A seção 5.5 aborda esse assunto mais profundamente, apresentando os critérios para medir o desempenho entre as redes neurais. No fluxograma apresentado na Figura 18, é recomendado que se não houver boas taxas de treinamento, retorne para a etapa de configuração dos dados de entrada ou configurações dos parâmetros da RNA, pois são as duas etapas que influenciam diretamente no valor da taxa de treinamento. Após o treinamento é necessário salvar a rede neural para que ela possa ser utilizada na etapa seguinte, de simulação com a rede neural treinada.

A terceira e última etapa do processo de treinamento é validar a rede neural realizando simulações com outros conjuntos de dados aquisitados do veículo. A Figura 19 ilustra essa última etapa. O primeiro passo é escolher qual das redes neurais treinadas será utilizada na simulação. Dentre as redes neurais disponíveis, estão aquelas que já foram treinadas e salvas na etapa anterior. O segundo passo é configurar a entrada que segue o mesmo processo da etapa anterior, porém é necessário se atentar que as variáveis de entrada para a simulação devem ser as mesmas e estar na mesma ordem das variáveis utilizadas no treinamento da rede neural que será utilizada para simular. Em nosso processo temos as 31 variáveis fixas, independente da configuração da rede neural que está sendo utilizada, somente precisamos nos atentar em manter sempre a mesma ordem das variáveis.

Figura 19 - Fluxograma para validação da RNA



Fonte: Autoria própria

Diferentemente da etapa anterior, nessa etapa não é necessário configurar a saída, pois não há um *target*, a rede neural irá simular o valor da riqueza apenas com o que aprendeu e com os dados de entrada.

A simulação irá produzir um valor de riqueza com base nos dados de entrada fornecidos. Após a simulação é necessário verificar e validar se o valor da riqueza produzido pela rede neural é confiável, contendo uma baixa taxa de erros. A seção 5.5 explica detalhadamente os critérios para realizar a verificação e validação da simulação da rede neural.

5.3 DESEMPENHO DO ALGORITMO LEVENBERG-MARQUARDT NO ESTUDO DE CASO

O conjunto de dados para aplicação das arquiteturas na etapa de treinamento é grande quando comparado com os trabalhos relacionados da seção 3. O conjunto de dados para o treinamento contém 482855 mil dados. Destes dados, 70% são destinados para treinamento, 15% para validação e 15% para testes. A divisão dos dados é feita de forma randômica. O conjunto de dados para simulação contém 76192 mil dados, e estes são utilizados para validar a rede realizando simulações com a rede treinada.

5.3.1 Levenberg-Marquardt com 24 Neurônios na Camada Oculta

Os resultados do treinamento com o algoritmo na primeira arquitetura (24 neurônios na camada oculta) estão apresentados na Tabela 3. Em cada iteração do treinamento a rede neural busca minimizar o valor do MSE e aumentar o valor da regressão, convergindo para um melhor treinamento. Cada iteração é conhecida como época, uma época equivale a uma iteração da rede neural. A época que a rede atinge seu melhor resultado de convergência é chamada de Melhor Época.

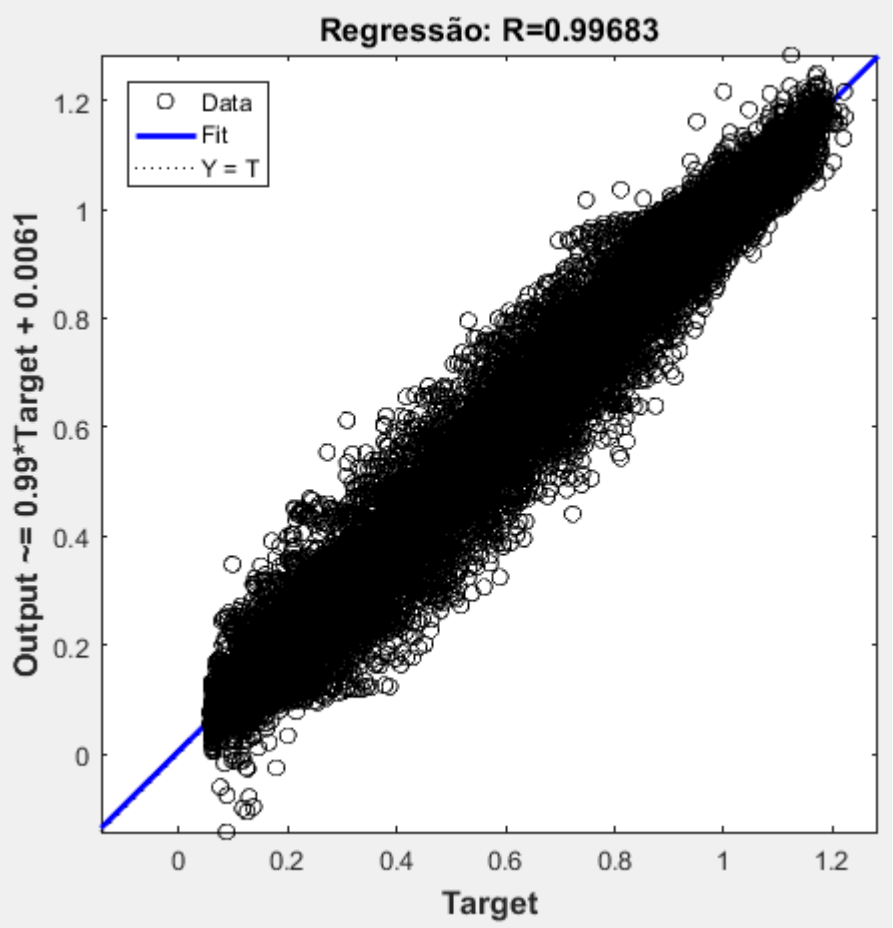
Tabela 3 - Informações sobre o treinamento com 24 neurônios

Informações do Treinamento	Dados
MSE	2.3840e-04
Regressão	0.99683
Épocas	204
Melhor Época	198
Tempo de Treinamento	2h e 27min

Fonte: Autoria própria

Para garantir que a rede não consiga atingir uma melhor convergência, sempre são realizadas mais 6 iterações (épocas) a partir a melhor época, logo sempre haverá a diferença de 6 iterações entre o total de épocas e a melhor época. O total de 6 épocas é um padrão da Toolbox de Rede Neural do MATLAB utilizado no desenvolvimento da dissertação, podendo variar conforme o problema, o total de épocas necessárias é apresentado melhor por (PRECHELT, 1998). Porém há um limite de épocas estipulado para que a rede não fique tentando convergir para sempre. O limite é de 1000 épocas, quando a rede atinge esse limite, o algoritmo encerra o treinamento e assume como melhor resultado o valor do estado atual de convergência. Nesses casos não haverá a diferença de 6 iterações entre o total de épocas e a melhor época.

Gráfico 4 - Taxa de Regressão para o LM com 24 neurônios



Fonte: Autoria própria

O Gráfico 4 apresenta o valor da regressão em forma de gráfico. A linha azul é o objetivo da regressão (treinamento), a linha tracejada é a relação do quão próximo o treinamento conseguiu chegar do seu objetivo, tendendo a se aproximar da linha azul. Os círculos são os dados do treinamento. Cada círculo representa um ponto do conjunto de dados, sendo assim, como temos um conjunto de dados de 482.855 mil, então teremos essa mesma quantidade de dados.

A seção 5.7 apresenta detalhadamente qual são os critérios para comparar todos os algoritmos em todas as arquiteturas, os valores expressos na Tabela 4 e Tabela 5 são os de maiores pesos na comparação. A Tabela 4 apresenta as taxas de erro e a taxa de acerto do algoritmo no treinamento.

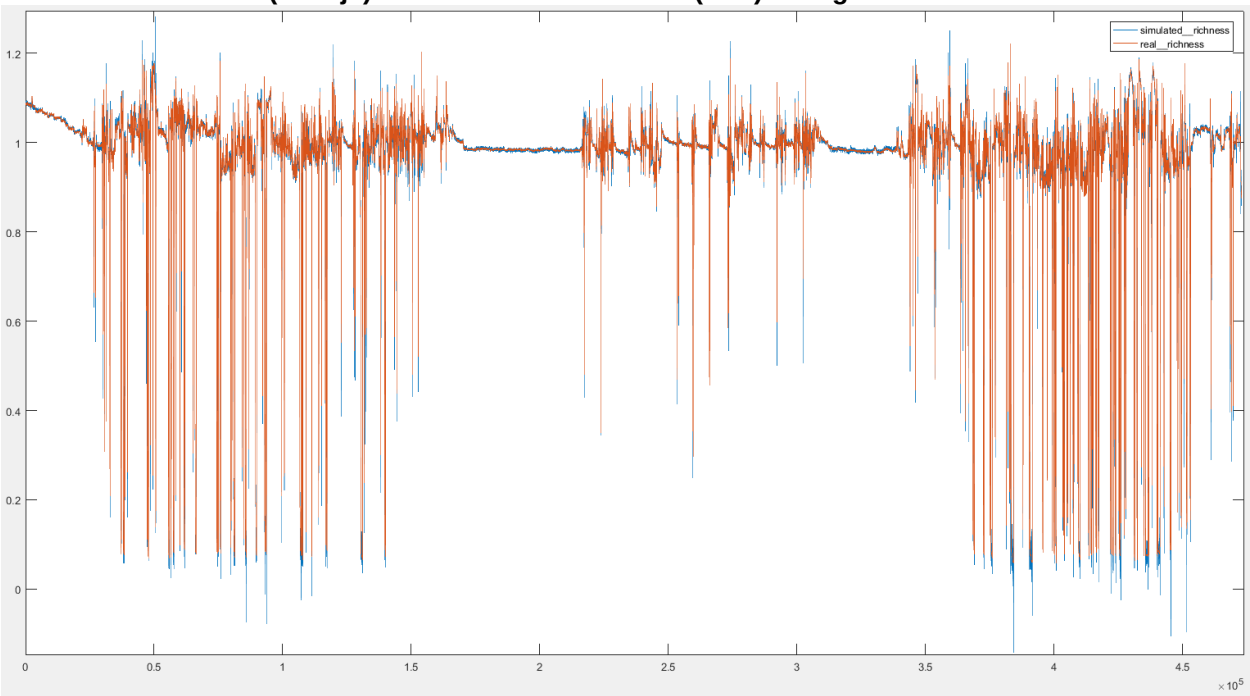
Tabela 4 - Taxas de erro e taxa de acerto do algoritmo LM no treinamento com 24 neurônios

ε ≤ 1%	ε ≤ 3%	ε ≤ 5%	ε ≤ 10%	ε > 10%	Taxa de Acerto
82.08	13.87	2.33	1.32	0.40	95.95%

Fonte: Autoria própria

Uma das maneiras de verificar visualmente a taxa de acerto entre a riqueza real e a riqueza simulada é graficamente. O Gráfico 5 apresenta o gráfico do valor real da riqueza e do valor gerado pela rede neural. Os valores reais estão em laranja e os valores da rede neural estão em azul. A grande quantidade de pontos dificulta a visualização do gráfico, porém é possível observar que o valor da rede neural segue na maioria dos pontos o valor real, porém ainda há a dispersão em alguns pontos, como por exemplo entre as regiões 3.5 a 4.5 do gráfico.

Gráfico 5 - Valor real (laranja) e o valor do treinamento (azul) no algoritmo LM com 24 neurônios



Fonte: Autoria própria

Assim como as taxas de erro e acerto no treinamento, tempos as taxas de erros e acerto na simulação, como apresentado na Tabela 5. A taxa na simulação é calculada com um novo conjunto de dados não conhecido pela rede neural, para poder validar o quanto a rede conseguiu aprender em seu treinamento e reproduzir seu treinamento na simulação. O novo conjunto de dados da simulação é composto por 76192 mil pontos e nessa etapa informamos apenas os valores das variáveis de entrada, sem informar o valor da saída, fazendo com que a rede gere sozinha esse valor. A forma de cálculo da Tabela 5 é a mesma da Tabela 4, a diferença é a utilização de um novo conjunto de dados e o erro é gerado pela saída produzida pela rede neural (simulação) subtraindo o valor real desejado (objetivo) da simulação.

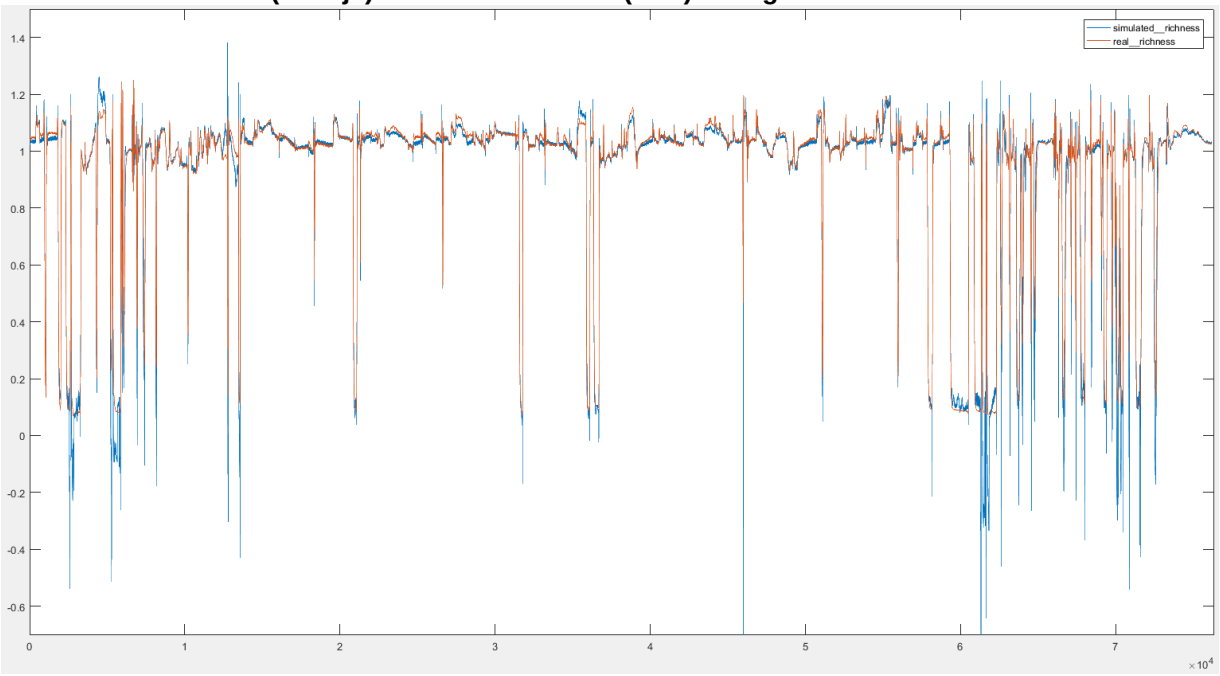
Tabela 5 - Taxas de erro e taxa de acerto do algoritmo LM na simulação com 24 neurônios

ε ≤ 1%	ε ≤ 3%	ε ≤ 5%	ε ≤ 10%	ε > 10%	Taxa de Acerto
54.06	30.52	5.63	4.66	5.13	84.58%

Fonte: Autoria própria

Como explicado anteriormente, a taxa de treinamento não reflete diretamente na taxa de simulação (que em nosso trabalho é a fator de maior relevância na comparação entre os algoritmos, como será explicado na seção 5.7). Nesse caso, há uma diferença de 11,37% entre a taxa de acerto no treinamento e a taxa de acerto na simulação, ainda assim, uma taxa de acerto de 84,58% na simulação é considerada alta e aceita para o estudo de caso em que o algoritmo é aplicado.

Gráfico 6 - Valor real (laranja) e o valor simulado (azul) no algoritmo LM com 24 neurônios



Fonte: Autoria própria

O Gráfico 6 apresenta a relação entre o valor real (laranja) e o valor simulado (azul). É possível observar que na grande maioria dos pontos a simulação segue fiel aos valores reais, porém há muitos pontos com picos de diferença, como entre o intervalo 0 e 1 e o intervalo entre 6 e 7. Como estamos trabalhando com dados reais aquisitados no veículo, há diferença entre as aquisições, pois cada motorista tem um padrão de condução e pode haver condições em que a rede não foi exposta nos dados utilizados no treinamento. Para corrigir esses picos é necessário analisar caso a caso na aquisição do veículo e tentar identificar as diferenças nos dados do treinamento e nos dados da simulação, porém esse não é o foco da dissertação, nosso objetivo é

comparar o desempenho dos diferentes algoritmos, expondo todos as mesmas condições de treinamento e simulação.

5.3.2 Levenberg-Marquardt com 48 Neurônios na Camada Oculta

O algoritmo LM aplicado na segunda arquitetura com 48 neurônios na camada oculta produziu resultados consideravelmente diferentes quando comparado com a aplicação na arquitetura anterior, que contém metade dos neurônios. A Tabela 6 apresenta as informações referentes ao treinamento do algoritmo.

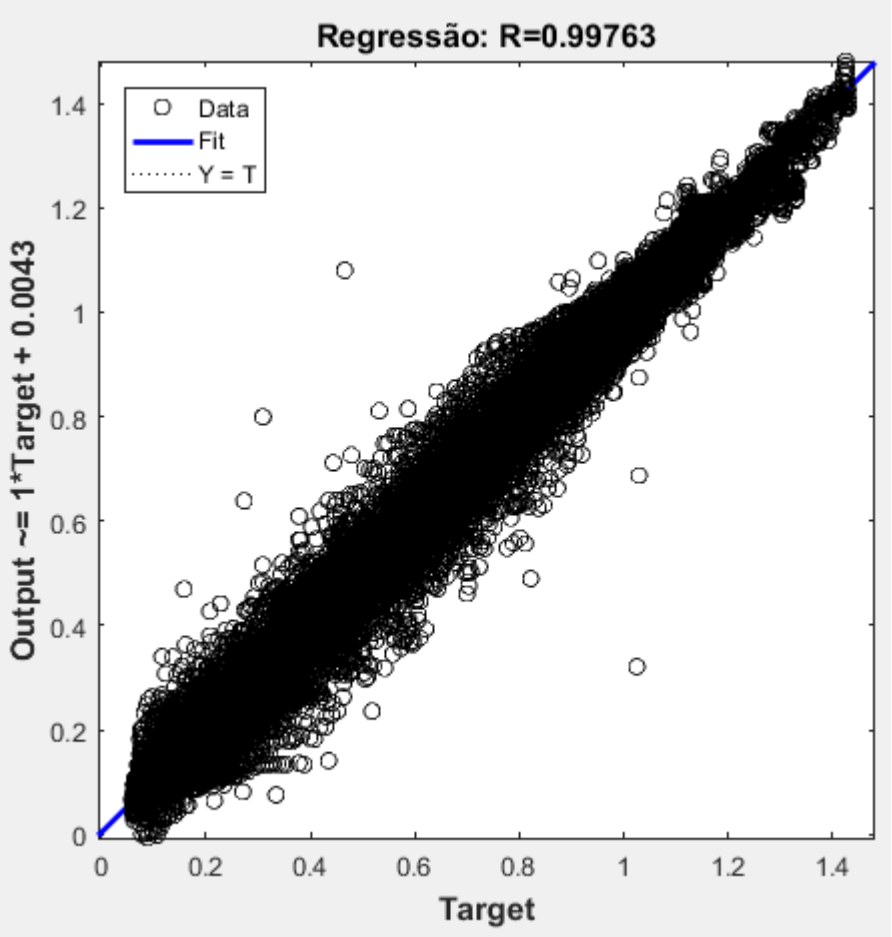
Tabela 6 - Informações sobre o treinamento com 48 neurônios

Informações do Treinamento	Dados
MSE	1.7276e-04
Regressão	0.99763
Épocas	466
Melhor Época	460
Tempo de Treinamento	19h e 58min

Fonte: Autoria própria

A primeira diferença que é possível analisar é a grande diferença entre o tempo para a convergência, que na arquitetura com 48 neurônios foi de quase 20 horas, contra 2h e 27min da arquitetura anterior. O aumento do tempo de treinamento já era esperado, pois o número de neurônios dobrou, e consequentemente o tempo também tende a aumentar.

Gráfico 7 - Taxa de Regressão para o LM com 48 neurônios



Fonte: Autoria própria

O tempo de treinamento foi maior, em contrapartida o valor do MSE melhorou, obtendo 1.7276e-04 contra 2.3840e-04 da primeira arquitetura e o valor da regressão também melhorou, obtendo 0.99763 contra 0.99683 da primeira arquitetura. O gráfico de regressão dessa arquitetura é apresentado no Gráfico 7.

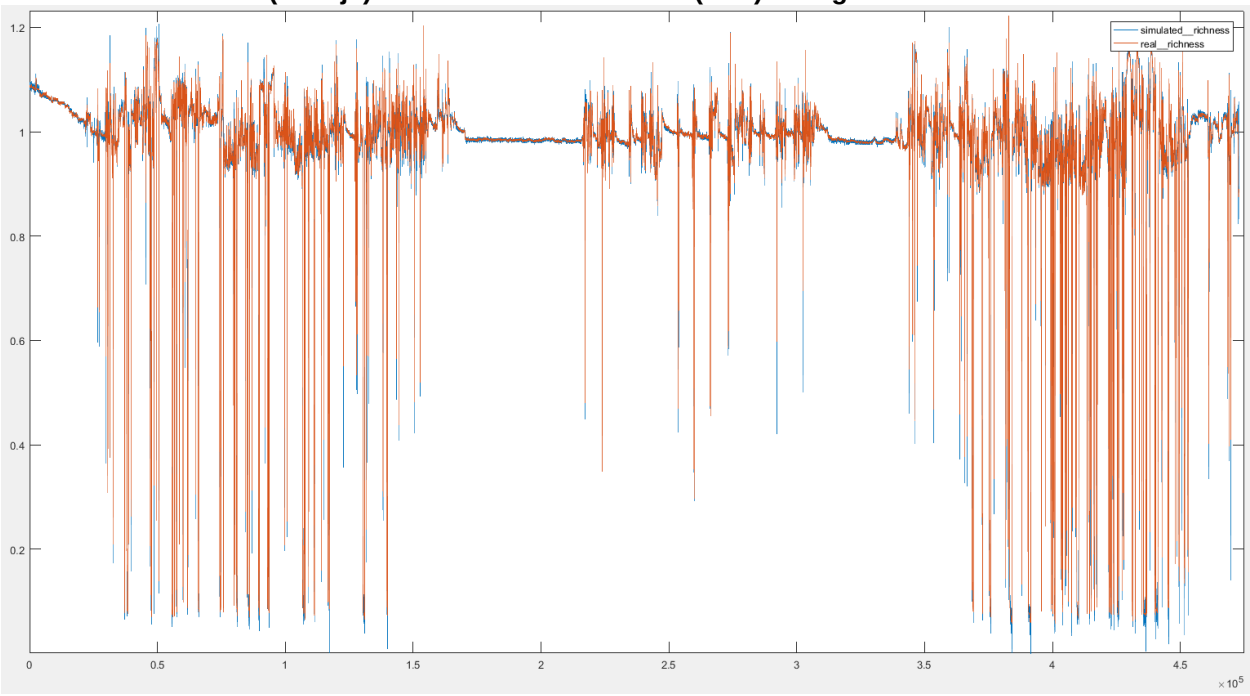
Tabela 7 - Taxas de erro e taxa de acerto do algoritmo LM no treinamento com 48 neurônios

ε ≤ 1%	ε ≤ 3%	ε ≤ 5%	ε ≤ 10%	ε > 10%	Taxa de Acerto
85.68	11.32	1.72	1.04	0.24	97%

Fonte: Autoria própria

Comparando as taxas de erro e a taxa de acerto com a primeira arquitetura, é possível observar que a taxa de erro ε ≤ 1% aumentou 3,6%, o que é algo positivo. As outras taxas de erro não tiveram mudanças significativas que impactassem a taxa de acerto. Em comparação, a taxa de acerto aumentou para 97%, variando 1,05% em relação a arquitetura anterior.

Gráfico 8 - Valor real (laranja) e o valor do treinamento (azul) no algoritmo LM com 48 neurônios



Fonte: Autoria própria

A Gráfico 8 apresenta o gráfico entre o valore real da riqueza (laranja) e o valor produzido pela rede neural no treinamento (azul). É possível observar que a rede conseguiu produzir um ótimo resultado, seguindo na maioria dos pontos fielmente o valor real, tendo apenas alguns pequenos picos de diferença, como no ponto 3.

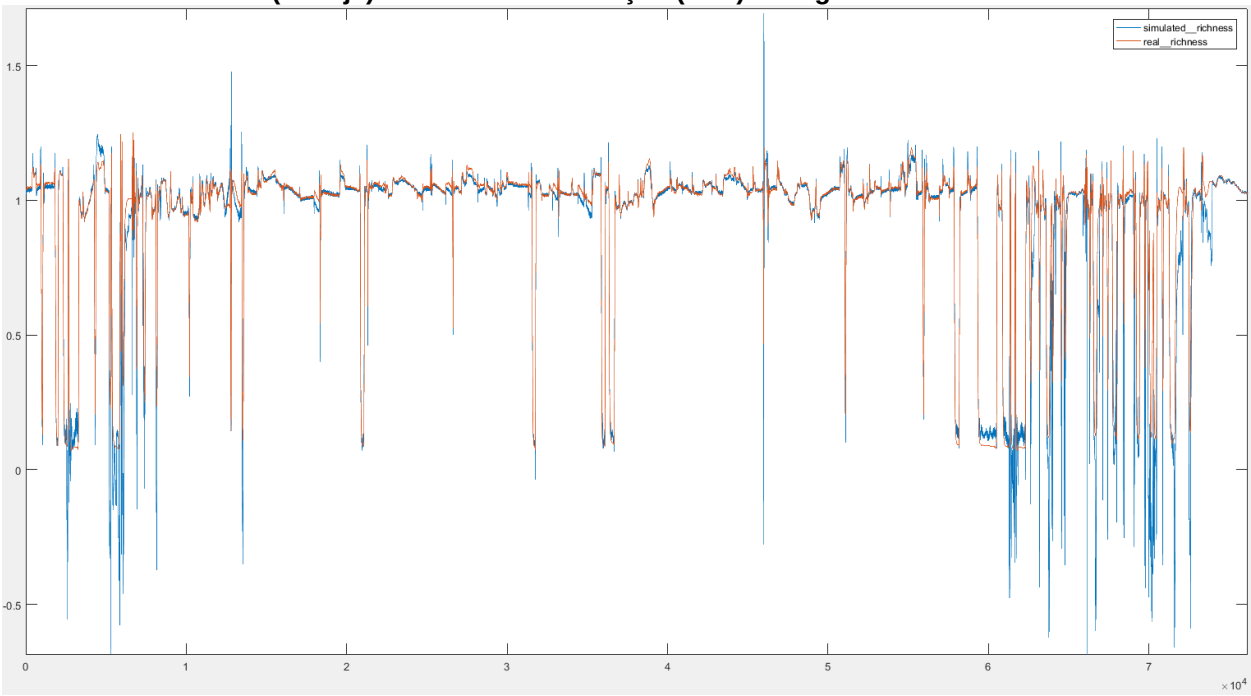
Tabela 8 - Taxas de erro e taxa de acerto do algoritmo LM na simulação com 48 neurônios

ε ≤ 1%	ε ≤ 3%	ε ≤ 5%	ε ≤ 10%	ε > 10%	Taxa de Acerto
56.79	22.74	7.57	5.54	7.36	79.53%

Fonte: Autoria própria

Como apresentado na Tabela 6 e na Tabela 7, os valores referentes ao MSE, Regressão e Taxa de Acerto no treinamento aumentaram em comparação com a primeira arquitetura, porém esse aumento no treinamento não refletiu na simulação, pelo contrário, a taxa de acerto na simulação diminuiu 5,05%, um valor muito considerável, como apresentado na Tabela 8. Sendo assim, na simulação a primeira arquitetura com 24 neurônios foi melhor que a segunda arquitetura com 48 neurônios.

Gráfico 9 - Valor real (laranja) e o valor da simulação (azul) no algoritmo LM com 48 neurônios



Fonte: Autoria própria

A Gráfico 9 apresenta o gráfico entre o valor da riqueza real (laranja) e o valor da riqueza simulada (azul). Em relação a primeira arquitetura, é possível observar claramente o aumento os picos de erros, principalmente entre os pontos 0 e 1 e do ponto 6 em diante.

5.3.3 Levenberg-Marquardt com 64 Neurônios na Camada Oculta

A terceira arquitetura com 64 neurônios conseguiu atingir um melhor valor de MSE em comparação com as duas primeiras arquiteturas, como apresentado na Tabela 9. Obteve também a maior taxa de regressão. O tempo de treinamento teve uma diferença de mais de 24h comparado com a primeira arquitetura e pouco mais de 7h comparado com a segunda arquitetura. É interessante observar que mesmo o tempo aumentando mais de 7h em relação a segunda arquitetura, a quantidade de épocas necessárias para realizar o treinamento do algoritmo foi menor, sendo que a diferença é de 112 épocas.

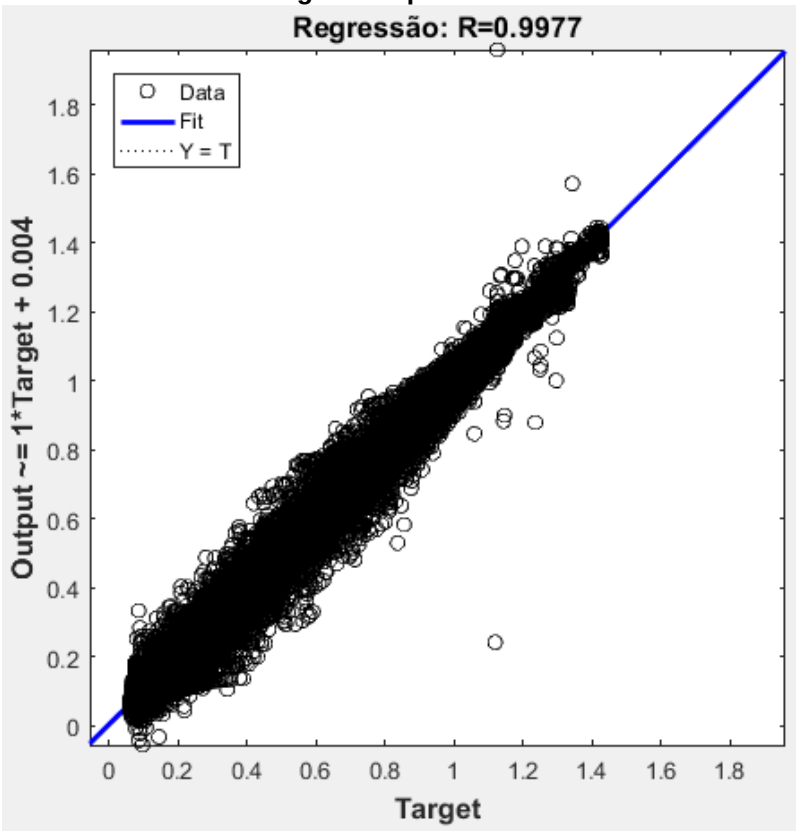
Tabela 9 - Informações sobre o treinamento com 64 neurônios

Informações do Treinamento	Dados
MSE	1.6781e-04
Regressão	0.9977
Épocas	354
Melhor Época	348
Tempo de Treinamento	27h e 04min

Fonte: Autoria própria

O valor da regressão em comparação com a arquitetura anterior aumentou pouquíssimo, não sendo de grande expressão. O Gráfico 10 apresenta o gráfico da taxa de regressão do treinamento do algoritmo LM com 64 neurônios na camada oculta.

Gráfico 10 - Taxa de Regressão para o LM com 64 neurônios



Fonte: Autoria própria

As taxas de erro e a taxa de simulação são bem parecidas com a arquitetura anterior, variando 1,03% em relação a primeira arquitetura e apenas 0,02% em relação a segunda arquitetura, como apresentado na Tabela 10.

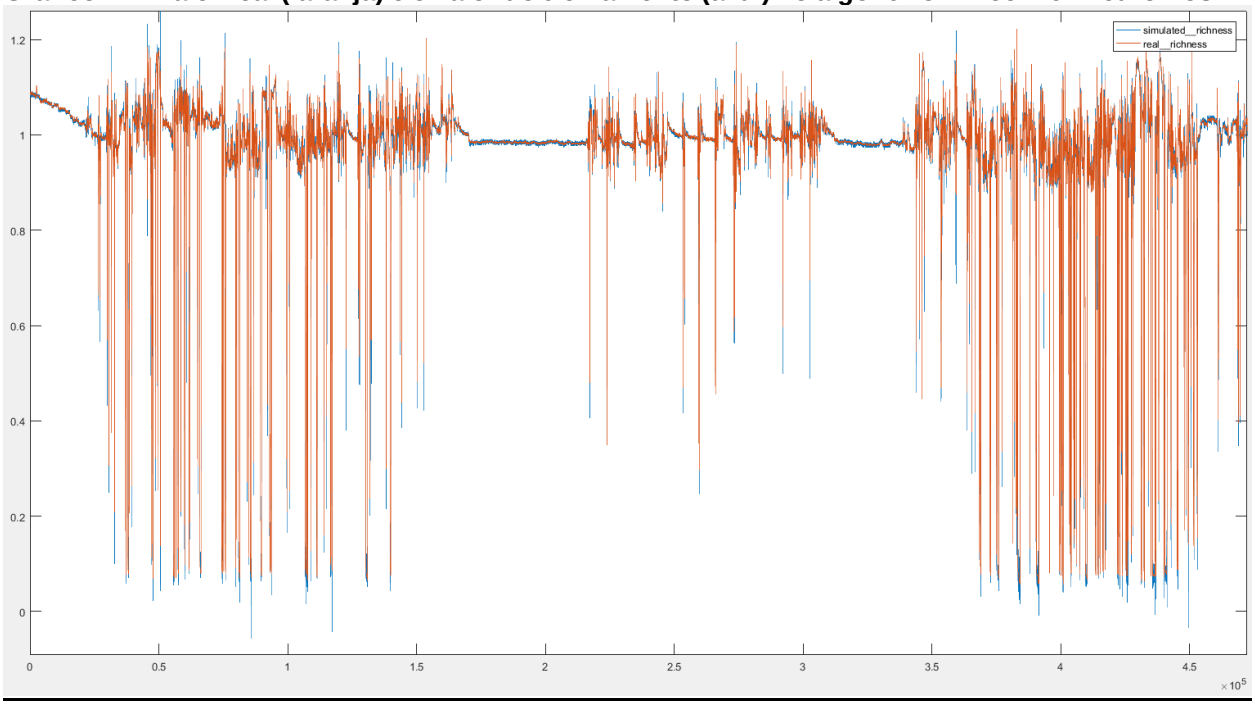
Tabela 10 - Taxas de erro e taxa de acerto do algoritmo LM no treinamento com 64 neurônios

ε ≤ 1%	ε ≤ 3%	ε ≤ 5%	ε ≤ 10%	ε > 10%	Taxa de Acerto
85.59	11.39	1.81	0.98	0.23	96.98%

Fonte: Autoria própria

O Gráfico 11 apresenta o gráfico entre o valore real da riqueza (laranja) e o valor do treinamento gerado pela rede neural (azul). Como não houve grandes variações entre a primeira e a segunda arquitetura, o gráfico se mantém bem parecido com as duas últimas e seguindo fielmente o valor real, tendo apenas alguns pequenos picos de maiores erros.

Gráfico 11 - Valor real (laranja) e o valor do treinamento (azul) no algoritmo LM com 64 neurônios



Fonte: Autoria própria

Em relação as taxas de erro e a taxa de acerto na simulação, comparando com a primeira arquitetura o valor da taxa de acerto diminuiu 3,83%, ocorrendo o mesmo quando a segunda arquitetura foi comparada com a primeira, onde os valores de treinamento como MSE e regressão aumentaram, porém, a taxa de simulação diminuiu. A Tabela 11 apresenta os valores referentes a simulação. Em relação a segunda arquitetura, houve um aumento da taxa de simulação de 1,22%. Sendo assim, a primeira arquitetura ainda continua com o melhor valor de simulação entre as 3 arquiteturas com o algoritmo LM.

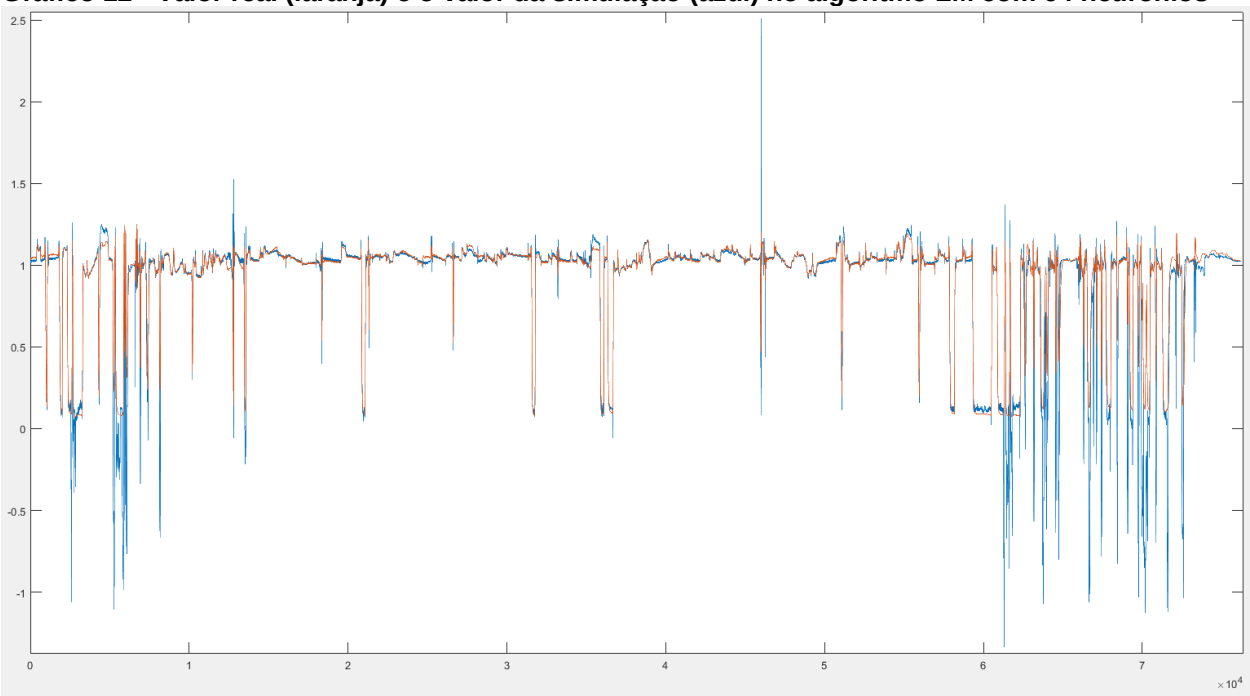
Tabela 11 - Taxas de erro e taxa de acerto do algoritmo LM na simulação com 64 neurônios

ε ≤ 1%	ε ≤ 3%	ε ≤ 5%	ε ≤ 10%	ε > 10%	Taxa de Acerto
51.48	29.27	6.34	5.64	7.27	80.75%

Fonte: Autoria própria

O Gráfico 12 apresenta o gráfico dos valores reais (laranja) e os valores gerados na simulação pela rede neural (azul). Os erros que aconteciam nas duas primeiras arquiteturas se manteram, porém o pico de erro entre os pontos 4 e 5 inverteu, anteriormente era para baixo e agora subiu.

Gráfico 12 - Valor real (laranja) e o valor da simulação (azul) no algoritmo LM com 64 neurônios



Fonte: Autoria própria

5.4 DESEMPENHO DO ALGORITMO BAYESIAN REGULARIZATION NO ESTUDO DE CASO

O conjunto de dados do treinamento para o algoritmo Bayesian Regularization é o mesmo utilizado no algoritmo Levenberg-Marquardt, com 482855 mil dados no treinamento divididos de forma randômica (70% treinamento, 15% testes e 15% validação) e 76192 mil dados para a simulação da rede neural.

5.4.1 Bayesian Regularization com 24 Neurônios na Camada Oculta

A primeira arquitetura com o algoritmo de treinamento BR parou o treinamento pelo limite de épocas, sendo assim o algoritmo não conseguiu convergir rapidamente, resultado em um alto tempo de treinamento, como apresentado na Tabela 12.

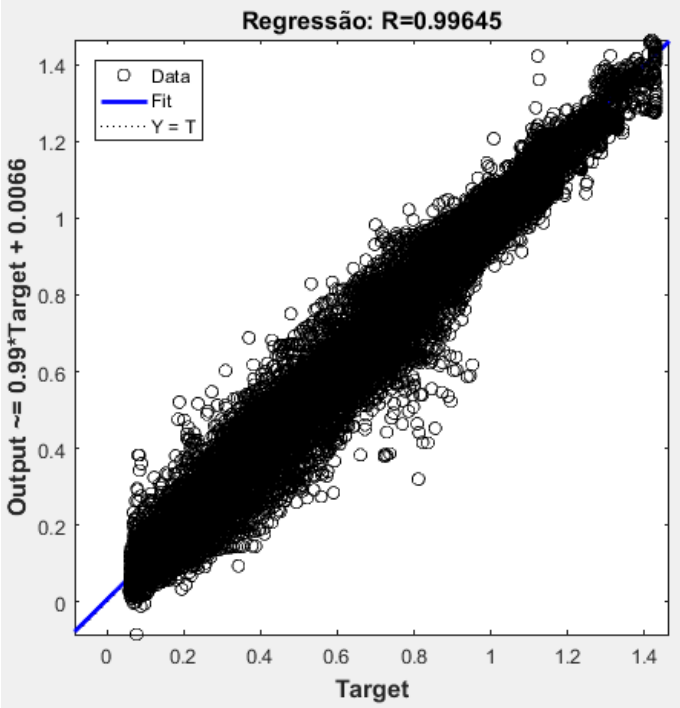
Tabela 12 - Informações sobre o treinamento com 24 neurônios

Informações do Treinamento	Dados
MSE	2.5880e-04
Regressão	0.99645
Épocas	1000
Melhor Época	1000
Tempo de Treinamento	11h e 50min

Fonte: Autoria própria

O algoritmo obteve um MSE de 2.5880e-04 e uma taxa de regressão de 0.99645. O Gráfico 13 apresenta o gráfico da regressão. Mesmo o algoritmo terminando seu treinamento por ter atingido o limite máximo de época, ainda sim conseguiu atingir uma ótima taxa de regressão e um bom MSE.

Gráfico 13 - Taxa de Regressão para o algoritmo BR com 24 neurônios



Fonte: Autoria própria

As taxas de erro altas (maiores que 3%) foram poucas, como apresentado na Tabela 13, principalmente o erro ε > 10% que é o pior erro possível, teve um percentual de erro de apenas 0.42%. A taxa de erro ε ≤ 1% ficou acima de 80%, que é algo bom, indicando que a maioria dos pontos tiveram uma diferença de apenas 1% entre o valor real e o valor do treinamento. Consequentemente a taxa de acerto também resultou em um bom resultado, atingindo 95,61%.

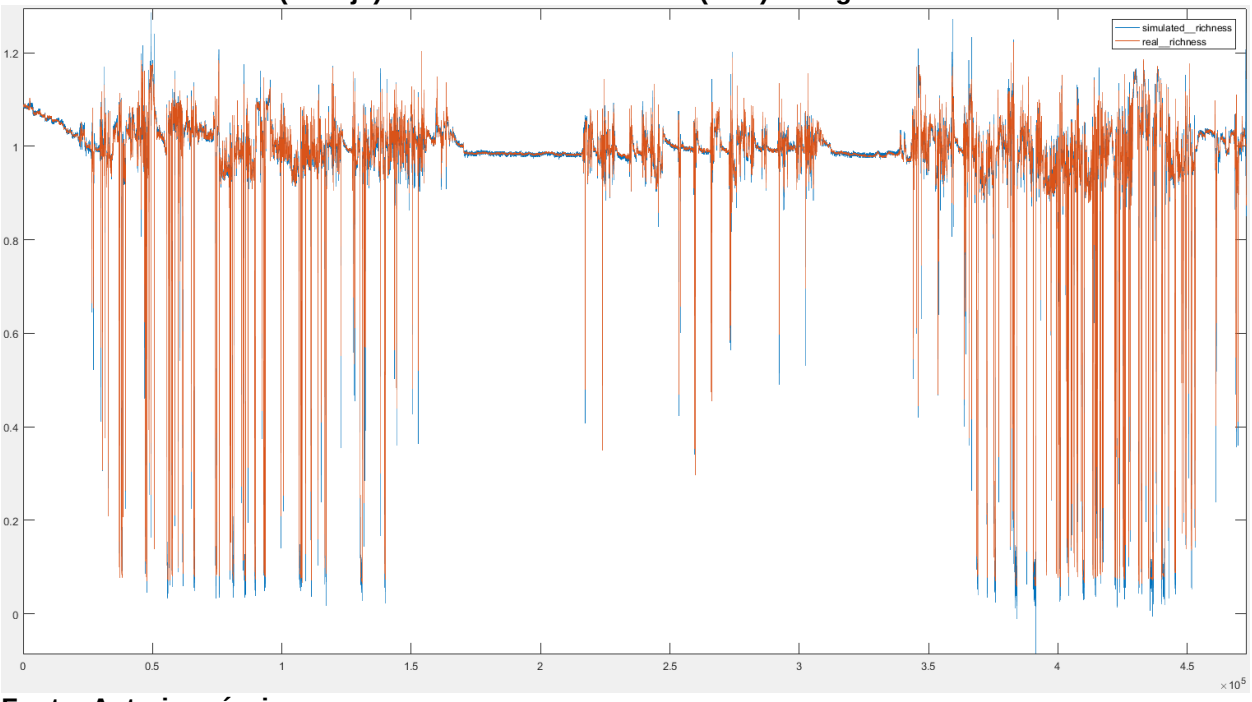
Tabela 13 - Taxas de erro e taxa de acerto do algoritmo BR no treinamento com 24 neurônios

ε ≤ 1%	ε ≤ 3%	ε ≤ 5%	ε ≤ 10%	ε > 10%	Taxa de Acerto
81.55	14.06	2.60	1.37	0.42	95.61%

Fonte: Autoria própria

Em relação ao gráfico do valor real e do valor do treinamento, este é apresentado no Gráfico 14. É possível observar que os pontos em azul (valor do treinamento) seguem com precisão os pontos em laranja (valor real).

Gráfico 14 - Valor real (laranja) e o valor do treinamento (azul) no algoritmo BR com 24 neurônios



Fonte: Autoria própria

Quanto a taxa de simulação apresentada na Tabela 14, obtivemos uma ótima taxa de acerto, superando 85% de acerto. O principal erro ε ≤ 1% se manteve em mais da metade dos pontos, atingindo um total de 54,71%, juntamente com o segundo principal erro ε ≤ 3% que obteve um percentual de 30,5%.

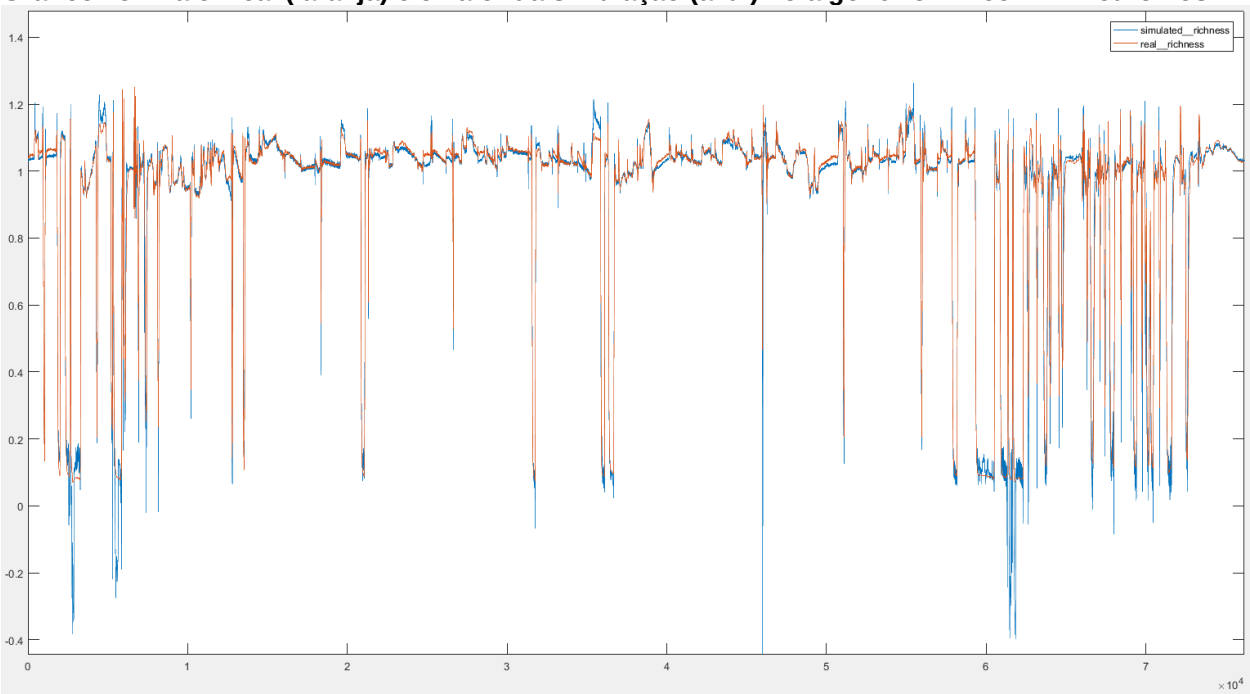
Tabela 14 - Taxas de erro e taxa de acerto do algoritmo BR na simulação com 24 neurônios

ε ≤ 1%	ε ≤ 3%	ε ≤ 5%	ε ≤ 10%	ε > 10%	Taxa de Acerto
54.71	30.50	6.29	4.62	3.88	85.21%

Fonte: Autoria própria

Mesmo o algoritmo BR atingindo boas taxas de simulação, alguns erros persistiram quando comparados com o algoritmo anterior, o LM. Como é possível observar pelo Gráfico 15, os pontos entre 0 e 1 obtiveram alguns picos grandes de erro entre o valor simulado (azul) e o valo real (laranja), como também ocorreu a partir do ponto 6, onde também houve maior variação de erros.

Gráfico 15 - Valor real (laranja) e o valor da simulação (azul) no algoritmo BR com 24 neurônios



Fonte: Autoria própria

Como já explicado anteriormente, é importante ressaltar que mesmo havendo os picos de erro, não necessariamente esses erros ocorrem por motivos do algoritmo de treinamento e sim por causa do conjunto de dados que é utilizado no treinamento e na simulação, podendo haver situações na simulação que não foram contempladas no treinamento.

5.4.2 Bayesian Regularization com 48 Neurônios na Camada Oculta

O treinamento com a segunda arquitetura de 48 neurônios na camada oculta obteve melhores resultados de MSE e Regressão quando comparados com a primeira arquitetura de 24 neurônios, porém o tempo de treinamento foi quase 4 vezes maior, de 11h e 50min com a primeira arquitetura para 43h e 39min com a segunda arquitetura, uma diferença de 31h e 49min. Assim como no treinamento da primeira arquitetura, o treinamento encerrou por atingir o limite máximo de épocas, apresentado na Tabela 15.

Tabela 15 - Informações sobre o treinamento com 48 neurônios

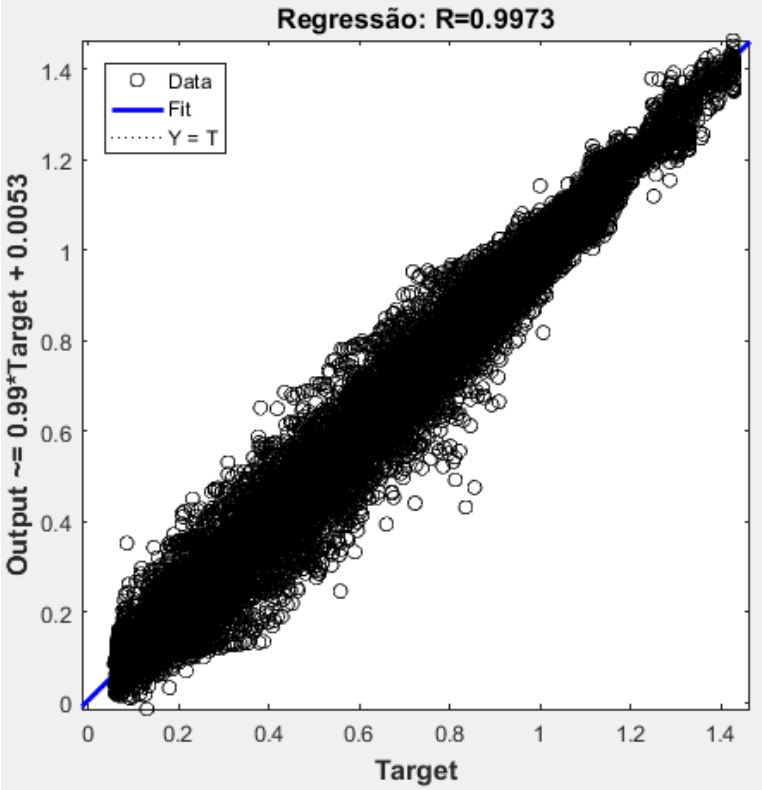
Informações do Treinamento	Dados
MSE	1.9713e-04
Regressão	0.9973
Épocas	1000
Melhor Época	1000
Tempo de Treinamento	43h e 39min

Fonte: Autoria própria

O valor do MSE abaixou de 2.5880e-04 (primeira arquitetura) para 1.9713e-04 (segunda arquitetura) e a Regressão aumentou de 0.99645 para 0.9973. O Gráfico 16 apresenta a taxa de regressão para o treinamento com a segunda arquitetura.

Em comparação com as taxas de erro e taxa de acerto no treinamento da primeira arquitetura, houve um aumento de 2,92% na taxa de erro $\varepsilon \leq 1\%$, porém houve uma diminuição de 1,9% na taxa de erro $\varepsilon \leq 3\%$, as outras taxas de erro não tiveram mudanças significativas. A taxa de acerto aumentou 1,02% em comparação com a primeira arquitetura.

Gráfico 16 - Taxa de Regressão para o algoritmo BR com 48 neurônios



Fonte: Autoria própria

Comparando as duas arquiteturas, a segunda teve um desempenho melhor no treinamento, pois teve melhora no MSE, Regressão e na Taxa de Acerto apresentado na Tabela 16.

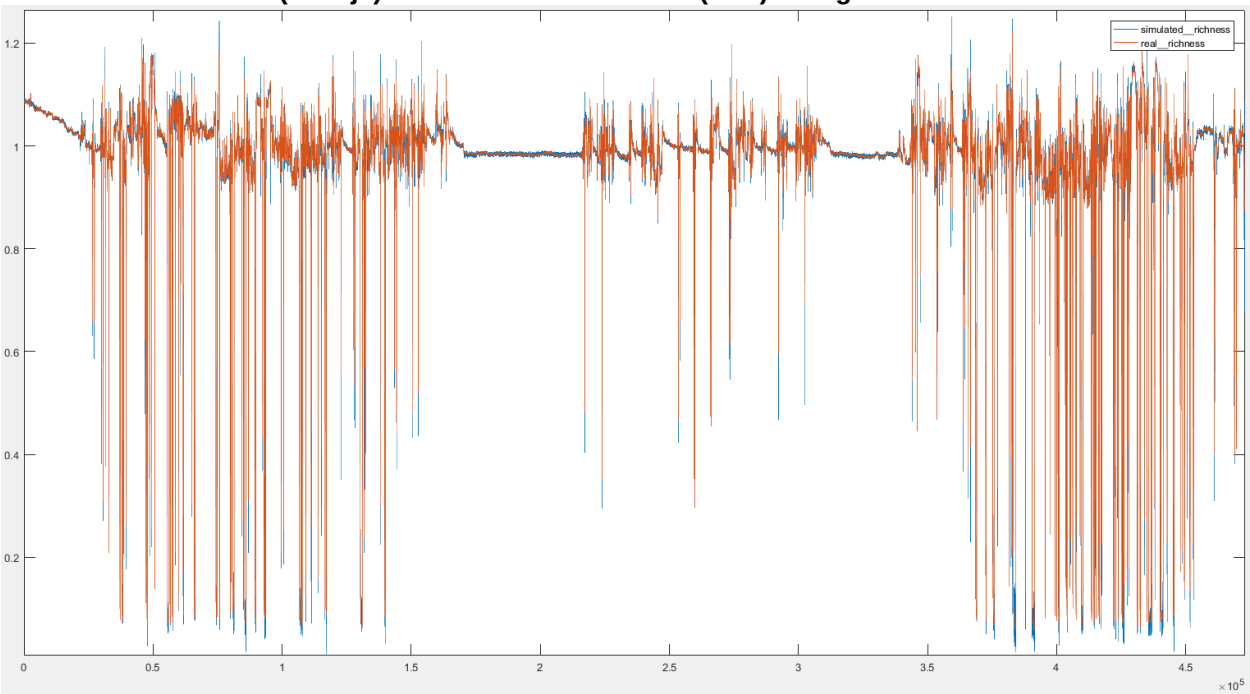
Tabela 16 - Taxas de erro e taxa de acerto do algoritmo BR no treinamento com 48 neurônios

ε ≤ 1%	ε ≤ 3%	ε ≤ 5%	ε ≤ 10%	ε > 10%	Taxa de Acerto
84.47	12.16	1.97	1.08	0.32	96.63%

Fonte: Autoria própria

O Gráfico 17 apresenta o gráfico entre o valor real (laranja) e o valor do treinamento (azul), como o aumento na taxa de acerto no treinamento foi de apenas 1,02%, não é possível distinguir as diferenças no gráfico do Gráfico 14 da primeira arquitetura e do Gráfico 17 da segunda arquitetura.

Gráfico 17 - Valor real (laranja) e o valor do treinamento (azul) no algoritmo BR com 48 neurônios



Fonte: Autoria própria

A melhora no treinamento não refletiu na simulação. Em comparação com a primeira arquitetura, o erro ε ≤ 1% se manteve parecido, diminuindo apenas 0,15%, enquanto o erro ε ≤ 3% diminuiu 2,5%, e as outras taxas de erro todas aumentaram, resultando em uma diminuição na taxa de acerto de 2,65%.

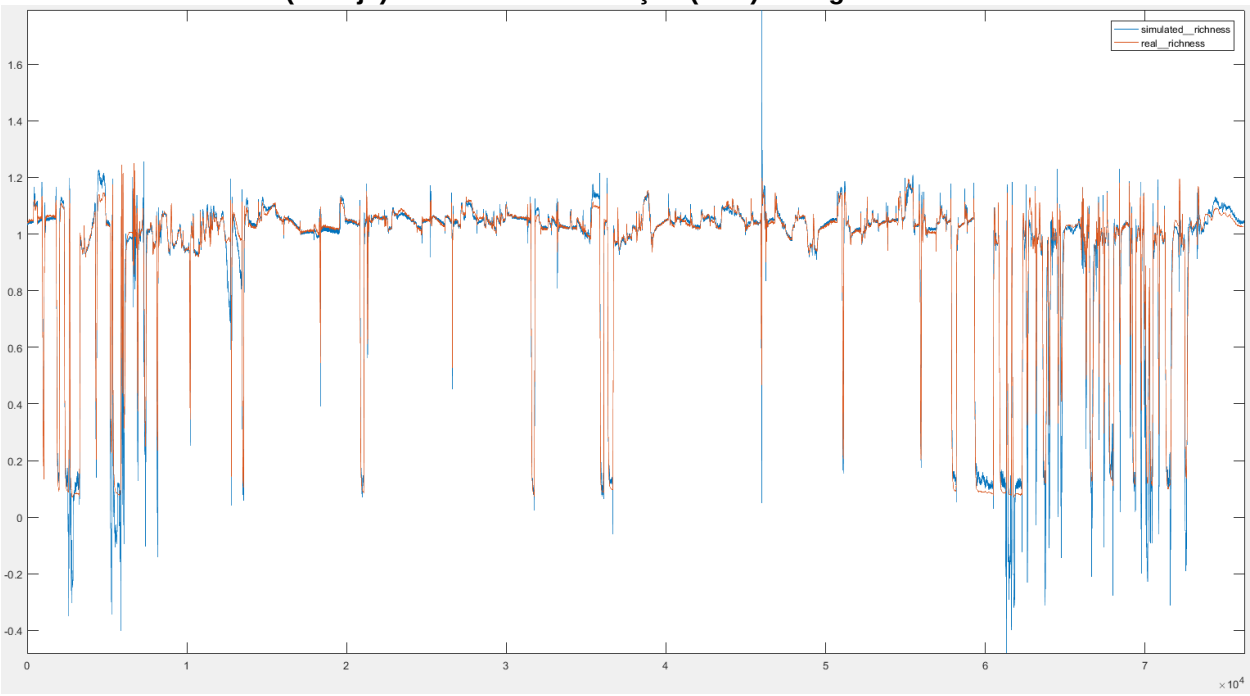
Tabela 17 - Taxas de erro e taxa de acerto do algoritmo BR na simulação com 48 neurônios

ε ≤ 1%	ε ≤ 3%	ε ≤ 5%	ε ≤ 10%	ε > 10%	Taxa de Acerto
54.56	28.00	6.61	4.80	6.03	82.56%

Fonte: Autoria própria

Como é possível observar na Tabela 17, na simulação a segunda arquitetura obteve um resultado inferior a primeira arquitetura, refletindo diretamente no gráfico entre o valor real e o valor simulado no Gráfico 18, onde é possível observar que os erros continuaram e até se alongaram em alguns pontos.

Gráfico 18 - Valor real (laranja) e o valor da simulação (azul) no algoritmo BR com 48 neurônios



Fonte: Autoria própria

5.4.3 Bayesian Regularization com 64 Neurônios na Camada Oculta

Assim como a primeira e a segunda arquitetura, o treinamento do algoritmo BR com a terceira arquitetura terminou devido ter atingido o número máximo de épocas, porém obteve o melhor MSE e Regressão das três arquiteturas. O tempo de treinamento foi alto, sendo necessário 76h e 19min, um pouco mais de 3 dias seguidos de treinamento.

Tabela 18 - Informações sobre o treinamento com 64 neurônios

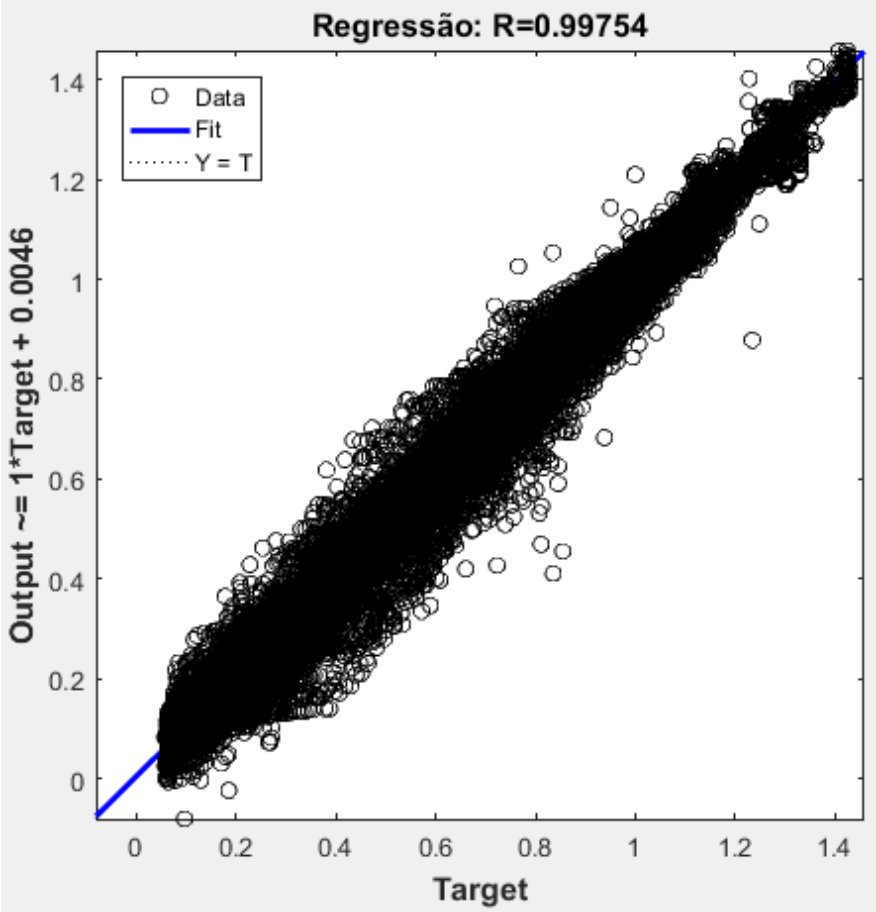
Informações do Treinamento	Dados
MSE	1.7941e-04
Regressão	0.99754
Épocas	1000
Melhor Época	1000
Tempo de Treinamento	76h e 19min

Fonte: Autoria própria

Comparando com a primeira arquitetura, a taxa de Regressão aumentou 0.00109 e comparando com a segunda arquitetura a taxa de Regressão aumentou

apenas 0.00024. O Gráfico 19 apresenta a taxa de regressão com a terceira arquitetura de 64 neurônios na camada oculta.

Gráfico 19 - Taxa de Regressão para o algoritmo BR com 64 neurônios



Fonte: Autoria própria

Quando comparamos as taxas de erro e a taxa de acerto da terceira arquitetura com as duas outras, a terceira arquitetura é a que obteve os melhores resultados, obtendo a melhor taxa de erro ε ≤ 1% com 84.86% e a menor taxa de erro ε > 10%, com apenas 0.26%, resultando em uma taxa de acerto de 96.70%, como apresentado na Tabela 19, sendo 1.09% maior que a primeira arquitetura e 0.07% maior que a segunda arquitetura.

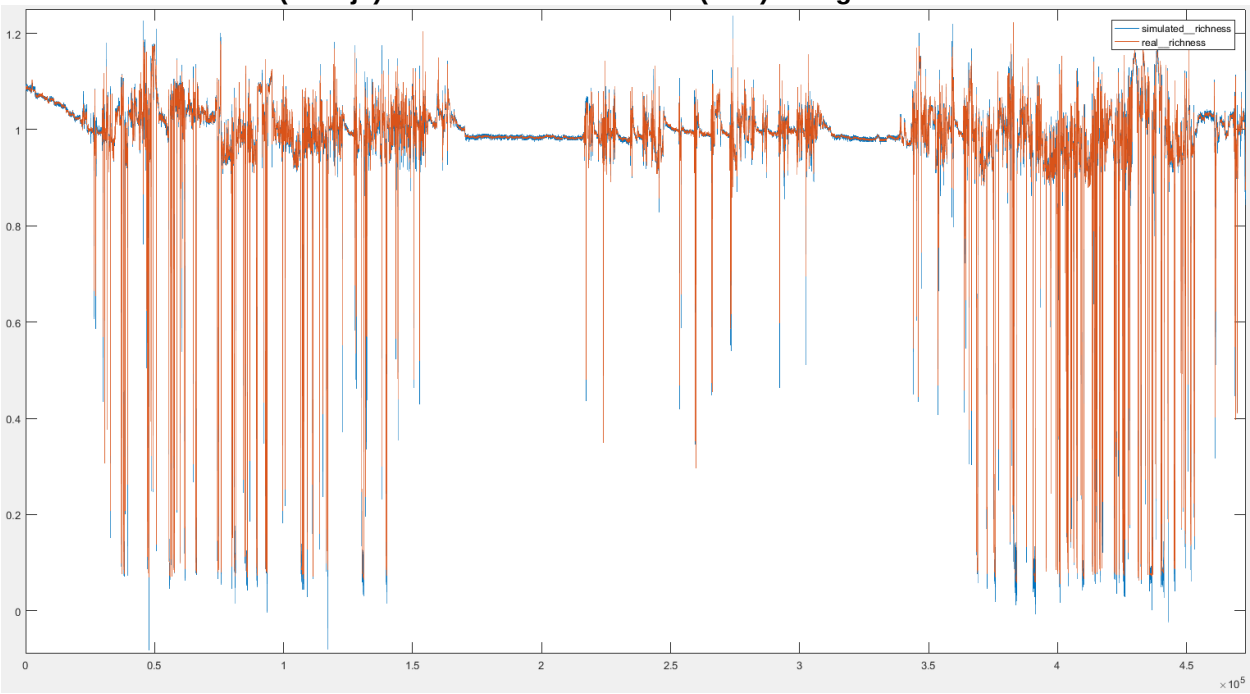
Tabela 19 - Taxas de erro e taxa de acerto do algoritmo BR no treinamento com 64 neurônios

ε ≤ 1%	ε ≤ 3%	ε ≤ 5%	ε ≤ 10%	ε > 10%	Taxa de Acerto
84.86	11.84	2.01	1.03	0.26	96.70%

Fonte: Autoria própria

Como a taxa de acerto foi alta, poucos pontos do valor do treinamento do Gráfico 20 não seguem o valor real.

Gráfico 20 - Valor real (laranja) e o valor do treinamento (azul) no algoritmo BR com 64 neurônios



Fonte: Autoria própria

Mais uma vez a melhora no treinamento não resultou na melhora da simulação. Em comparação com a primeira arquitetura, a taxa de acerto diminuiu 2.89%, enquanto em comparação com a segunda arquitetura esse valor foi de 0.24%.

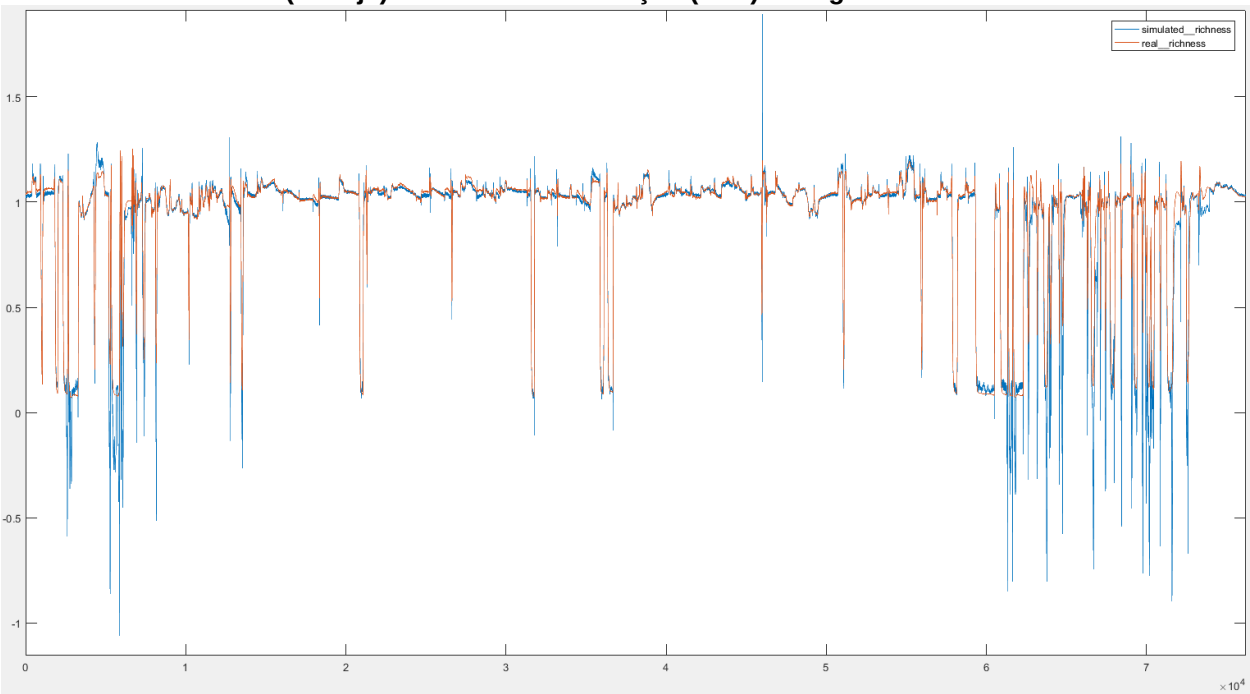
Tabela 20 - Taxas de erro e taxa de acerto do algoritmo BR na simulação com 64 neurônios

ε ≤ 1%	ε ≤ 3%	ε ≤ 5%	ε ≤ 10%	ε > 10%	Taxa de Acerto
52.27	30.05	5.52	4.72	7.44	82.32%

Fonte: Autoria própria

Como a taxa de acerto na simulação diminuiu em comparação com as duas outras arquiteturas, não houve melhora no gráfico entre o valor real e o valor simulado, apresentado no Gráfico 21, mantendo os pontos de grande dispersão.

Gráfico 21 - Valor real (laranja) e o valor da simulação (azul) no algoritmo BR com 64 neurônios



Fonte: Autoria própria

5.5 DESEMPENHO DO ALGORITMO SCALED CONJUGATE GRADIENT NO ESTUDO DE CASO

O treinamento com o algoritmo Scaled Conjugate Gradient também utiliza o mesmo conjunto de dados dos dois algoritmos anteriores, com 482855 mil dados. Destes dados, 70% são destinados para treinamento, 15% para validação e 15% para testes. E o mesmo conjunto de 76192 mil dados para a validação da rede neural.

5.5.1 Scaled Conjugate Gradient com 24 Neurônios na Camada Oculta

O treinamento do algoritmo SCG com a primeira arquitetura de 24 neurônios produziu resultados completamente diferente dos dois algoritmos vistos anteriormente, o LM e o BR. O primeiro fato que chama atenção é o seu tempo de treinamento, sendo necessário apenas 9min para a rede convergir com um alto número de épocas, 785 ao todo, portanto cada época não levou muito tempo para ser realizada, pouco mais de 1 segundo para cada época.

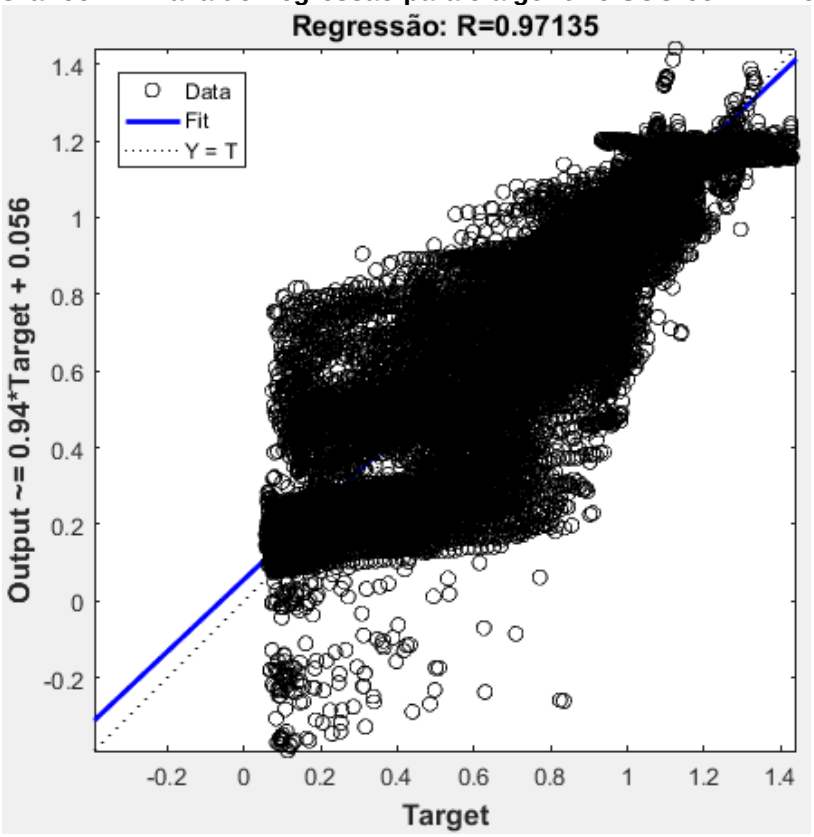
Tabela 21 - Informações sobre o treinamento com 24 neurônios

Informações do Treinamento	Dados
MSE	0.0020
Regressão	0.97135
Épocas	785
Melhor Época	779
Tempo de Treinamento	9min

Fonte: Autoria própria

Outro fato interessante que foi completamente diferente dos dois outros algoritmos é o valor MSE, que atingiu um valor extremamente baixo, o mais próximo de 0 até o momento, como apresentado na Tabela 21. A Regressão obteve um valor bom, porém é o menor de todos os treinamentos comparados até o momento. É possível observar pelo Gráfico 22 que a regressão teve um comportamento não registrado até o momento nos dois primeiros algoritmos. Os pontos dos dados do gráfico estão muito mais dispersos do que os outros gráficos de regressão, que tendiam seguir mais a linha azul.

Gráfico 22 - Taxa de Regressão para o algoritmo SCG com 24 neurônios



Fonte: Autoria própria

Os valores das taxas de erro e da taxa de acerto no treinamento são apresentados na Tabela 22. Todas as taxas de acerto no treinamento do algoritmo LM e o algoritmo BR foram superiores a 95%, já o algoritmo SCG cerca de 15% a menos que todos os outros, alcançado apenas 80.91% de taxa de acerto.

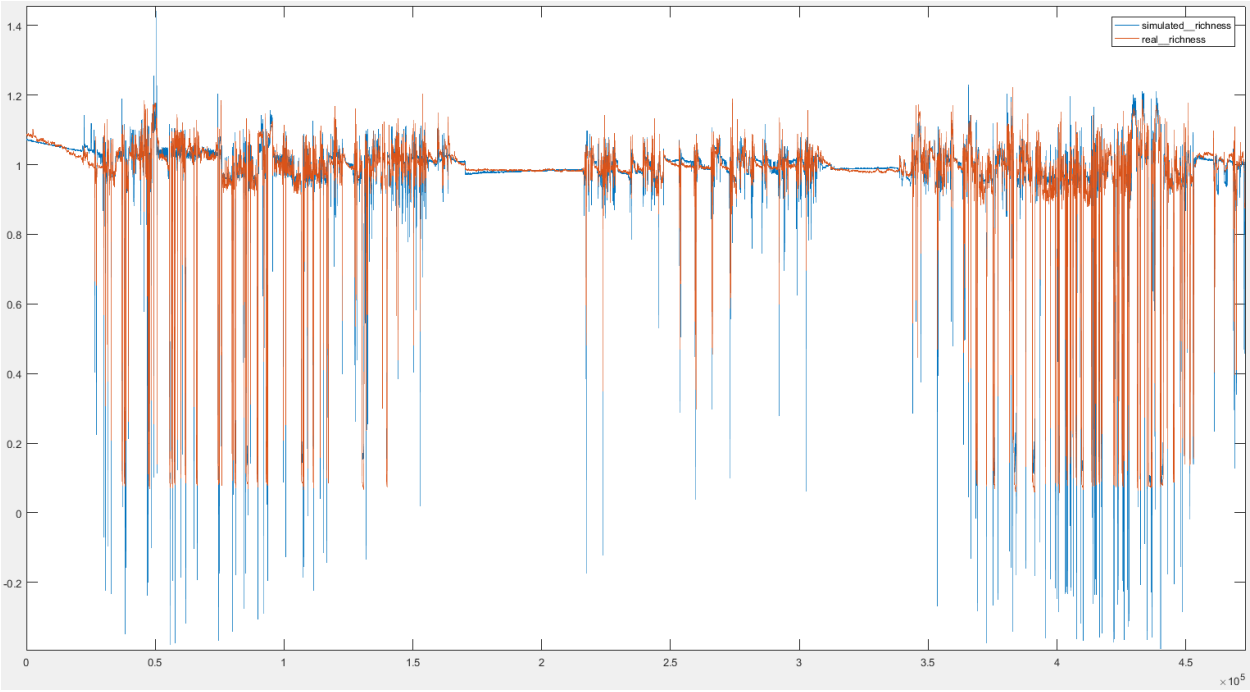
Tabela 22 - Taxas de erro e taxa de acerto do algoritmo SCG no treinamento com 24 neurônios

ε ≤ 1%	ε ≤ 3%	ε ≤ 5%	ε ≤ 10%	ε > 10%	Taxa de Acerto
44.72	36.19	10.26	5.84	2.99	80.91%

Fonte: Autoria própria

Como a taxa de acerto do algoritmo foi baixa, consequentemente os erros acima de 3% entre o valor real e o valor do treinamento aumentarem, refletindo no gráfico entre esses dois valores que é apresentado no Gráfico 23. É possível observar claramente muitos picos de erros que não ocorriam nos outros algoritmos.

Gráfico 23 - Valor real (laranja) e o valor do treinamento (azul) no algoritmo SCG com 24 neurônios



Fonte: Autoria própria

Pela primeira vez na simulação a taxa de erro ε ≤ 1% ficou abaixo da taxa de erro ε ≤ 3%, o que é algo ruim para a simulação, pois o objetivo é manter o maior percentual de erro na menor taxa de erro possível, que é o erro ε ≤ 1%. A taxa de acerto da simulação também obteve valor inferiores aos outros algoritmos, sendo que

a diferença entre o melhor treinamento dos outros algoritmos com a mesma arquitetura é superior a 20%.

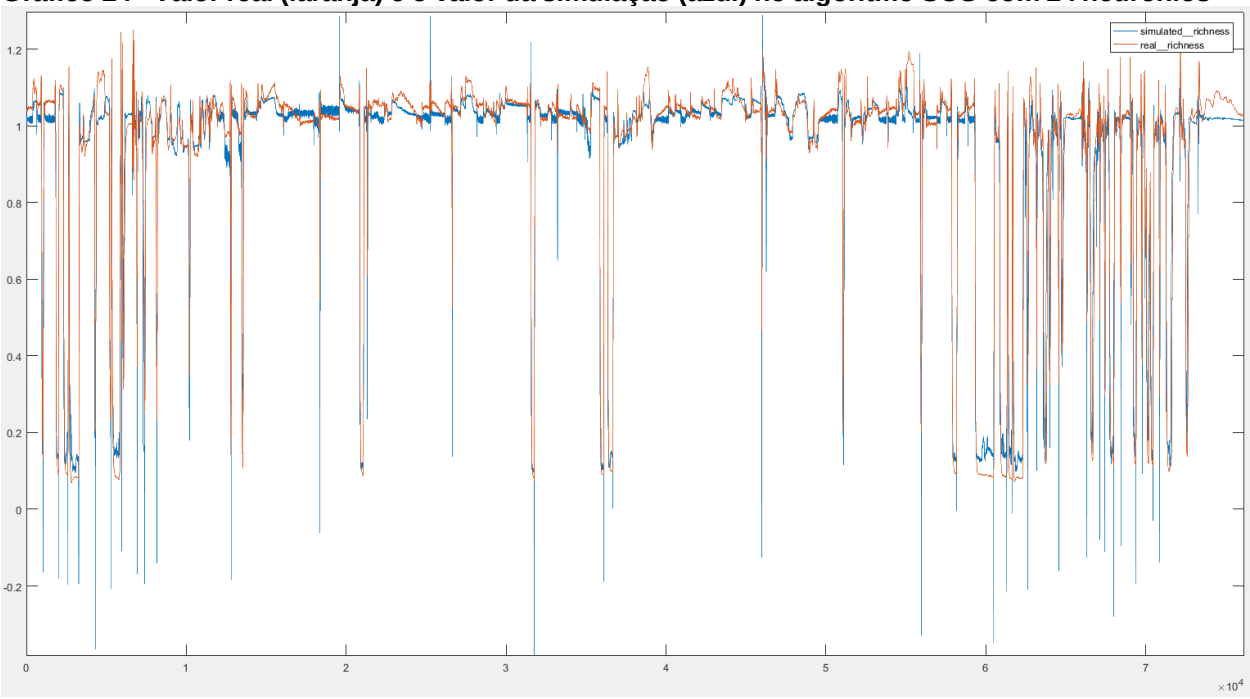
Tabela 23 - Taxas de erro e taxa de acerto do algoritmo SCG na simulação com 24 neurônios

ε ≤ 1%	ε ≤ 3%	ε ≤ 5%	ε ≤ 10%	ε > 10%	Taxa de Acerto
24.02	39.27	19.34	9.72	7.65	63.29%

Fonte: Autoria própria

Como a simulação obteve uma taxa de acerto na simulação baixa, como apresentado na Tabela 23, esse resultado afeta também o gráfico entre o valor real e o valor simulado apresentado no Gráfico 24. Assim como os outros algoritmos, o algoritmo SCG manteve os grandes picos de erro entre o intervalo 0 e 1 e a partir do ponto 6, além de ampliar a quantidade de pontos que estão com um ruim resultado, devido à baixa taxa de acerto.

Gráfico 24 - Valor real (laranja) e o valor da simulação (azul) no algoritmo SCG com 24 neurônios



Fonte: Autoria própria

5.5.2 Scaled Conjugate Gradient com 48 Neurônios na Camada Oculta

Pela primeira vez entre o treinamento dos algoritmos com as três arquiteturas, a segunda arquitetura que tem o dobro de neurônios da primeira arquitetura obteve

um resultado inferior em MSE e Regressão no treinamento. A primeira arquitetura com o algoritmo SCG obteve um valor de 0.0020, enquanto a segunda arquitetura obteve um valor de 0.0043, como apresentado na Tabela 24. O algoritmo com a primeira arquitetura obteve um valor de 0.97135, enquanto com a segunda abaixou para 0.94018.

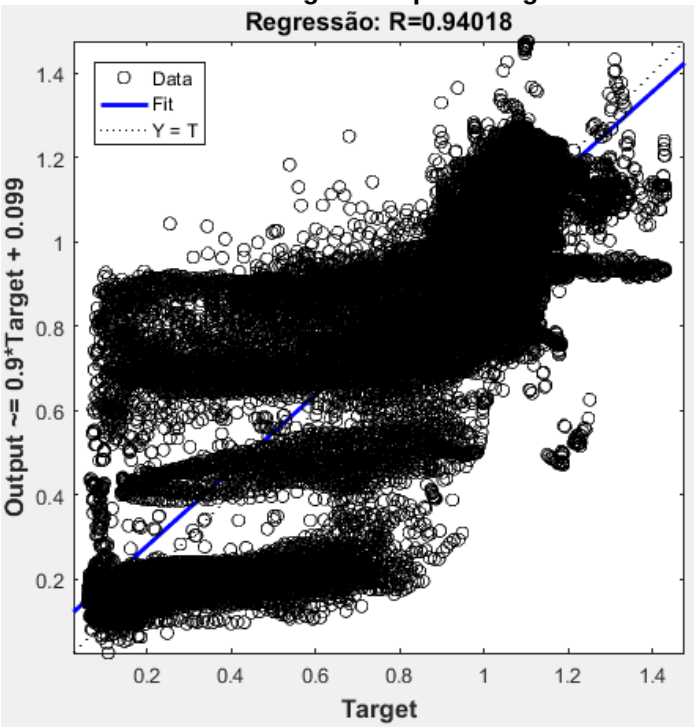
Tabela 24 - Informações sobre o treinamento com 48 neurônios

Informações do Treinamento	Dados
MSE	0.0043
Regressão	0.94018
Épocas	199
Melhor Época	193
Tempo de Treinamento	3min

Fonte: Autoria própria

Mesmo com os valores do MSE e da Regressão sendo piores que a primeira arquitetura, a quantidade de épocas e o tempo necessário para o algoritmo convergir foi menor, porém não tão relevante assim, sendo apenas 6 min mais rápido. A regressão apresentada no Gráfico 25 é o mais disperso até o momento, mesmo o gráfico da regressão da primeira arquitetura (Gráfico 22) sendo totalmente diferente dos outros algoritmos já analisados, a segunda arquitetura (Gráfico 25) contém uma maior quantidade de pontos de dados espalhados pelo gráfico e atinge valores maiores de dispersão.

Gráfico 25 - Taxa de Regressão para o algoritmo SCG com 48 neurônios



Fonte: Autoria própria

A taxa de acerto no treinamento abaixou em relação ao treinamento com o algoritmo na primeira arquitetura, uma diferença de 14.22%, ficando próximo do valor da simulação da primeira arquitetura que foi de 63.29%.

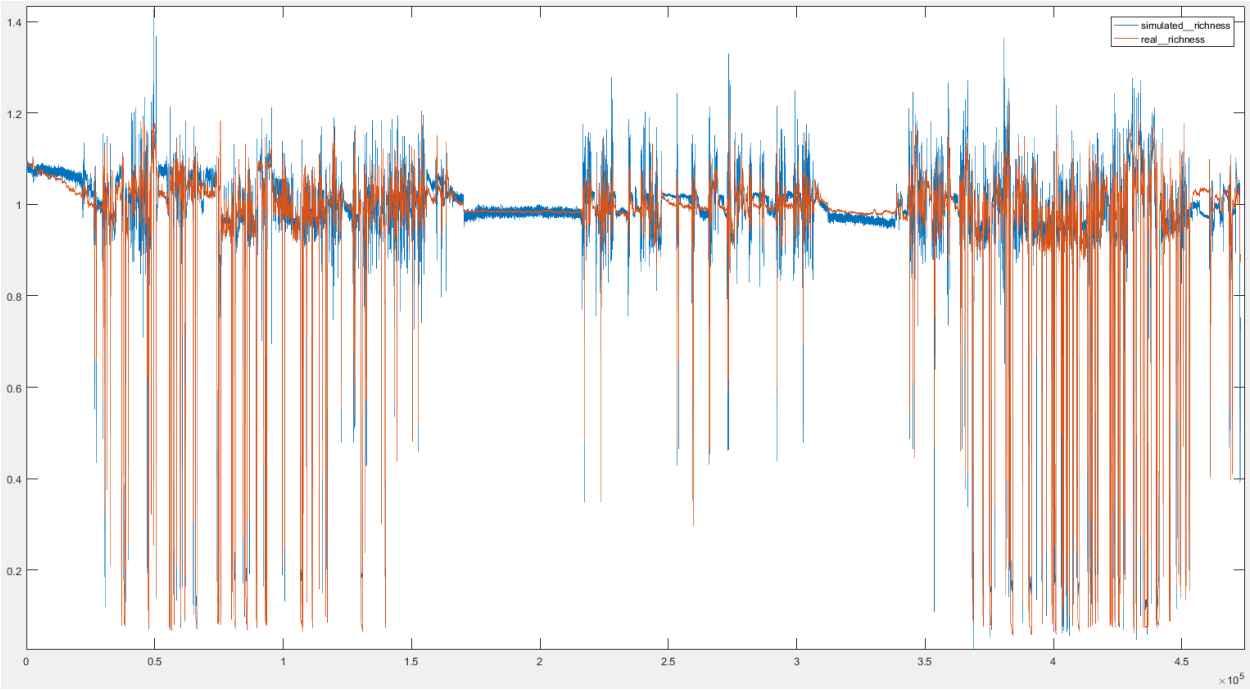
Tabela 25 - Taxas de erro e taxa de acerto do algoritmo SCG no treinamento com 48 neurônios

ε ≤ 1%	ε ≤ 3%	ε ≤ 5%	ε ≤ 10%	ε > 10%	Taxa de Acerto
30.05	36.64	15.34	12.07	5.90	66.69%

Fonte: Autoria própria

As taxas de erro acima de 3% apresentadas na Tabela 25 são altas e a taxa de acerto baixa reflete diretamente no gráfico entre o valor real e o valor do treinamento apresentado no Gráfico 26, onde é possível observar alta dispersão entre os valores.

Gráfico 26 - Valor real (laranja) e o valor do treinamento (azul) no algoritmo SCG com 48 neurônios



Fonte: Autoria própria

Como o treinamento não obteve um bom resultado apresentado na Tabela 24, também não obteve uma boa taxa de regressão e uma baixa taxa de acerto (Tabela 25), consequentemente o resultado da simulação não foi diferente, como é possível observar na Tabela 26. O erro ε ≤ 1% foi baixíssimo, obtendo apenas 12.79% e o erro ε > 10% obtendo a maior porcentagem, com o resultado de 26.66%, consequentemente a taxa de acerto também foi baixa, com apenas 36.44%.

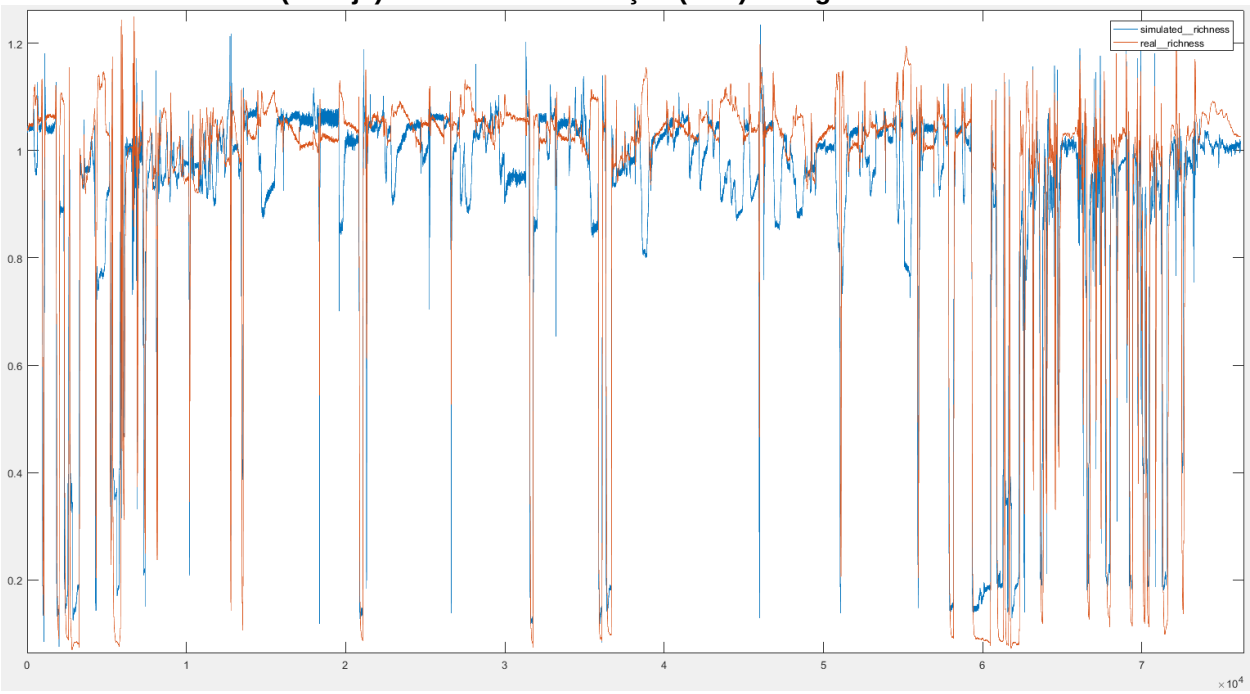
Tabela 26 - Taxas de erro e taxa de acerto do algoritmo SCG na simulação com 48 neurônios

ε ≤ 1%	ε ≤ 3%	ε ≤ 5%	ε ≤ 10%	ε > 10%	Taxa de Acerto
12.79	23.65	17.93	18.97	26.66	36.44%

Fonte: Autoria própria

O resultado da simulação apresentado no Gráfico 27 é bem inferior aos treinamentos já analisados até o momento e em apenas pouquíssimos casos o valor simulado (azul) segue o valor real (laranja).

Gráfico 27 - Valor real (laranja) e o valor da simulação (azul) no algoritmo SCG com 48 neurônios



Fonte: Autoria própria

5.5.3 Scaled Conjugate Gradient com 64 Neurônios na Camada Oculta

O treinamento com a terceira arquitetura teve um resultado intermediário entre a primeira arquitetura e a segunda arquitetura. O valor do MSE de 0.0031 é pior do que a primeira arquitetura (0.0020), porém é melhor do que a segunda arquitetura (0.0043). A Regressão de 0.9565 segue a mesma regra, sendo inferior a primeira arquitetura (0.97135), porém superior a segunda arquitetura (0.94018).

Tabela 27 - Informações sobre o treinamento com 64 neurônios

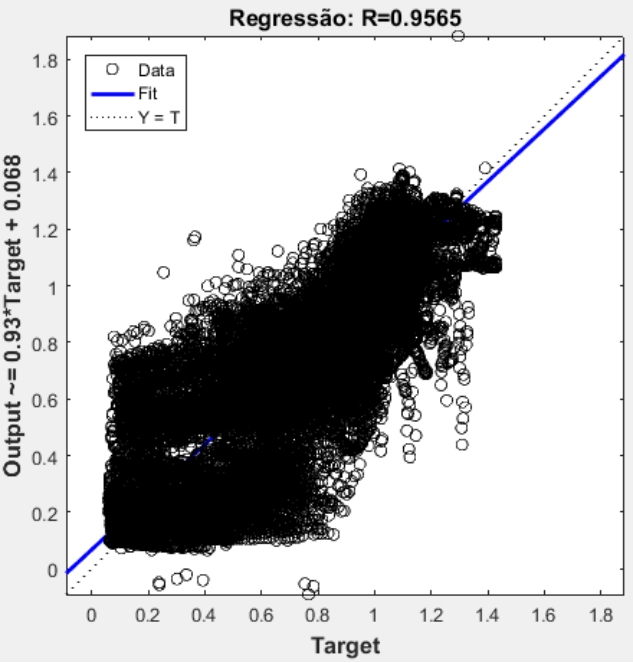
Informações do Treinamento	Dados
MSE	0.0031
Regressão	0.9565
Épocas	1000
Melhor Época	1000
Tempo de Treinamento	27min

Fonte: Autoria própria

Diferente das outras duas arquiteturas, o algoritmo SCG com a terceira arquitetura terminou o treinamento por ter atingido o limite máximo de épocas, como apresentado na Tabela 27. Entretanto, mesmo atingindo o limite máximo de épocas,

o tempo de treinamento foi baixo. O gráfico da Regressão apresentado no Gráfico 28 é o menos disperso entre as três arquiteturas com o algoritmo SCG, sendo que os dados se concentram em regiões diferentes das duas outras arquiteturas.

Gráfico 28 - Taxa de Regressão para o algoritmo SCG com 64 neurônios



Fonte: Autoria própria

A Tabela 28 apresenta as taxas de erro e a taxa de acerto do treinamento para a terceira arquitetura. Comparando com o treinamento da primeira arquitetura, temos uma taxa de acerto 7.72% inferior, porém quando comparado com a segunda arquitetura, temos uma taxa de acerto 6.5% superior, logo a terceira arquitetura fica com o segundo melhor resultado do treinamento do algoritmo SCG.

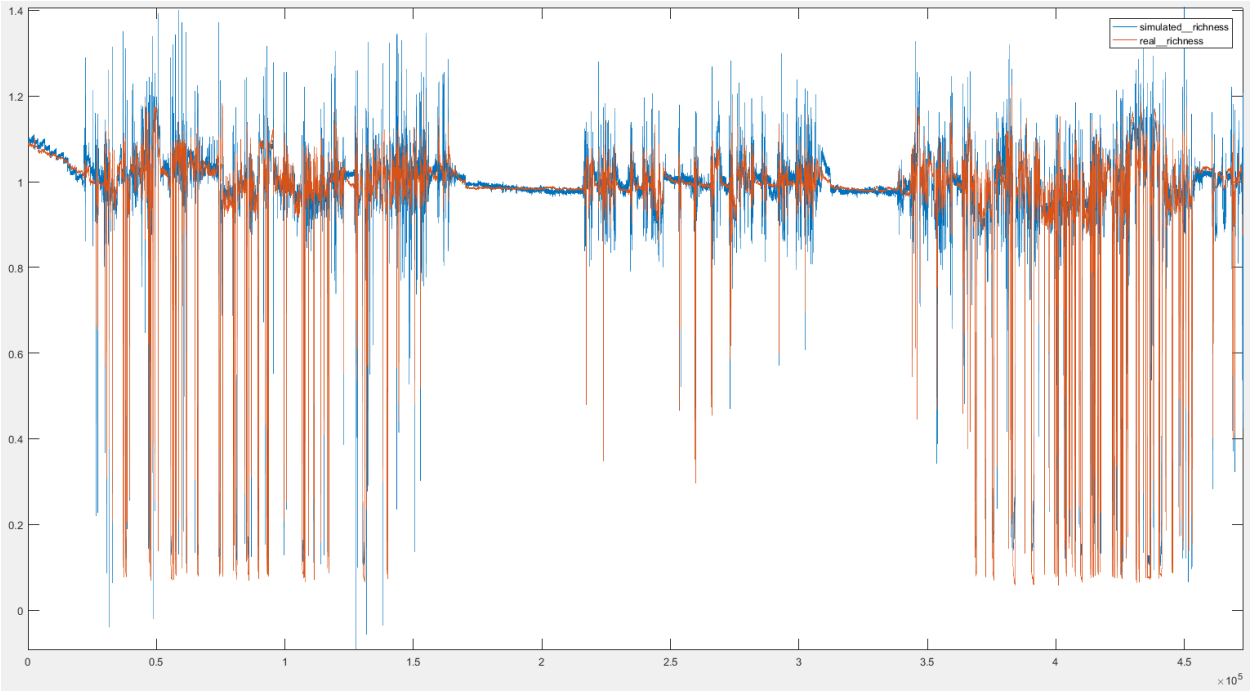
Tabela 28 - Taxas de erro e taxa de acerto do algoritmo SCG no treinamento com 64 neurônios

ε ≤ 1%	ε ≤ 3%	ε ≤ 5%	ε ≤ 10%	ε > 10%	Taxa de Acerto
37.72	35.47	12.80	9.56	4.45	73.19%

Fonte: Autoria própria

Mesmo a taxa de acerto no treinamento aumentando em relação a segunda arquitetura, o gráfico apresentado no Gráfico 29 ainda apresenta grandes diferenças entre o valor real e o valor produzido pelo treinamento.

Gráfico 29 - Valor real (laranja) e o valor do treinamento (azul) no algoritmo SCG com 64 neurônios



Fonte: Autoria própria

Assim como aconteceu na taxa de acerto do treinamento, a taxa de acerto na simulação fica em segundo lugar dentre as três arquiteturas, sendo 17.89% inferior que a primeira arquitetura e 8.96% superior a segunda arquitetura. Porém, a taxa de acerto na simulação ainda é baixa, com apenas 45.40%. A Tabela 29 apresenta as taxas de erro e a taxa de acerto para o treinamento com a terceira arquitetura utilizando o algoritmo SCG.

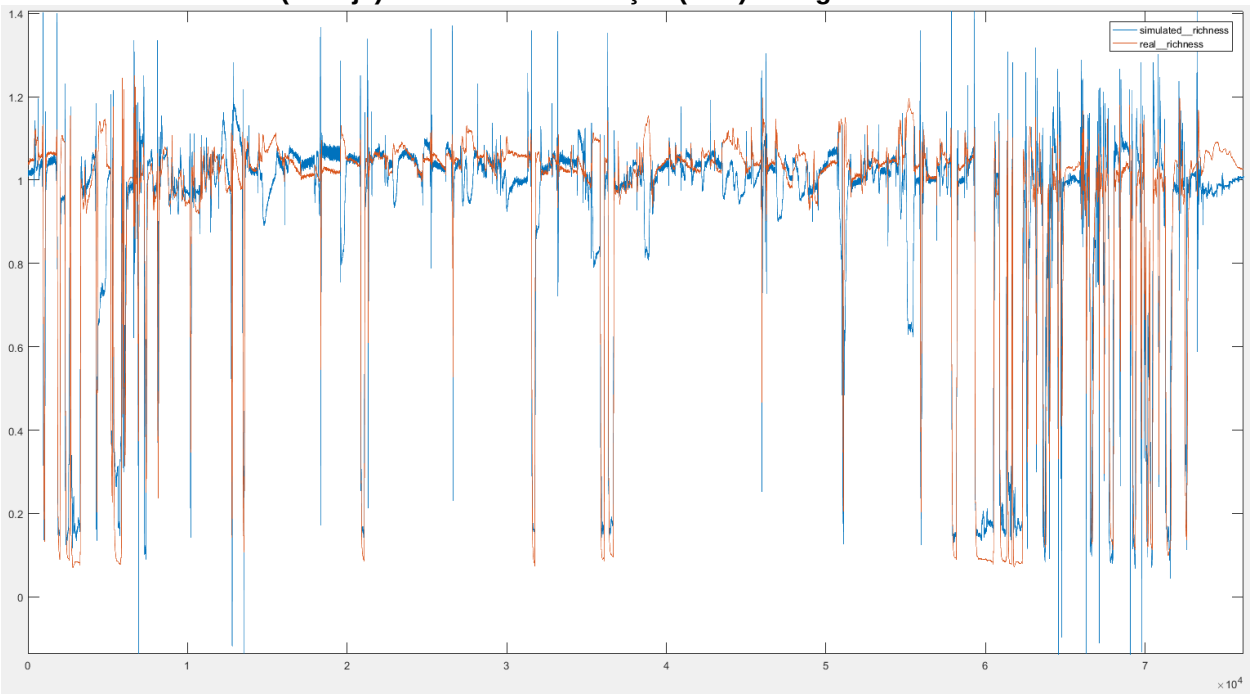
Tabela 29 - Taxas de erro e taxa de acerto do algoritmo SCG na simulação com 64 neurônios

ε ≤ 1%	ε ≤ 3%	ε ≤ 5%	ε ≤ 10%	ε > 10%	Taxa de Acerto
16.20	29.20	15.60	19.90	19.10	45.40%

Fonte: Autoria própria

O Gráfico 30 da simulação com a terceira arquitetura apresentado se assemelha muito ao Gráfico 27. Como as duas arquiteturas obtiveram resultados baixos de taxa de acerto na simulação, seus gráficos de simulação vão ficar muito distantes do valor real, sendo o gráfico inteiro composto por erros de grande escala.

Gráfico 30 - Valor real (laranja) e o valor da simulação (azul) no algoritmo SCG com 64 neurônios



Fonte: Autoria própria

5.6 DESEMPENHO DO ALGORITMO RESILIENT BACKPROPAGATION NO ESTUDO DE CASO

Assim como os três primeiros algoritmos, o Resilient Backpropagation também é treinado com os 482855 mil dados e simulado com o conjunto de 76192 mil dados, também realizando o treinamento nas três arquiteturas da rede, com 24 neurônios, 48 neurônios e 64 neurônios na camada oculta.

5.6.1 Resilient Backpropagation com 24 Neurônios na Camada Oculta

O treinamento do algoritmo Rprop com a primeira arquitetura produziu resultados diferentes de todos os algoritmos com a mesma arquitetura já analisados. A primeira diferença é o alto MSE, que teve um resultado de 9.4539e-04, como apresentado na Tabela 30. Outra diferença é que o algoritmo terminou sua execução pelo limite máximo de épocas, porém a melhor época não foi a última como estava ocorrendo em todos os treinamentos que terminavam por atingir o limite, a melhor época foi uma anterior. Não há nenhum problema em a melhor época ser a anterior

ao limite máximo, isso quer dizer somente que a época 999 teve um resultado melhor que a 1000 e provavelmente o algoritmo estava entrando em um estado de validação.

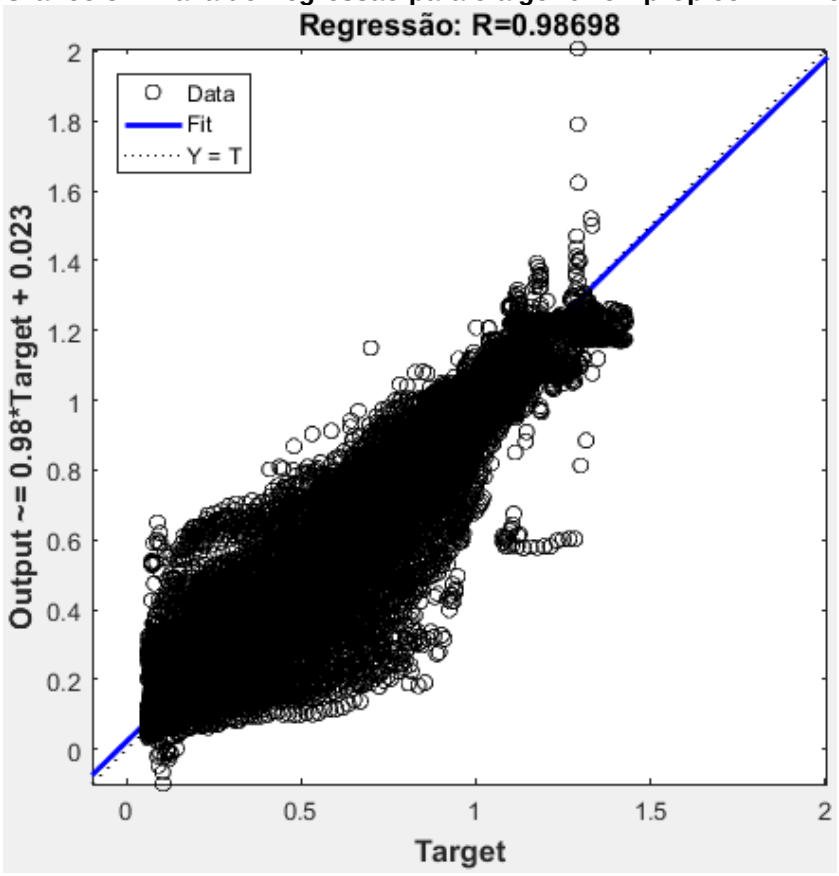
Tabela 30 - Informações sobre o treinamento com 24 neurônios

Informações do Treinamento	Dados
MSE	9.4539e-04
Regressão	0.98698
Épocas	1000
Melhor Época	999
Tempo de Treinamento	6min

Fonte: Autoria própria

O tempo de treinamento para o algoritmo convergir foi o segundo menor até o momento entre todos os algoritmos em todas as arquiteturas. Apesar do alto valor do MSE, o treinamento produziu uma boa regressão com 0.98698, ilustrado no Gráfico 31.

Gráfico 31 - Taxa de Regressão para o algoritmo Rprop com 24 neurônios



Fonte: Autoria própria

O algoritmo obteve bons resultados no treinamento com a primeira arquitetura, superando 90% na taxa de acerto e mais da metade dos erros são menores ou iguais a 1%. Os erros grandes acima de 3% somados resultam em apenas 8,46% dos pontos.

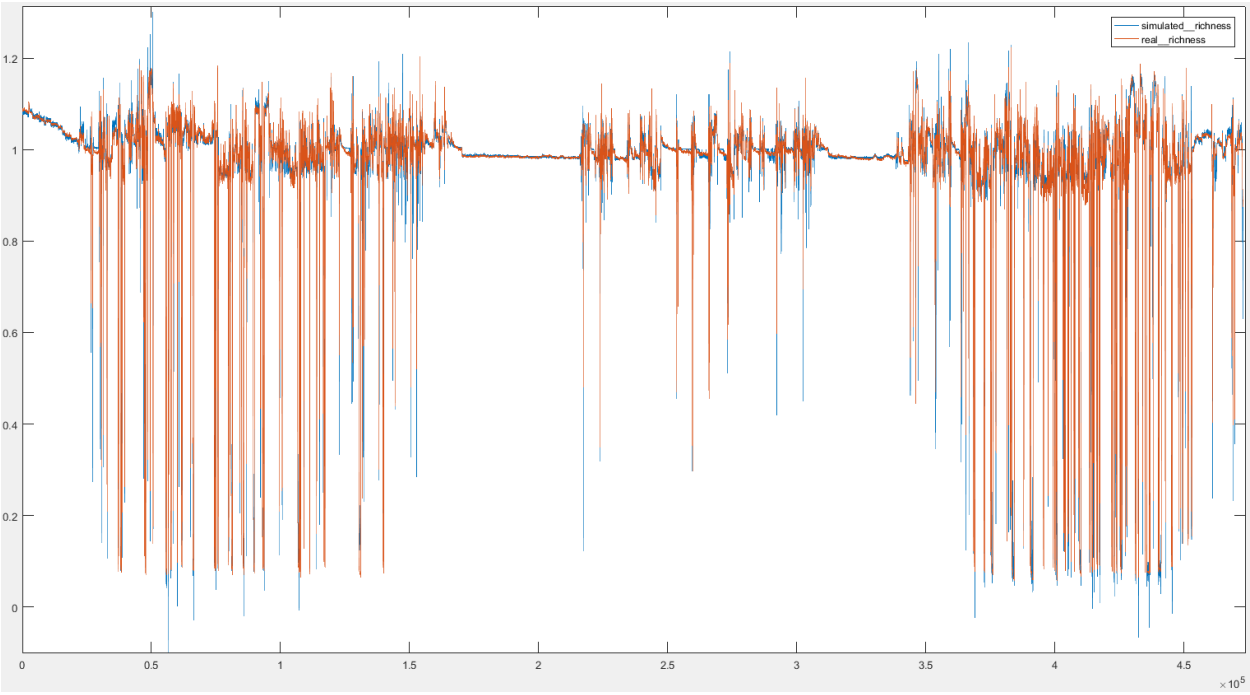
Tabela 31 - Taxas de erro e taxa de acerto do algoritmo Rprop no treinamento com 24 neurônios

ε ≤ 1%	ε ≤ 3%	ε ≤ 5%	ε ≤ 10%	ε > 10%	Taxa de Acerto
68.15	23.39	4.48	2.40	1.58	91.54%

Fonte: Autoria própria

O gráfico entre o valor real e valor do treinamento apresentado no Gráfico 32 contém apenas alguns pontos de maiores erros visíveis, como nos pontos 1.5 e 3, porém na maioria dos pontos os valores são bem próximos.

Gráfico 32 - Valor real (laranja) e o valor do treinamento (azul) no algoritmo Rprop com 24 neurônios



Fonte: Autoria própria

A simulação obteve um resultado muito bom, sendo a terceira maior taxa de acerto entre todos os algoritmos e arquiteturas analisados até o momento e com a diferença que o tempo de treinamento da rede foi de apenas 6 minutos, muito menos que os outros dois treinamentos que produziram resultados de simulação maiores que o Rprop com a primeira arquitetura. Uma diferença importante a se notar é que mesmo a taxa de acerto da simulação sendo alta, os valores estão bem divididos entre os

erros ε ≤ 1% e ε ≤ 3%. Para diferenciar algoritmos que tenham taxas de acerto parecidas, o algoritmo com melhor desempenho é o que contém mais erros ε ≤ 1%, pois esse erro se mantém mais próximo do valor real.

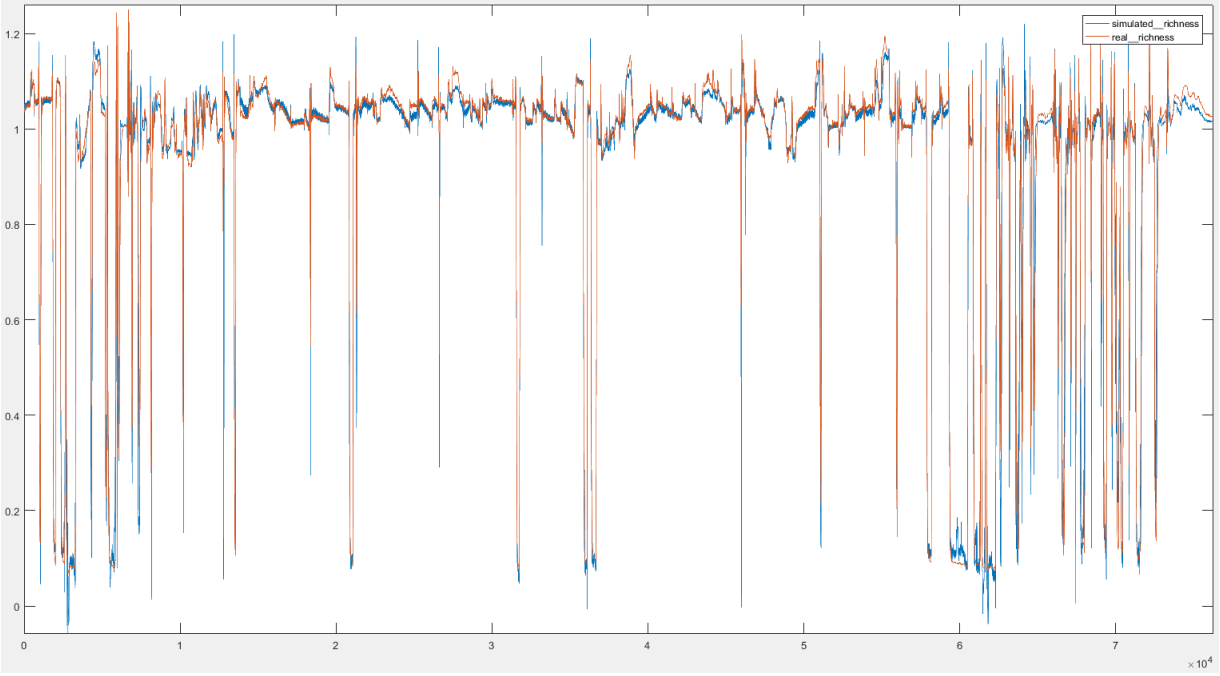
Tabela 32 - Taxas de erro e taxa de acerto do algoritmo Rprop na simulação com 24 neurônios

ε ≤ 1%	ε ≤ 3%	ε ≤ 5%	ε ≤ 10%	ε > 10%	Taxa de Acerto
46.03	38.03	6.54	4.51	4.89	84.06%

Fonte: Autoria própria

O Gráfico 33 apresenta o gráfico entre o valor real e o valor simulado. A simulação segue na maioria dos pontos o valor real, porém não tão perfeitamente quanto os algoritmos que tiveram taxa de acerto altas e melhores taxas de erro ε ≤ 1%, sendo possível diferenciar o gráfico entre esses algoritmos. Contudo vale ressaltar novamente o tempo de treinamento do algoritmo, que foi bem inferior aos outros que produziram bons resultados.

Gráfico 33 - Valor real (laranja) e o valor da simulação (azul) no algoritmo Rprop com 24 neurônios



Fonte: Autoria própria

5.6.2 Resilient Backpropagation com 48 Neurônios na Camada Oculta

O treinamento do algoritmo Rprop com a terceira arquitetura de 48 neurônios obteve resultado parecidos com o treinamento com a primeira arquitetura. O valor 8.7321e-04 é abaixo do que a primeira arquitetura, porém ainda é um valor alto. A regressão 0.98798 é superior a regressão da primeira arquitetura. O tempo de treinamento foi 4 minutos superior a primeira arquitetura, porém também foi um tempo baixo, apenas 10 minutos.

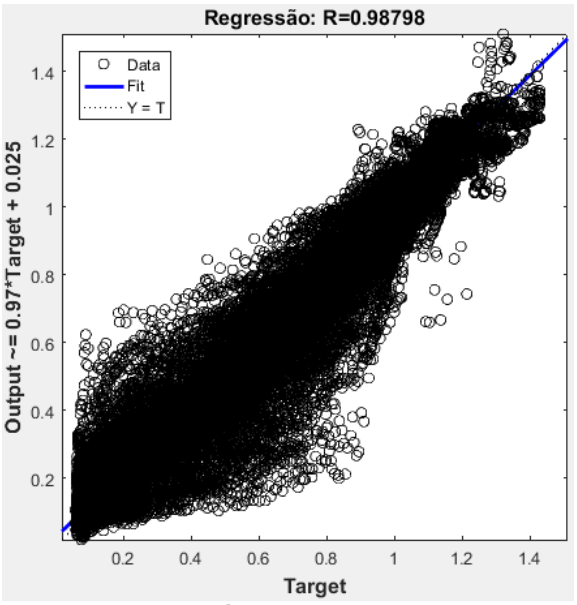
Tabela 33 - Informações sobre o treinamento com 48 neurônios

Informações do Treinamento	Dados
MSE	8.7321e-04
Regressão	0.98798
Épocas	1000
Melhor Época	1000
Tempo de Treinamento	10min

Fonte: Autoria própria

O gráfico da regressão do treinamento apresentado no Gráfico 34 é mais homogêneo o espalhamento dos pontos de dados em comparação com o Gráfico 31 referente ao treinamento da primeira arquitetura.

Gráfico 34 - Taxa de Regressão para o algoritmo Rprop com 48 neurônios



Fonte: Autoria própria

As taxas de erro e a taxa de treinamento do algoritmo com 48 neurônios foi bem parecido com o treinamento do algoritmo com 24 neurônios. Apenas o erro ε ≤ 1% que teve uma variação de mais de 1%, os outros erros tiveram uma variação de menos de 1% para mais ou para menos. A taxa de acerto também se manteve muito parecida, obtendo uma diferença de apenas 0.54% menor do que a taxa de acerto da primeira arquitetura.

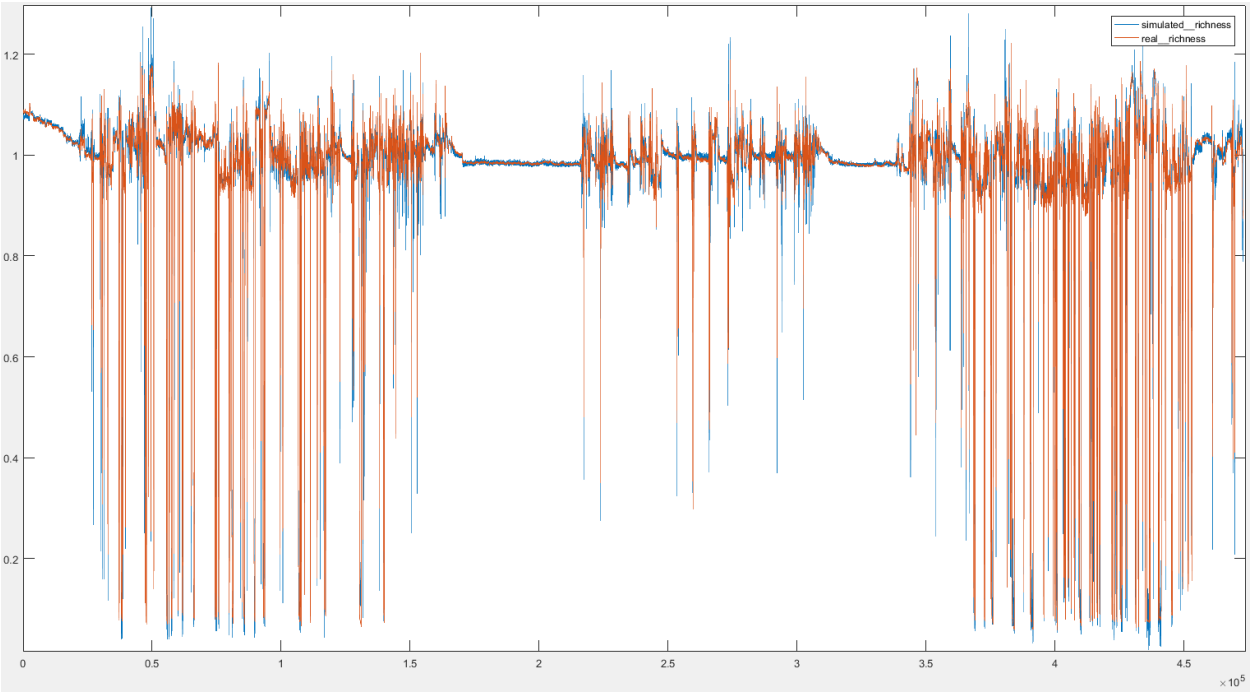
Tabela 34 - Taxas de erro e taxa de acerto do algoritmo Rprop no treinamento com 48 neurônios

ε ≤ 1%	ε ≤ 3%	ε ≤ 5%	ε ≤ 10%	ε > 10%	Taxa de Acerto
66.91	24.09	4.81	2.63	1.56	91.00%

Fonte: Autoria própria

Como as taxas de erro e a taxa de acerto variaram pouquíssimo entre a primeira e a segunda arquitetura, o gráfico entre o valor real e o valor do treinamento das duas arquiteturas são praticamente idênticos, como é possível observar no Gráfico 35.

Gráfico 35 - Valor real (laranja) e o valor do treinamento (azul) no algoritmo Rprop com 48 neurônios



Fonte: Autoria própria

Diferentemente das taxas de erro e da taxa de acerto no treinamento, na simulação não aconteceu o mesmo e houve uma maior variação. O erro ε ≤ 1%

diminuiu 1.35% e o erro ε ≤ 3% diminuiu 3.32%, resultando em uma taxa de acerto de 79.39%, ainda alta, porém abaixo da taxa de acerto da primeira arquitetura.

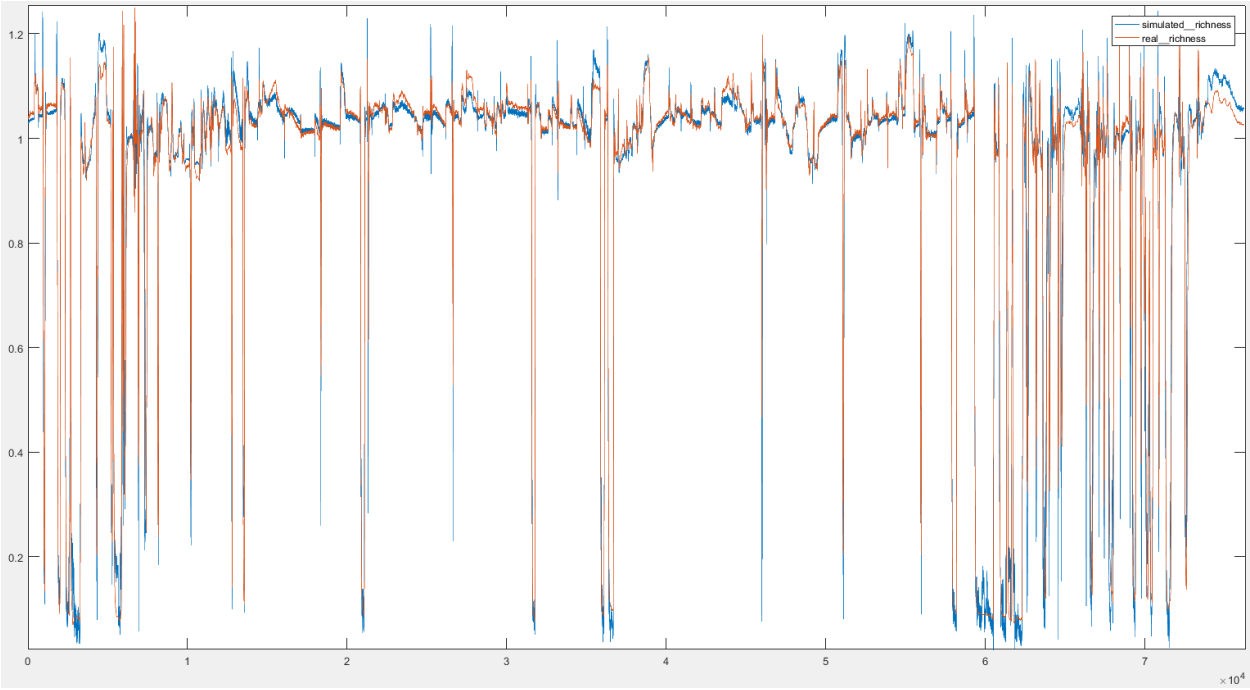
Tabela 35 - Taxas de erro e taxa de acerto do algoritmo Rprop na simulação com 48 neurônios

ε ≤ 1%	ε ≤ 3%	ε ≤ 5%	ε ≤ 10%	ε > 10%	Taxa de Acerto
44.68	34.71	9.84	5.75	5.02	79.39%

Fonte: Autoria própria

No Gráfico 36 entre o valor da simulação e o valor real é possível perceber que o valor da simulação não segue tão corretamente o valor real, pois a taxa de acerto é alta, porém é bem distribuída entre os erros ε ≤ 1% e ε ≤ 3%.

Gráfico 36 - Valor real (laranja) e o valor da simulação (azul) no algoritmo Rprop com 48 neurônios



Fonte: Autoria própria

5.6.3 Resilient Backpropagation com 64 Neurônios na Camada Oculta

O treinamento com o algoritmo Rprop na terceira arquitetura com 64 neurônios na camada oculta produziu um valor de MSE totalmente diferente do treinamento do algoritmo com as duas primeiras arquiteturas. O MSE obtido 0.0015 é ótimo, pois está

mais próximo de 0, enquanto a primeira arquitetura obteve um MSE de 9.4539e-04 e a segunda arquitetura 8.7321e-04.

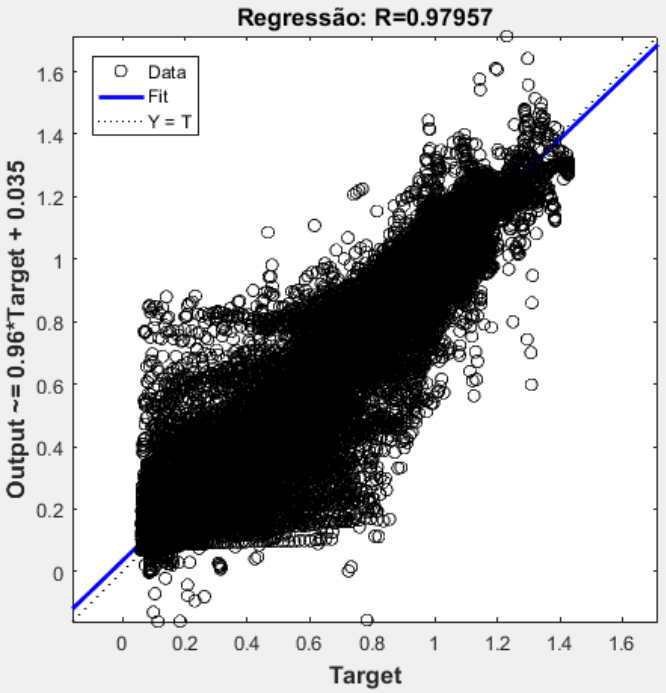
Tabela 36 - Informações sobre o treinamento com 64 neurônios

Informações do Treinamento	Dados
MSE	0.0015
Regressão	0.97957
Épocas	1000
Melhor Época	1000
Tempo de Treinamento	14min

Fonte: Autoria própria

Assim como nas duas primeiras arquiteturas, o treinamento do algoritmo terminou devido ao limite máximo de épocas atingidas, como é possível observar na Tabela 36. O tempo de treinamento também foi baixo, apenas 8 minutos a mais que a primeira arquitetura e 4 minutos a mais que a segunda arquitetura. A regressão do treinamento obteve um valor de 0.97957, ficando abaixo da primeira arquitetura que teve uma regressão de 0.98698 e também abaixo da segunda arquitetura com uma regressão de 0.98798. O Gráfico 37 apresenta a taxa de regressão do treinamento.

Gráfico 37 - Taxa de Regressão para o algoritmo Rprop com 64 neurônios



Fonte: Autoria própria

A taxa de acerto do treinamento do algoritmo é apresentada na Tabela 37. O erro ε ≤ 1% diminuiu cerca de 10% em relação as duas outras arquiteturas, enquanto o erro ε ≤ 3% aumentou cerca de 5%, fazendo com que a taxa de acerto do algoritmo decaísse 5.59% em relação a primeira arquitetura e 5.05% em relação a segunda arquitetura.

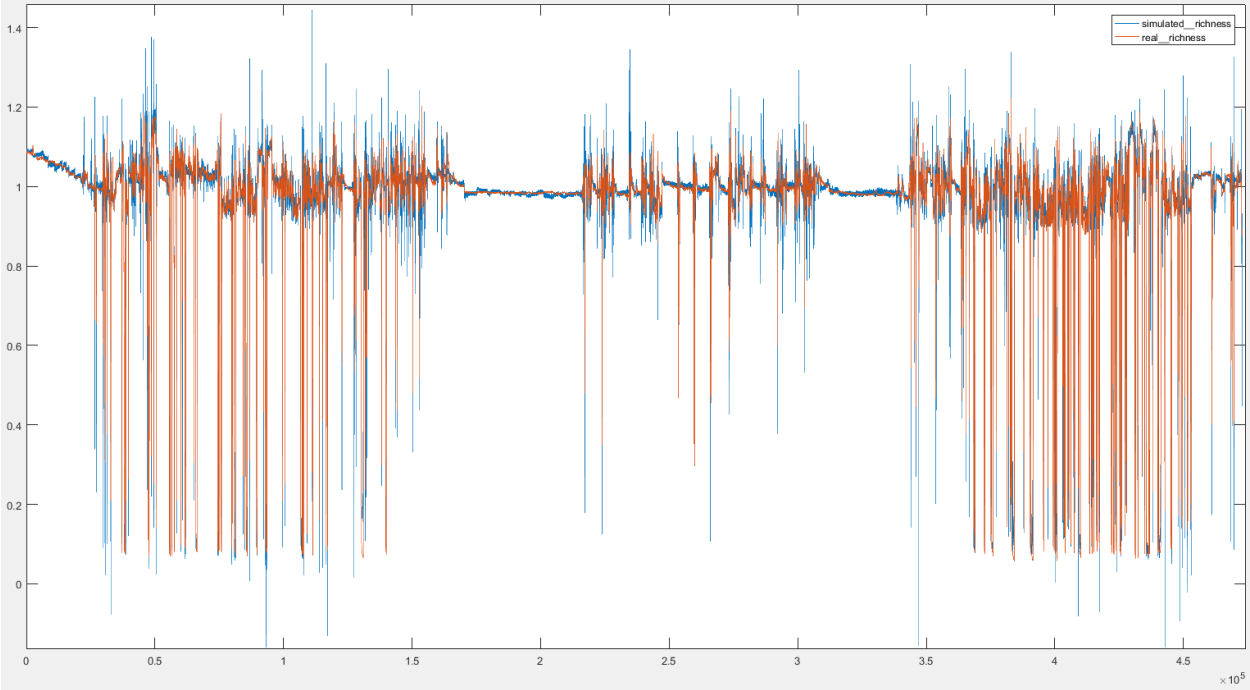
Tabela 37 - Taxas de erro e taxa de acerto do algoritmo Rprop no treinamento com 64 neurônios

ε ≤ 1%	ε ≤ 3%	ε ≤ 5%	ε ≤ 10%	ε > 10%	Taxa de Acerto
56.83	29.12	7.34	4.22	2.49	85.95%

Fonte: Autoria própria

Com a diminuição da taxa de acerto, juntamente com o decaimento do erro ε ≤ 1% e do aumento do erro ε ≤ 3%, o gráfico entre o valor real e o valor do treinamento apresentado no Gráfico 38 apresenta erros que não apareciam nas duas primeiras arquiteturas.

Gráfico 38 - Valor real (laranja) e o valor do treinamento (azul) no algoritmo Rprop com 64 neurônios



Fonte: Autoria própria

A taxa de acerto no treinamento decaindo cerca de 5% em relação as duas primeiras arquiteturas, fez com que a taxa de simulação também ficasse abaixo das duas outras arquiteturas. Em comparação com a primeira arquitetura, a taxa de acerto

da simulação foi 20.97% menor e em comparação com a segunda arquitetura foi 16.3% menor.

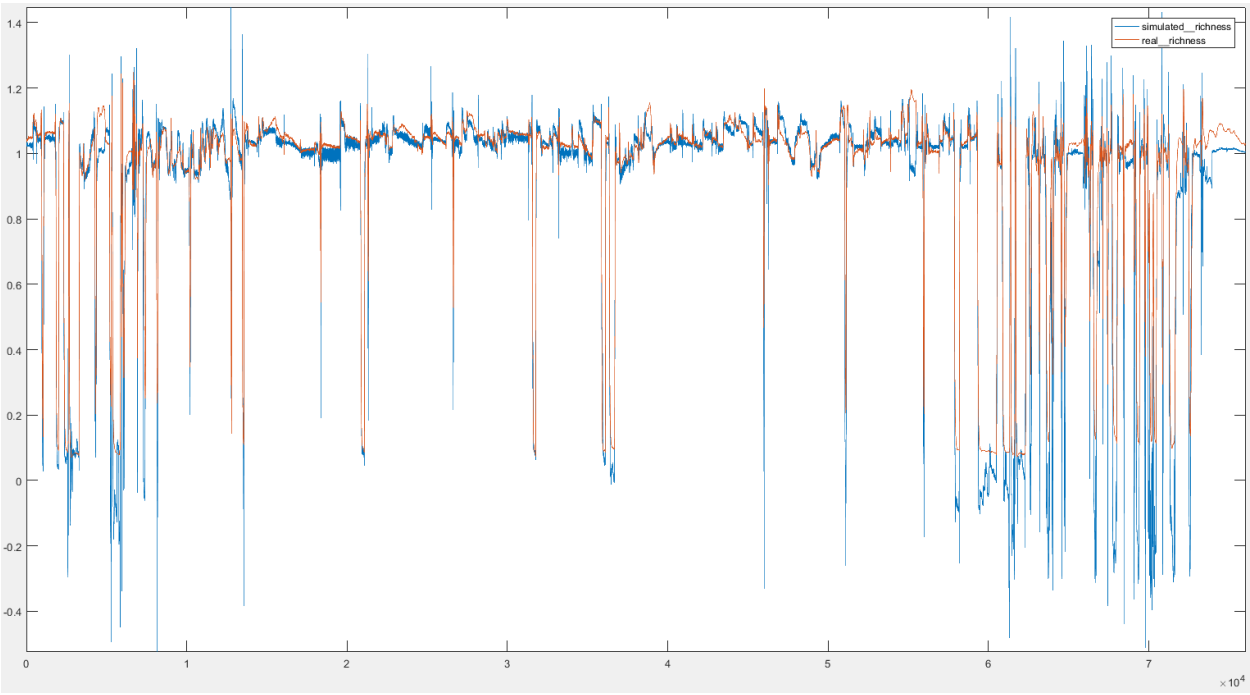
Tabela 38 - Taxas de erro e taxa de acerto do algoritmo Rprop na simulação com 64 neurônios

ε ≤ 1%	ε ≤ 3%	ε ≤ 5%	ε ≤ 10%	ε > 10%	Taxa de Acerto
25.22	37.87	13.77	10.33	12.81	63.09%

Fonte: Autoria própria

Como a taxa de acerto na simulação não foi alta, é possível observar com maior facilidade os erros entre o valor real e o valor simulado no Gráfico 39.

Gráfico 39 - Valor real (laranja) e o valor da simulação (azul) no algoritmo Rprop com 64 neurônios



Fonte: Autoria própria

5.7 CRITÉRIOS DE COMPARAÇÃO DO DESEMPENHO ENTRE OS ALGORITMOS

O objetivo principal e geral do trabalho é prover uma comparação entre os diferentes algoritmos para treinamento de redes neurais aplicados a problemas não lineares, como o estudo de caso que é um motor a combustão interna. Para poder prover uma comparação honesta e de qualidade, visando mostrar os pontos fortes de fracos de cada algoritmo aplicado em cada arquitetura, foram definidos parâmetros

fixos que serão comparados e o grau de importância de cada parâmetro. Com o desenvolvimento da dissertação ao longo do tempo foi observado que nem sempre boas taxas de performance e taxas de acerto no treinamento resultam em boas taxas acerto de simulação.

São 5 os critérios de avaliação de cada algoritmo: porcentagem de acerto na simulação (diferença entre o valor simulado e o valor real), porcentagem de acerto no treinamento (diferença entre o valor treinado e o valor real), taxa de regressão, performance do treinamento (MSE) e tempo de convergência.

Para poder classificar os algoritmos facilmente, foram criados critérios baseados em pontuação, sendo que dentre os 5 critérios de avaliação existe um intervalo de pontos que o algoritmo pode receber pelo seu desempenho. O intervalo de pontos é em função do quão importante acreditamos que é o critério, sendo assim, a taxa de acerto na simulação do algoritmo tem uma faixe de pontos maior que o tempo de treinamento, pois se o algoritmo produz um bom resultado não há problema em demorar levar mais tempo de treinamento que outro algoritmo que não produza um bom resultado mas tenha um tempo de treinamento menor.

O primeiro critério é a Taxa de Acerto na Simulação e seu intervalo de pontos vai de 10 (maior pontuação) até 6 (menor pontuação), como apresentado na Tabela 39. O algoritmo só pontua se sua taxa de acerto for maior ou igual a 75%, abaixo desse valor o algoritmo não pontua.

Simulação

Tabela 39 – Pontuação para as Taxas de Acerto dos Algoritmos na

Taxa de Acerto (Simulação)	Pontos
Maior que 95%	10
Maior que 90%	9
Maior que 85%	8
Maior que 80%	7
Maior/Igual a 75%	6

Fonte: Autoria própria

O segundo critério de avaliação é a taxa de acerto no treinamento, apresentado na Tabela 40. O intervalo em que o algoritmo pode pontuar é o mesmo da taxa de acerto na simulação, porém os pontos que cada taxa de treinamento pode receber é de 8 (maior pontuação) até 1 (menor pontuação). A pontuação no treinamento é menor que na simulação pois teoricamente o algoritmo produz melhores

resultados no treinamento que na simulação e não necessariamente um algoritmo que produz um bom resultado no treinamento também produz um bom resultado na simulação.

Treinamento

Tabela 40 - Pontuação para as Taxas de Acerto dos Algoritmos no

Taxa de Acerto (Treinamento)	Pontos
Maior que 95%	8
Maior que 90%	6
Maior que 85%	4
Maior que 80%	2
Maior/Igual a 75%	1

Fonte: Autoria própria

O terceiro critério de avaliação é a taxa de regressão do algoritmo, apresentado na Tabela 41. Em nossa avaliação, a taxa de regressão tem um peso menor na qualidade do treinamento do algoritmo, sendo assim o máximo de pontos que um algoritmo pode receber é 4, o mínimo é 0.5 e o algoritmo pode também não pontuar se obtiver taxa de regressão menor que 0.95.

Tabela 41 - Pontuação para as taxas de Regressão

Regressão	Pontos
Maior que 0.99	4
Maior que 0.98	3
Maior que 0.97	2
Maior que 0.96	1
Maior/Igual a 0.95	0.5

Fonte: Autoria própria

O quarto critério de avaliação é o MSE apresentado na Tabela 42. O MSE tem o mesmo peso que a Regressão, sendo assim o intervalo de pontos possíveis é igual. A diferença do MSE é que todos os algoritmos pontuam, pois MSE maior ou igual a 8 recebem 0.5 de pontuação.

Tabela 42 - Pontuação para as taxas de MSE

MSE	Pontos
Menor que 2	4
Menor que 4	3
Menor que 6	2
Menor que 8	1
Maior/Igual a 8	0.5

Fonte: Autoria própria

O quinto e último critério de avaliação é o tempo de treinamento do algoritmo, apresentado na Tabela 43. Como explicado anteriormente, o tempo de treinamento tem um peso baixo em nossa avaliação, consequentemente o intervalo de pontos que esse critério recebe é menor. Igual ao MSE, todos os algoritmos pontuam, mesmo se o tempo de treinamento for elevado.

Tabela 43 - Pontuação para os Tempos de Treinamento

Tempo de Treinamento	Pontos
Menor que 6hs	3
Menor que 12hs	2.5
Menor que 18hs	2
Menor que 24hs	1.5
Maior/Igual a 24hs	1

Fonte: Autoria própria

Em caso de empate na pontuação dos algoritmos, o desempate segue a ordem dos critérios de avaliação, sendo assim o primeiro critério de desempate é o algoritmo que obteve uma maior taxa de acerto na simulação, o segundo critério de desempate é o algoritmo que obteve uma maior taxa de acerto no treinamento, o terceiro critério de desempate é o algoritmo que obteve uma maior taxa de regressão, o quarto critério de desempate é o algoritmo que obteve um menor MSE e por fim o quinto critério de desempate é o algoritmo que obteve um menor tempo de treinamento.

5.8 COMPARAÇÃO DO DESEMPENHO ENTRE OS ALGORITMOS

A comparação entre os algoritmos será realizada em duas etapas. Na primeira etapa os algoritmos são comparados pelas arquiteturas que foram aplicadas, com o objetivo de identificar os pontos fortes e fracos de cada algoritmo em cada arquitetura. Na segunda etapa todos os algoritmos e todas as arquiteturas são comparadas igualmente, utilizando os pontos e critérios definidos na seção 5.7, com o objetivo de identificar qual o algoritmo e arquitetura obteve a melhor solução no estudo de caso aplicado.

A primeira comparação é dos algoritmos aplicados na primeira arquitetura com 24 neurônios, o resultado de cada algoritmo é apresentado na Tabela 44. Em cada umas das informações (MSE, Regressão, Épocas, Tempo de Treinamento, Taxa de Acerto no Treinamento e Taxa de Acerto na Simulação) é destacado em negrito o algoritmo que obteve o melhor resultado. Na taxa de MSE, o algoritmo que obteve o melhor resultado foi o Scaled Conjugate Gradient, seguido pelo Levenberg-Marquardt que está quase empatado com o Bayesan Regularization, e por último e com uma grande diferença dos outros três está o Resilient Backpropagation. Na regressão o algoritmo Levenberg-Marquardt obteve o melhor resultado, porém apenas 0.00038 melhor que o algoritmo Bayesan Regularization que obteve o segundo melhor resultado. Com a terceira melhor regressão ficou o algoritmo Resilient Backpropagation e logo em seguida o algoritmo Scaled Conjugate Gradient.

O número total de Épocas não é um requisito descrito na seção 5.7, porém nessa primeira etapa de comparação esse valor será considerado como uma informação extra sobre o desempenho dos algoritmos. Com o melhor resultado no total de épocas está o algoritmo Levenberg-Marquardt, em seguida vem o algoritmo Scaled Conjugate Gradient com 581 épocas a mais que o primeiro algoritmo. Por fim estão empatados com 1000 épocas os algoritmos Bayesan Regularization e Resilient Backpropagation.

Tabela 44 - Informações sobre o treinamento dos algoritmos com a primeira arquitetura (24 neurônios)

Algoritmo	MSE	Regressão	Épocas	Tempo de Treinamento	Taxa de Acerto (Treinamento)	Taxa de Acerto (Simulação)
Levenberg-Marquardt	2.3840e-04	**0.99683**	**204**	2h e 27min	**95.95%**	84.58%
Bayesan Regularization	2.5880e-04	0.99645	1000	11h e 50min	95.61%	**85.21%**
Scaled Conjugate Gradient	**0.0020**	0.97135	785	9min	80.91%	63.29%
Resilient Backpropagation	9.4539e-04	0.98698	1000	**6min**	91.54%	84.06%

Fonte: Autoria própria

Mesmo o algoritmo Resilient Backpropagation terminando o treinamento pelo limite máximo de épocas, o seu tempo de treinamento foi o melhor, totalizando apenas 6 minutos. Em segundo lugar está o algoritmo Scaled Conjugate Gradient com 3 minutos a mais que o primeiro. Já os algoritmos Levenberg-Marquardt e Bayesan Regularization levaram mais tempo para convergir, 2h e 27min e 11h e 50min consecutivamente. O algoritmo com a taxa de acerto mais alta no treinamento foi o Levenberg-Marquardt, com uma diferença pequena para o segundo algoritmo Bayesan Regularization, ambos os algoritmos atingiram uma taxa de acerto no treinamento superior a 95%. Em terceira posição está o algoritmo Resilient Backpropagation, com uma taxa de acerto acima de 90% e em quarto lugar ficou o algoritmo Scaled Conjugate Gradient com uma diferença de 10.63% do terceiro algoritmo. Por fim, em relação a taxa de acerto na simulação, o algoritmo com o melhor resultado foi o Bayesan Regularization, sendo o único algoritmo que superou os 85% de acerto. Em seguido está o algoritmo Levenberg-Marquardt, com uma taxa de acerto de 84.58%, seguido pelo algoritmo Resilient Backpropagation com uma taxa de acerto de 84.06%. Em última colocação ficou o algoritmo Scaled Conjugate Gradient, com uma taxa de acerto bem abaixo dos outros três algoritmos.

Pelas informações apresentadas, chegamos a conclusão que o algoritmo Levenberg-Marquardt obteve o melhor desempenho na primeira arquitetura de 24 neurônios, pois dos quesitos observados na Tabela 44, o algoritmo é o melhor em três deles.

A segunda análise da primeira etapa é do treinamento dos algoritmos com a segunda arquitetura de 48 neurônios, como apresentado na Tabela 45. Novamente o algoritmo Scaled Conjugate Gradient obteve a melhor taxa de MSE, assim como ocorreu na primeira arquitetura, seguido pelos algoritmos Levenberg-Marquardt e Bayesan Regularization, que obtiveram um MSE próximos, ambos abaixo de 2, já o algoritmo Resilient Backpropagation obteve novamente um MSE alto e ficou em último nesse quesito. Assim como na arquitetura anterior, o algoritmo Levenberg-Marquardt obteve a melhor taxa de Regressão, seguido pelo algoritmo Bayesan Regularization, ambos com uma taxa acima de 0.99. Em terceiro ficou o algoritmo Resilient Backpropagation, também com uma taxa de regressão alta, porém abaixo de 0.99. Em última posição ficou o algoritmo Scaled Conjugate Gradient, com uma taxa de regressão bem abaixo dos outros três algoritmos. Diferente do treinamento com a primeira arquitetura, na segunda arquitetura o algoritmo que obteve o menor número de épocas foi o Scaled Conjugate Gradient, com apenas 199 épocas. Em segundo ficou o algoritmo Levenberg-Marquardt com 466 épocas, seguido pelos algoritmos Bayesan Regularization e Resilient Backpropagation empatados com 1000 épocas cada.

Tabela 45 - Informações sobre o treinamento dos algoritmos com a segunda arquitetura (48 neurônios)

Algoritmo	MSE	Regressão	Épocas	Tempo de Treinamento	Taxa de Acerto (Treinamento)	Taxa de Acerto (Simulação)
Levenberg-Marquardt	1.7276e-04	**0.99763**	466	19h e 58min	**97%**	79.53%
Bayesan Regularization	1.9713e-04	0.9973	1000	43h e 39min	96.63%	**82.56%**
Scaled Conjugate Gradient	**0.0043**	0.94018	**199**	**3min**	66.69%	36.44%
Resilient Backpropagation	8.7321e-04	0.98798	1000	10min	91%	79.39%

Fonte: Autoria própria

O tempo de treinamento dos algoritmos surpreendeu, pois o melhor tempo foi menor que o melhor tempo da primeira arquitetura. O algoritmo Scaled Conjugate Gradient obteve o menor tempo de treinamento, com apenas 3 minutos. O algoritmo Resilient Backpropagation também obteve um baixo tempo de treinamento, com 10

minutos. Já os algoritmos Levenberg-Marquardt e Bayesan Regularization foram extremamente mais demorados que os dois primeiros, totalizando um tempo 19h e 58min e 43h e 39min consecutivamente.

Em relação a taxa de acerto dos algoritmos no treinamento, o algoritmo com melhor resultado foi o Levenberg-Marquardt, com uma taxa de acerto de 97%, logo em seguida com uma taxa de acerto de 96.63 ficou algoritmo Bayesan Regularization. Em terceira posição está o algoritmo Resilient Backpropagation, com uma diferença de mais de 5% entre os dois primeiros, porém ainda a taxa de acerto é maior que 90%. Em quarta colocação ficou o algoritmo Scaled Conjugate Gradient, com uma taxa de acerto muito inferior aos três primeiros, conseguindo 66.69%. A taxa de acerto na simulação abaixou em todos os algoritmos quando comparada com a arquitetura anterior, porém ainda houveram altas taxas, como o algoritmo Bayesan Regularization que obteve 82.56%, seguido pelos algoritmos Levenberg-Marquardt com 79.53% e Resilient Backpropagation 79.39%. O único algoritmo que obteve uma baixíssima taxa de acerto foi o Scaled Conjugate Gradient com apenas 36.44%.

Na segunda arquitetura é difícil definir o algoritmo que obteve o melhor desempenho, pois o algoritmo Scaled Conjugate Gradient obteve o melhor resultado em três quesitos, porém sua taxa de acerto no treinamento e na simulação foi a pior, bem abaixo dos outros três algoritmos. Se considerarmos os quesitos de maior importância, que são as taxas de acerto, os algoritmos Bayesan Regularization e Levenberg-Marquardt foram os que obtiveram os melhores resultados e podem ser classificados como os de melhor desempenho na segunda arquitetura.

A terceira comparação da primeira etapa são os algoritmos aplicados na terceira arquitetura, como apresentado na Tabela 46. Diferentemente das duas primeiras arquiteturas analisadas, o algoritmo que obteve o melhor MSE na terceira arquitetura foi o Resilient Backpropagation com 0.0015, superando o algoritmo Scaled Conjugate Gradient que ficou na segunda colocação com 0.0031. Os resultados de Regressão seguiram o mesmo que ocorreu nas duas primeiras arquiteturas, o algoritmo Levenberg-Marquardt com a melhor taxa, seguido pelos algoritmos Bayesan Regularization, Resilient Backpropagation e Scaled Conjugate Gradient. Na quantidade de épocas, o algoritmo Levenberg-Marquardt também obteve o melhor resultado com 354 e os outros três algoritmos seguiram empatados, todos com 1000 épocas. Mesmo com um número de neurônios alto, os algoritmos Resilient Backpropagation e Scaled Conjugate Gradient conseguiram convergir em um baixo

tempo de treinamento, o primeiro com 14min e o segundo com 27min. O algoritmo Levenberg-Marquardt ficou em terceiro lugar com 27hs e 04min e o algoritmo Bayesan Regularization em quarto lugar com 76h e 19min.

Tabela 46 - Informações sobre o treinamento dos algoritmos com a segunda arquitetura (64 neurônios)

Algoritmo	MSE	Regressão	Épocas	Tempo de Treinamento	Taxa de Acerto (Treinamento)	Taxa de Acerto (Simulação)
Levenberg-Marquardt	1.6781e-04	**0.9977**	**354**	27h e 04min	**96.98%**	80.75%
Bayesan Regularization	1.7941e-04	0.99754	1000	76h e 19min	96.70%	**82.32%**
Scaled Conjugate Gradient	0.0031	0.9565	1000	27min	73.19%	45.40%
Resilient Backpropagation	**0.0015**	0.97957	1000	**14min**	85.95%	63.09%

Fonte: Autoria própria

Os percentuais dos algoritmos nas taxas de acerto no treinamento e na simulação se manteram consistentes, sendo que os algoritmos ficaram nas mesmas posições em ambos as taxas. Sendo o algoritmo Levenberg-Marquardt com o melhor resultado na taxa de acerto no treinamento e o algoritmo Bayesan Regularization com o melhor resultado na taxa de acerto na simulação.

Assim como na segunda arquitetura, é difícil definir o algoritmo que obteve o melhor desempenho, pois o algoritmo Levenberg-Marquardt e Resilient Backpropagation obtiveram melhores resultados em dois quesitos, porém ambos têm uma taxa de acerto na simulação menor que o algoritmo Bayesan Regularization. Devido à dificuldade de definição de melhor desempenho entre os algoritmos que foi criado os critérios apresentados na seção 5.7 e serão utilizados a seguir na segunda etapa de comparação

Nesta segunda etapa de comparação todos ao algoritmos e arquiteturas serão comparados igualmente com base na pontuação da seção 5.7. A Tabela 47 apresenta os algoritmos/arquiteturas ordenados pela pontuação total de cada um em ordem decrescente. Na primeira coluna da Tabela 47 está qual é o algoritmo e em parênteses o número de neurônios da arquitetura, sendo que 24 neurônios representam a primeira arquitetura, 48 neurônios a segunda arquitetura e 64 neurônios a terceira

arquitetura. Da coluna 2 a 6 estão as informações que são consideradas nos critérios de avaliação, os parênteses dentro das informações do algoritmo são referentes a pontuação que o algoritmo obteve (8p = 8 pontos, 4p = 4 pontos e assim por diante).

O treinamento do algoritmo Bayesan Regularization na primeira arquitetura obteve a maior pontuação, totalizando 25.5 pontos, sendo que grande parte desses pontos são pelas altas taxas de acerto na simulação e no treinamento, que são os principais quesitos avaliados. Mesmo sendo necessário quase 12 horas para o algoritmo convergir, as taxas de acerto (treinamento e simulação), regressão e MSE compensam, pois são ótimas quando comparadas com os outros algoritmos. O segundo algoritmo com o melhor desempenho foi o Levenberg-Marquardt também na primeira arquitetura, totalizando 25 pontos, apenas 0.5 abaixo do primeiro colocado. É interessante observar que o algoritmo Levenberg-Marquardt (24 neurônios) obteve melhores resultados de Taxa de Acerto (Treinamento), Regressão, MSE e Tempo de Treinamento que o algoritmo Bayesan Regularization (24 neurônios) que ficou com a primeira posição, porém o que diferenciou os dois algoritmos foi a Taxa de Acerto na Simulação, colocando o algoritmo Bayesan Regularization (24 neurônios) a frente do Levenberg-Marquardt (24 neurônios).

Na terceira posição temos o algoritmo Resilient Backpropagation (24 neurônios) com 24 pontos, empatado pelo número de pontos com os algoritmos Bayesan Regularization (48 neurônios), Bayesan Regularization (64 neurônios) e Levenberg-Marquardt (64 neurônios). Como explicado na seção 5.7, o critério de desempate de pontos utilizado nesse caso foi a porcentagem da taxa de acerto na simulação.

Analisando os três primeiros algoritmos/arquiteturas com maior pontuação, podemos concluir que a arquitetura que obteve maior desempenho do treinamento foi a primeira com 24 neurônios, pois os três melhores resultados foram obtidos com essa arquitetura. A segunda conclusão que podemos ter sobre a Tabela 47 é que analisando os 6 primeiros colocados com maior pontuação, os algoritmos com maior desempenho foram Bayesan Regularization e Levenberg-Marquardt, independente da arquitetura que foi utilizada, pois eles aparecem em 5 posições das 6 primeiras, sendo que o algoritmo Bayesan Regularization está entre as 6 primeiras com todas as arquiteturas (24, 48 e 64 neurônios). Entre as 6 primeiras o algoritmo Levenberg-Marquardt aparece duas vezes, com a primeira arquitetura (24 neurônios) na segunda posição, e com a terceira arquitetura (48 neurônios) na sexta posição. Na sétima

posição está também o algoritmo Levenberg-Marquardt com a segunda arquitetura (48 neurônios) apenas 0.5 abaixo da sexta posição.

Tabela 47 - Pontuação final do desempenho de cada algoritmo em ordem decrescente

Algoritmo	Taxa de Acerto (Simulação)	Taxa de Acerto (Treinamento)	Regressão	MSE	Tempo de Treinamento	Total de Pontos
Bayesan Regularization (24 neurônios)	85.21% (8p)	95.61% (8p)	0.99645 (4p)	2.5880e-04 (3p)	11h e 50min (2.5p)	25.5
Levenberg-Marquardt (24 neurônios)	84.58% (7p)	95.95% (8p)	0.99683 (4p)	2.3840e-04 (3p)	2h e 27min (3p)	25
Resilient Backpropagation (24 neurônios)	84.06% (7p)	91.54% (6p)	0.98698 (3p)	9.4539e-04 (0.5)	6min (3p)	24
Bayesan Regularization (48 neurônios)	82.56% (7p)	96.63% (8p)	0.9973 (4p)	1.9713e-04 (4p)	43h e 39min (1p)	24
Bayesan Regularization (64 neurônios)	82.32% (7p)	96.70% (8p)	0.99754 (4p)	1.7941e-04 (4p)	76h e 19min (1p)	24
Levenberg-Marquardt (64 neurônios)	80.75% (7p)	96.98% (8p)	0.9977 (4p)	1.6781e-04 (4p)	27h e 04min (1p)	24
Levenberg-Marquardt (48 neurônios)	79.53% (6p)	97% (8p)	0.99763 (4p)	1.7276e-04 (4p)	19h e 58min (1.5p)	23.5
Resilient Backpropagation (48 neurônios)	79.39% (6p)	91% (6p)	0.98798 (3p)	8.7321e-04 (0.5)	10min (3p)	18.5
Resilient Backpropagation (64 neurônios)	63.09% (-)	85.95% (4p)	0.97957 (2p)	0.0015 (4p)	14min (3p)	13
Scaled Conjugate Gradient (24 neurônios)	63.29% (-)	80.91% (2p)	0.97135 (2p)	0.0020 (4p)	9min (3p)	11
Scaled Conjugate Gradient (64 neurônios)	45.40% (-)	73.19% (-)	0.9565 (0.5)	0.0031 (4p)	27min (3p)	7.5
Scaled Conjugate Gradient (48 neurônios)	36.44% (-)	66.69% (-)	0.94018 (-)	0.0043 (4p)	3min (3p)	7

Fonte: Autoria própria

Da oitava posição em diante os algoritmos obtiveram um total de pontuação bem abaixo dos 7 primeiros, sendo que o algoritmo Resilient Backpropagation (48 neurônios) que está na oitava posição foi o único que pontuou no quesito Taxa de Acerto (Simulação), e os dois últimos colocados Scaled Conjugate Gradient (64 neurônios) e Scaled Conjugate Gradient (48 neurônios) além de não pontuarem na taxa de acerto da simulação, também não pontuaram na taxa de acerto do treinamento.

A terceira conclusão que podemos obter da Tabela 47 é que os algoritmos com piores desempenhos foram Scaled Conjugate Gradient, que ficou com as três últimas posições e o algoritmo Resilient Backpropagation, com as posições 8 e 9, porém obteve a terceira colocação quando treinado com a primeira arquitetura.

Podemos concluir que os critérios e pontuações definidos na seção 5.7 conseguiram classificar bem o desempenho dos algoritmos nas diferentes arquiteturas, podendo visualizar claramente quais os algoritmos e arquiteturas obtiveram os melhores desempenhos e quais os que não conseguiram obter bons desempenhos.

6 CONCLUSÃO

Este trabalho apresentou um estudo comparativo entre os algoritmos Levenberg-Marquardt, Bayesian Regularization Scaled Conjugate Gradient e Resilient Backpropagation no treinamento de redes neurais artificiais para a resolução de problemas não-lineares, utilizando como estudo de caso um motor a combustão interna ciclo Otto com o objetivo de treinar a rede neural para aprender a formação da mistura ar-combustível.

Para realizar o treinamento dos algoritmos, o trabalho também apresentou uma ferramenta desenvolvida em MATLAB para facilitar o processo de manipulação dos dados, configuração e treinamento da rede neural.

Cada algoritmo foi treinado com três arquiteturas diferentes de redes neurais, a primeira com 24 neurônios na camada oculta, a segunda com 48 neurônios na camada oculta e a terceira com 64 neurônios na camada oculta, produzindo 12 treinamentos diferentes entre os algoritmos. Para poder classificar e avaliar o desempenho dos algoritmos, o trabalho também apresentou critérios de avaliação dos algoritmos baseado em pontuações, sendo assim cada algoritmo recebeu uma quantidade de pontos por cada critério e seu desempenho final foi classificado pelo total de pontos somados do algoritmo.

Com a avaliação do desempenho de cada algoritmo em cada arquitetura, foi possível observar que a arquitetura em que o algoritmo foi treinado influencia diretamente em seu resultado, sendo que os três melhores desempenhos dos algoritmos foram utilizando a primeira arquitetura com 24 neurônios na camada oculta. Outro fator observado é que algoritmos que obtiveram resultados de desempenho semelhantes possuem tempo de treinamento muito distintos, chegando a ter mais de três dias de diferença entre dois algoritmos que obtiveram a mesma pontuação, o algoritmo Resilient Backpropagation (24 neurônios) que atingiu 24 pontos e teve um tempo de treinamento de 6 minutos contra o algoritmo Bayesan Regularization (64 neurônios) que obteve a mesma pontuação de 24 pontos porém teve um tempo de treinamento de mais de 76 horas.

6.1 TRABALHOS FUTUROS

Os possíveis trabalhos futuros que podem ser realizados a partir desse trabalho são:

- Análise do desempenho dos algoritmos em outro estudo de caso.
- Aplicação dos algoritmos em outras arquiteturas.
- Estudo e análise de desempenho de outros algoritmos de retropropagação.

REFERÊNCIAS

AUSTIN COMMUNITY COLLEGE. **Variable Valve Timing**. Disponível em: <http://www.austincc.edu/wkibbe/vvt.htm>. Acesso em: 10 abr. 2018.

CAY, Y. Prediction of a gasoline engine performance with artificial neural network. **Fuel**, v. 111, p. 324-331, 2013.

CORMEN, T. H. et al. **Introduction to algorithms**. MIT press, 2009.

DENG, J. et al. The applications of artificial neural networks to engines. In: **Artificial neural networks-industrial and control engineering applications**. InTech, 2011.

DHAR, V. K. et al. Comparative performance of some popular artificial neural network algorithms on benchmark and function approximation problems. **Pramana**, v. 74, n. 2, p. 307-324, 2010.

FOGAÇA, J. R. V. **Funcionamento do Motor a Combustão Interna**. Disponível em: <https://mundoeducacao.bol.uol.com.br/quimica/funcionamento-motor-combustao-interna.htm>. Acesso em: 02 fev. 2018.

GAVIN, H. The Levenberg-Marquardt method for nonlinear least squares curve-fitting problems. **Department of Civil and Environmental Engineering, Duke University**, p. 1-15, 2011.

HAGAN, M. T. et al. **Neural network design**. Boston: Pws Pub., 1996.

HAYKIN, S. **Neural networks**: a comprehensive foundation. Prentice Hall PTR, 1994.

JACOBS, R. A. Increased rates of convergence through learning rate adaptation. **Neural networks**, v. 1, n. 4, p. 295-307, 1988.

KAYRI, M. Predictive abilities of bayesian regularization and Levenberg–Marquardt algorithms in artificial neural networks: a comparative empirical study on social data. **Mathematical and Computational Applications**, v. 21, n. 2, p. 20, 2016.

KIŞI, Ö.; UNCUOĞLU, E. Comparison of three back-propagation training algorithms for two case studies. **Indian Journal of Engineering & Materials Sciences**, v. 12, p. 434-442, 2005.

KRENKER, A.; BESTER, J.; KOS, A. Introduction to the artificial neural networks. In: **Artificial neural networks-methodological advances and biomedical applications**. InTech, 2011.

LI, G.; SHI, J. Applications of Bayesian methods in wind energy conversion systems. **Renewable Energy**, v. 43, p. 1-8, 2012.

MATLAB R2016b, The MathWorks, Inc., Natick, Massachusetts, United States.

MCCULLOCH, W. S.; PITTS, W. A logical calculus of the ideas immanent in nervous activity. **The bulletin of mathematical biophysics**, v. 5, n. 4, p. 115-133, 1943.

MORÉ, J. J. The Levenberg-Marquardt algorithm: implementation and theory. In: **Numerical analysis**. Springer, Berlin, Heidelberg, 1978. p. 105-116.

MØLLER, M. F. A scaled conjugate gradient algorithm for fast supervised learning. **Neural networks**, v. 6, n. 4, p. 525-533, 1993.

NELLES, O. **Nonlinear system identification: from classical approaches to neural networks and fuzzy models**. Springer Science & Business Media, 2013.

PARLAK, A. et al. Application of artificial neural network to predict specific fuel consumption and exhaust temperature for a diesel engine. **Applied Thermal Engineering**, v. 26, n. 8, p. 824-828, 2006.

PEREIRA, M. L. **Análise de Gases**: Apostila técnica. Centro de Desenvolvimento de Tecnologia Mecânica – CDTM. Belo Horizonte/MG. Treinamento Técnico Automotivo. 2001, 18p.

PRECHELT, L. Early stopping-but when?. In: **Neural Networks: Tricks of the trade**. Springer, Berlin, Heidelberg, 1998. p. 55-69.

PUJATTI, F. J. P. **Desenvolvimento de um sistema de gerenciamento eletrônico para motores de ignição por centelha**. 2007.

RIEDMILLER, M. **Rprop-description and implementation details**. 1994.

ROSENBLATT, F. The perceptron: A probabilistic model for information storage and organization in the brain. **Psychological review**, v. 65, n. 6, p. 386, 1958.

RUMELHART, D. E.; HINTON, G. E.; WILLIAMS, R. J. **Learning internal representations by error propagation**. California Univ San Diego La Jolla Inst for Cognitive Science, 1985.

TAYLOR, C. F. **The Internal-combustion Engine in Theory and Practice: Combustion, fuels, materials, design**. MIT press, 1985.

THE MATHWORKS. **Neural Network Toolbox**. 2018. Disponível em: <https://www.mathworks.com/products/neural-network.html>. Acesso em: 03 abr. 2018.

TOLLENAERE, T. SuperSAB: fast adaptive back propagation with good scaling properties. **Neural networks**, v. 3, n. 5, p. 561-573, 1990.

YUE, Z.; SONGZHENG, Z.; TIANSHI, L. Bayesian regularization BP Neural Network model for predicting oil-gas drilling cost. In: **Business Management and Electronic Information (BMEI), 2011 International Conference on**. IEEE, 2011. p. 483-487.

APÊNDICE A – Ferramenta Desenvolvida para o Treinamento das Redes Neurais Artificiais

Ferramenta Desenvolvida para o Treinamento das Redes Neurais Artificiais

A ferramenta foi desenvolvida com o objetivo de facilitar o treinamento da rede neural para o estudo de caso, possibilitando muitos ganhos com seu uso, tais como redução de tempo no treinamento, pois facilita a manipulação dos dados, facilidade para realizar o treinamento, adaptação e validação da rede, facilidade para configurar a rede e visualizar os resultados produzidos. A ferramenta não é o foco da dissertação, porém é um resultado extra que trouxe ganhos e auxiliou no processo de desenvolvimento da dissertação.

Figura 20 - Interação entre a ferramenta desenvolvida com o MATLAB e a Neural Network Toolbox

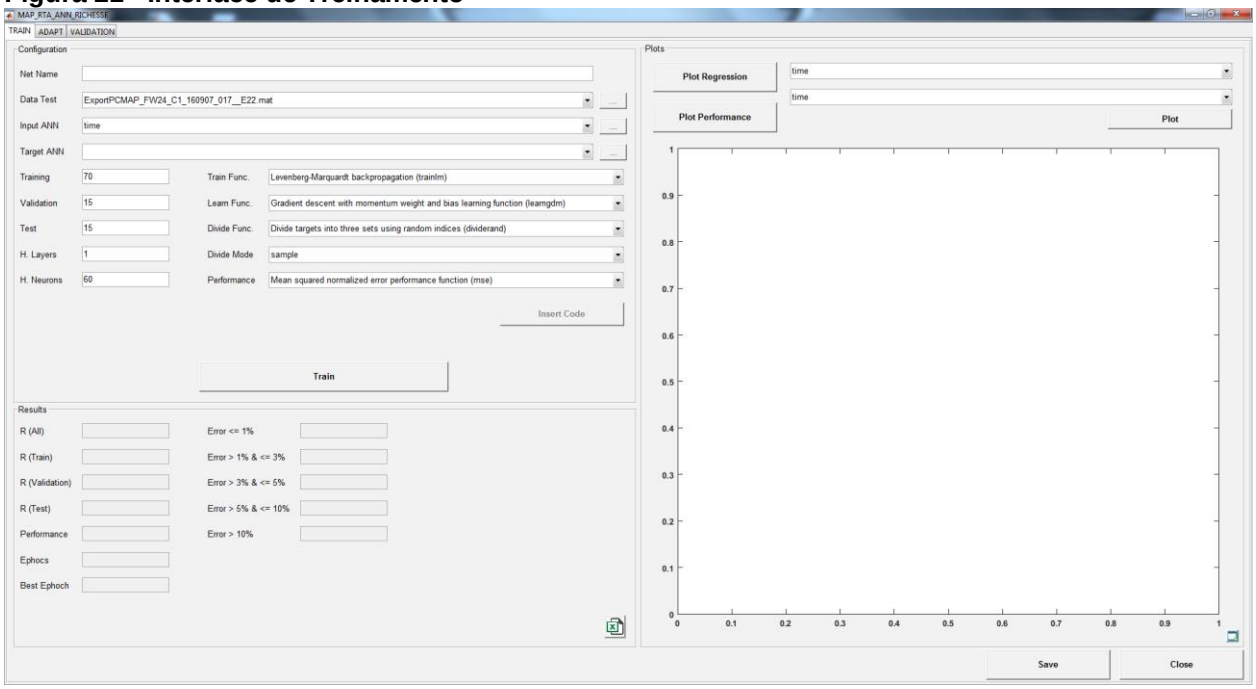


Fonte: Autoria própria

A Figura 20 apresenta a interação entre a ferramenta desenvolvida com o MATLAB e a Toolbox de Rede Neural. A comunicação com o MATLAB é devido ao tratamento dos dados, todos o código de construção da ferramenta e manipulação dos dados é realizado pelo MATLAB, após a manipulação dos dados a ferramenta se comunica com a Neural Network Toolbox para realizar os treinamentos, pois é através da toolbox é possível criar as arquiteturas de rede neural e aplicar os diferentes algoritmos de treinamento.

A interface para treinamento apresentada na Figura 21 é dividida em 3 partes principais, que são: Configuração, Resultados e Gráficos. Todas as 3 partes serão explicadas separadamente a seguir.

Figura 21 - Interface de Treinamento



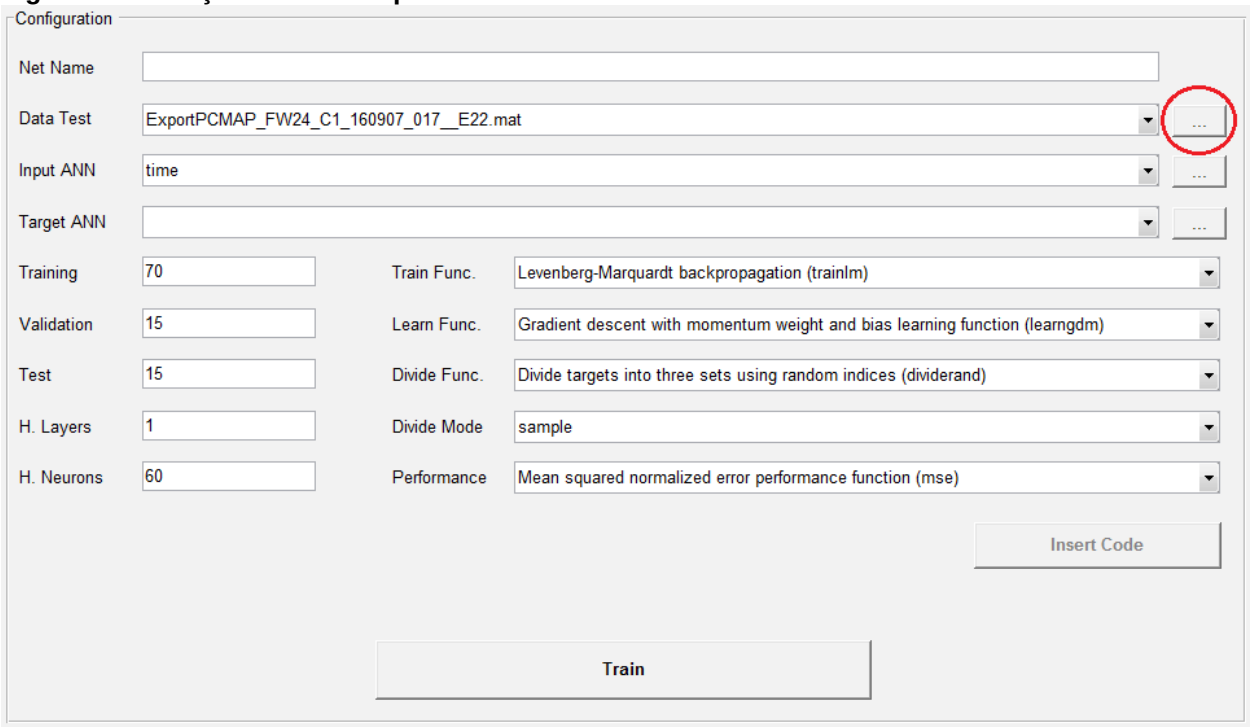
Fonte: Autoria própria

O primeiro passo para treinar uma RNA é configurar os dados de entrada e seus parâmetros.

No campo *Net Name* é necessário informar qual o nome da RNA, que será utilizado para salvar a rede após o treinamento. É recomendado que o nome da RNA não contenha espaços, acentos ou pontos.

O botão do campo *Data Test* ilustrado pela Figura 22 abre a ferramenta *Charge Essais*. O *Charge Essais* é uma ferramenta desenvolvida pela Renault para importar para o MATLAB os dados dos ensaios realizados no veículo. Os ensaios carregados importados ficam listados no campo *Data Test*, sendo possível selecionar entre todos os ensaios carregados.

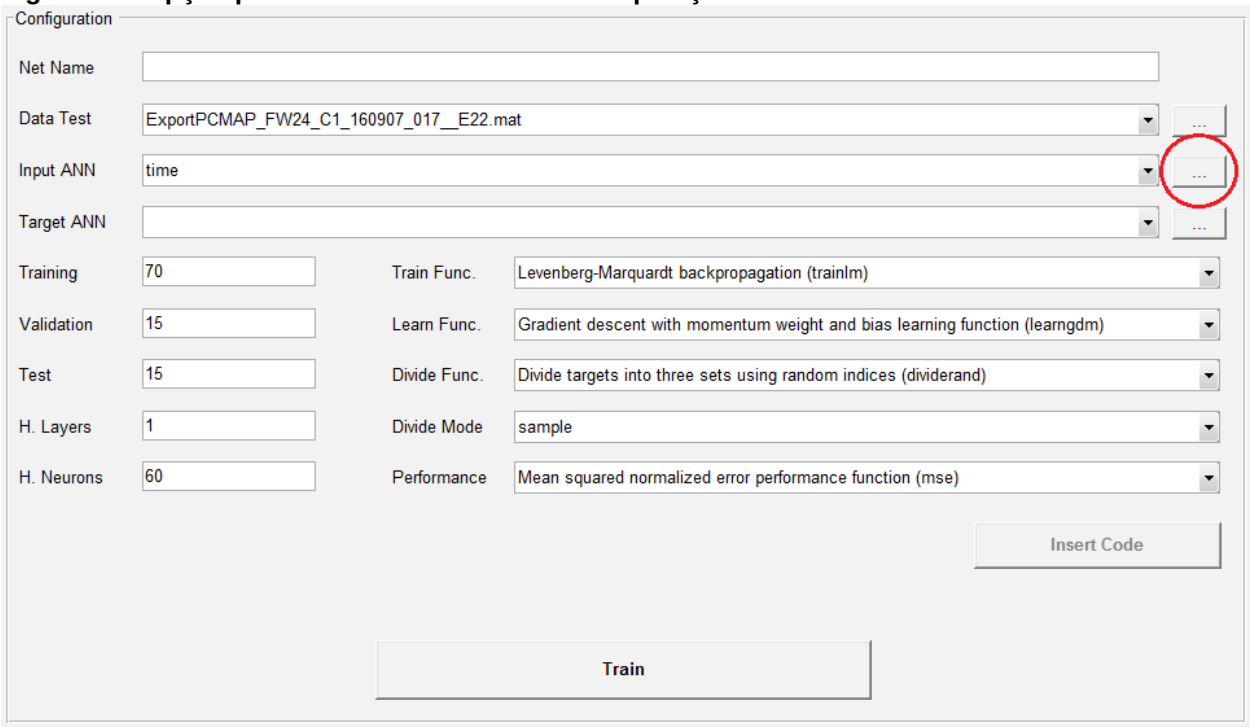
Figura 22 - Seleção dos dados para treinamento



Fonte: Autoria própria

O campo *Input ANN* (Artificial Neural Network) lista todas as variáveis do ensaio selecionado no campo *Data Test*. No campo *Input ANN* é selecionado qual será a entrada para o treinamento. A entrada é formada por um conjunto de variáveis. Para construir um conjunto de variáveis é necessário clicar no botão a direta do campo que seleciona as variáveis, indicado pelo círculo vermelho na Figura 23.

Figura 23 - Opção para abrir a ferramenta de manipulação dos dados



Fonte: Autoria própria.

A interface de configuração de dados tem como objetivo criar o conjunto de dados para o treinamento da rede. Essa interface está ilustrada na Figura 23 e é a principal da ferramenta, onde é possível manipular os dados de diversas maneiras, devido a isso, muitas funções estão disponíveis nessa interface, cada uma delas será explicada individualmente a seguir:

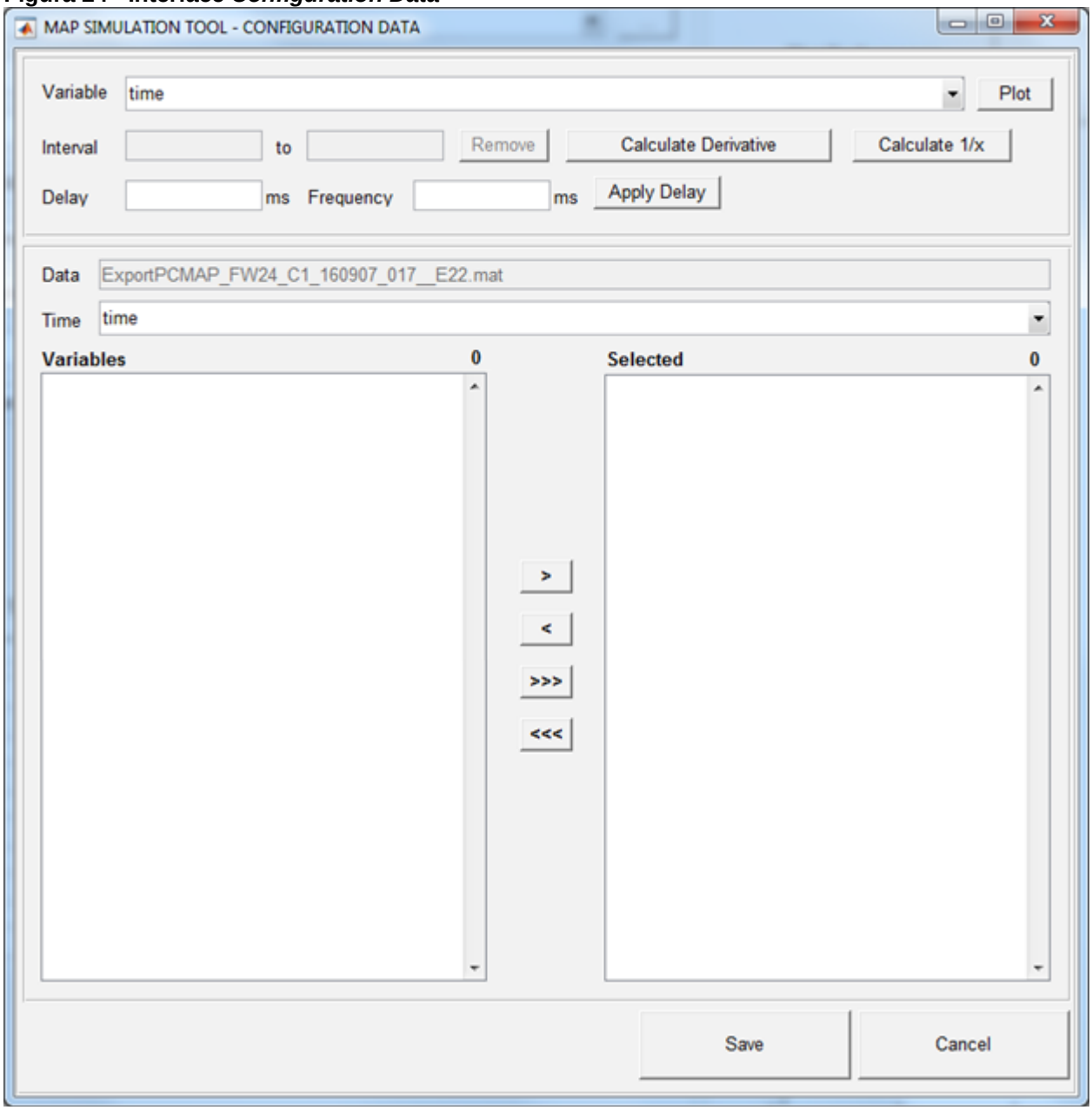
- *Variable*: variável em que as operações serão realizadas, pode ser apenas uma variável ou para realizar alguma operação em todas as variáveis basta selecionar a variável "*All*".
- *Plot*: apresenta o gráfico da variável selecionada, não é possível ver o gráfico quando está selecionada a opção "*All*", somente variáveis individuais.
- *Interval*: remove os pontos entre o intervalo inicial e final informados. Para treinar a rede é necessário que todas as variáveis utilizadas tenham o mesmo número de dados, sendo assim, somente é possível remover os pontos quando a variável selecionada for "*All*", aplicando a remoção em todas variáveis ao mesmo tempo.
- *Calculate Derivative*: calcula a derivada, uma nova variável será criada com a extensão _der no final (a variável original não é afetada). É

possível realizar o cálculo para todas as variáveis ao mesmo tempo, basta deixar a opção "*All*" selecionada. Para calcular a derivada é necessária uma variável de tempo, sendo assim, deve-se selecionar no campo *Time* qual variável será utilizada para calcular a derivada, por padrão a variável *time* (se existir) já vem selecionada.

- *Calculate* 1/x: calcula o inverso da variável (utilizado para transformar o lambda em riqueza). Cria uma nova variável com a extensão _1/x no final (a variável original não é afetada).
- *Delay*: tempo de atraso em *ms* que será aplicado na variável.
- *Frequency*: qual a frequência em que o ensaio foi carregado no *Charge Essais*. A frequência deve estar correta para que o *delay* seja aplicado corretamente.
- *Data*: informa qual o nome do ensaio que está sendo configurado.
- *Time*: para calcular a derivada é necessário ter uma variável de tempo, esse campo seleciona qual a variável de tempo que será utilizada no cálculo da derivada.

Todas as variáveis ficam listadas no quadro *Variables* a esquerda. Para criar o conjunto de dados que será utilizado na rede neural basta selecionar as variáveis desejadas e utilizar os botões centrais com as devidas setas para transferir para o quadro da direita, que apresenta todas as variáveis selecionadas e as que serão utilizadas como entrada no treinamento da rede neural. Para alterar o nome de uma variável listada no quadro *Variables*, basta clicar duas vezes sobre a variável desejada.

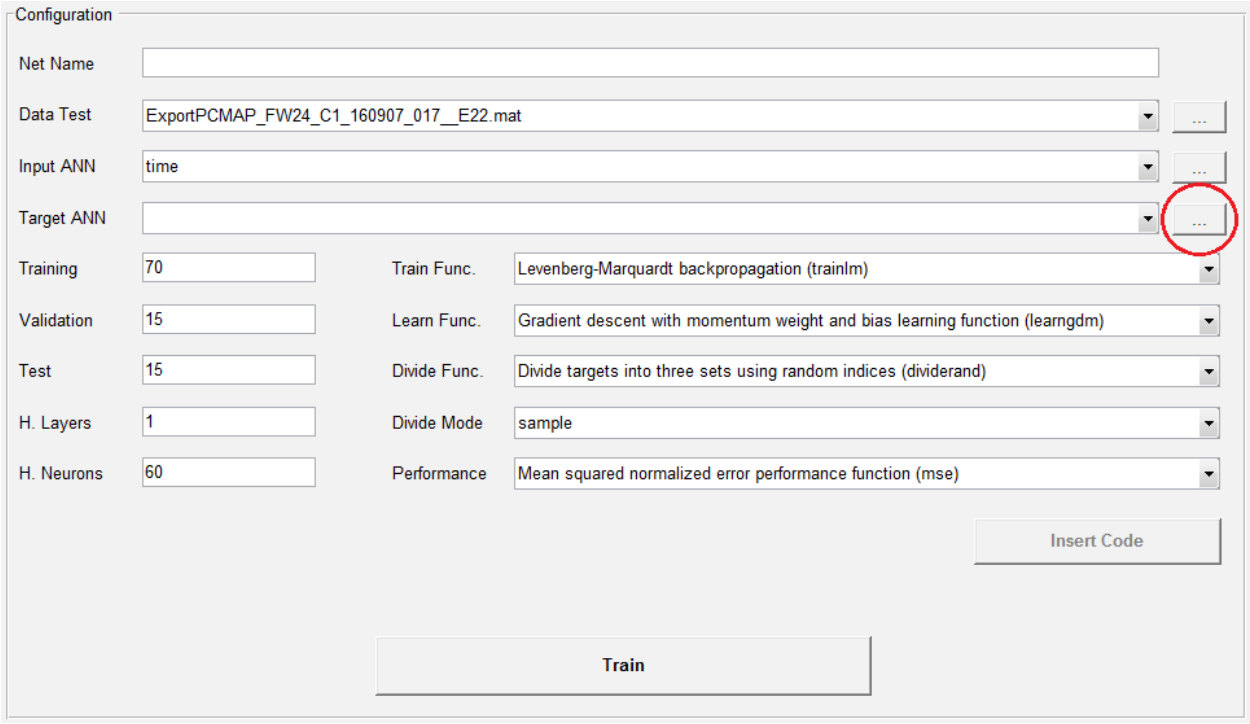
Figura 24 - Interface *Configuration Data*



Fonte: Autoria própria

No campo *Target ANN* é selecionado qual será a saída (objetivo) para o treinamento. A saída é formada por apenas uma variável. A variável de saída só irá aparecer para ser selecionada após a configuração, que é realizada clicando no botão a direita (indicado em vermelho na Figura 25). Após a configuração, a variável será listada pelo campo *Target ANN* e disponível para ser utilizada.

Figura 25 - Opção para abrir a ferramenta de configuração do objetivo de treinamento da rede neural

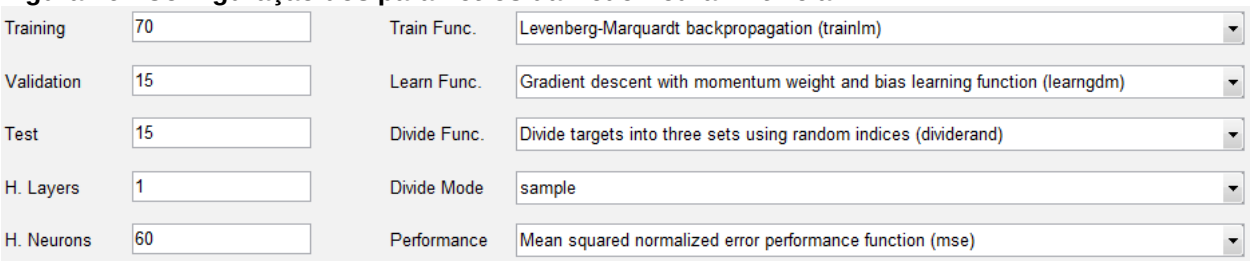


Fonte: Autoria própria

A configuração da variável de saída é idêntica a configuração das variáveis de entrada, todas as funções também estão disponíveis, a única diferença é que somente é possível selecionar uma variável, pois somente uma variável é utilizada como objetivo de saída da rede neural.

É necessário também configurar os parâmetros referentes a rede neural artificial, modificando os parâmetros ilustrados na Figura 26. A configuração é individual de cada RNA, as configurações utilizadas na dissertação serão apresentadas no capítulo 6.

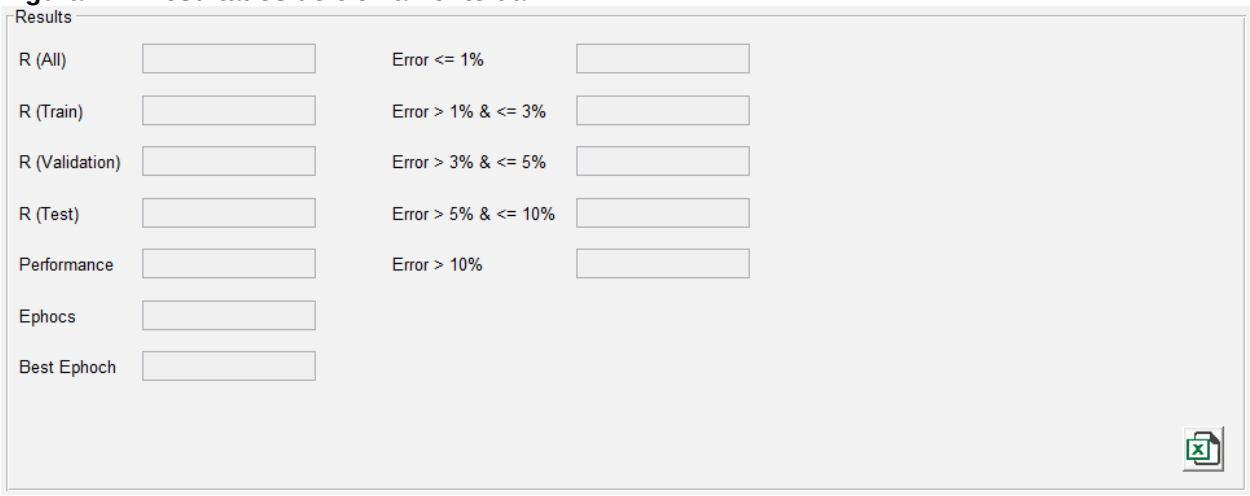
Figura 26 - Configuração dos parâmetros da Rede Neural Artificial



Fonte: Autoria própria.

Após o treinamento os resultados serão apresentados na seção *Results*, apresentados na Figura 27. Todos os treinamentos também são salvos em um arquivo de log, que pode ser acessado através do botão localizado no canto inferior direito.

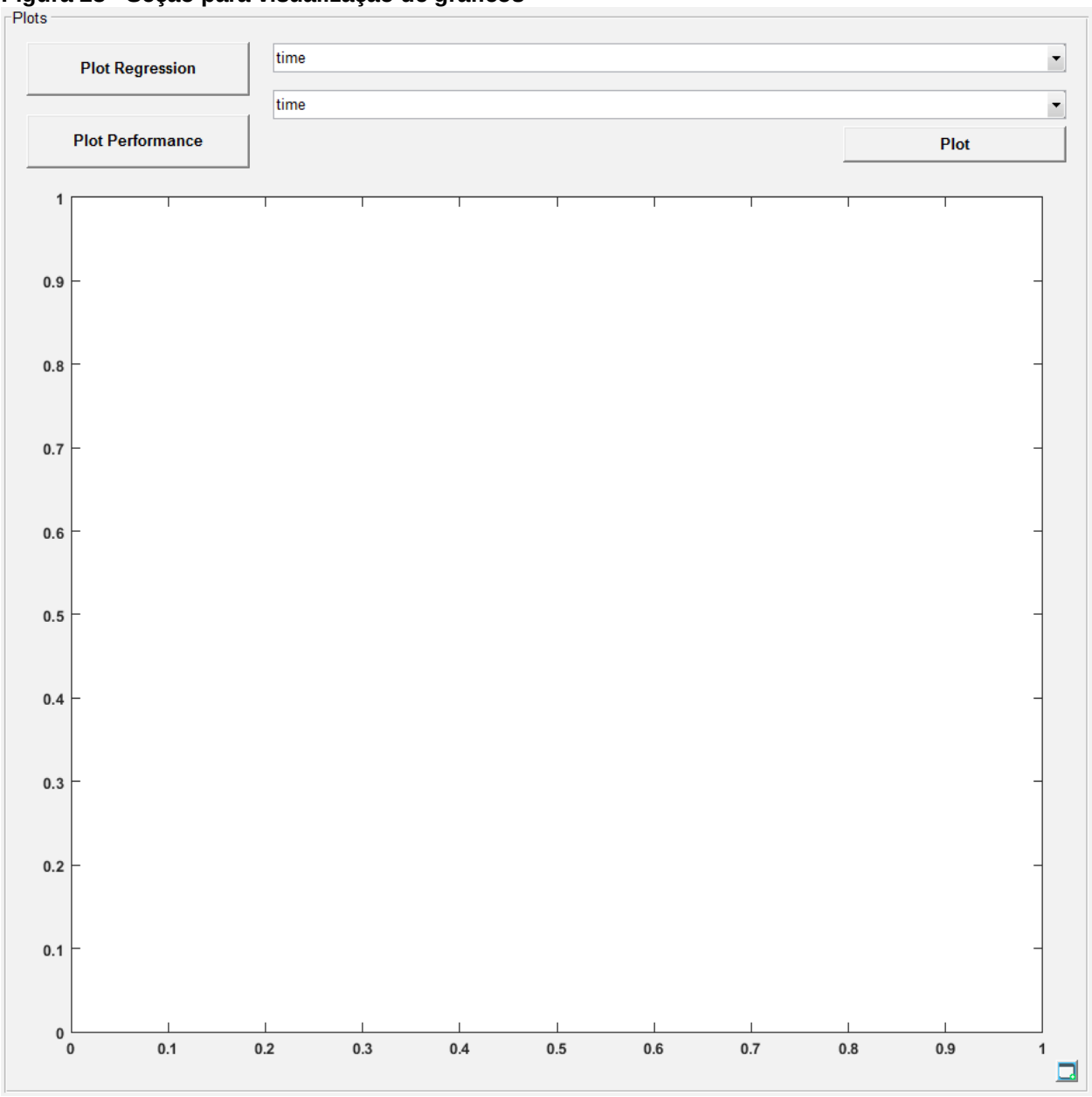
Figura 27 - Resultados do treinamento da RNA



Fonte: Autoria própria.

Na seção *Plots* (Figura 28) é possível visualizar o gráfico das variáveis do ensaio carregado, os gráficos referentes ao treinamento da RNA (*Plot Regression* e *Plot Performance*) e o valor da saída gerada pela RNA, sendo assim é possível comparar o *Target ANN* com o valor gerado pela RNA, selecionando as duas variáveis para gerar o gráfico.

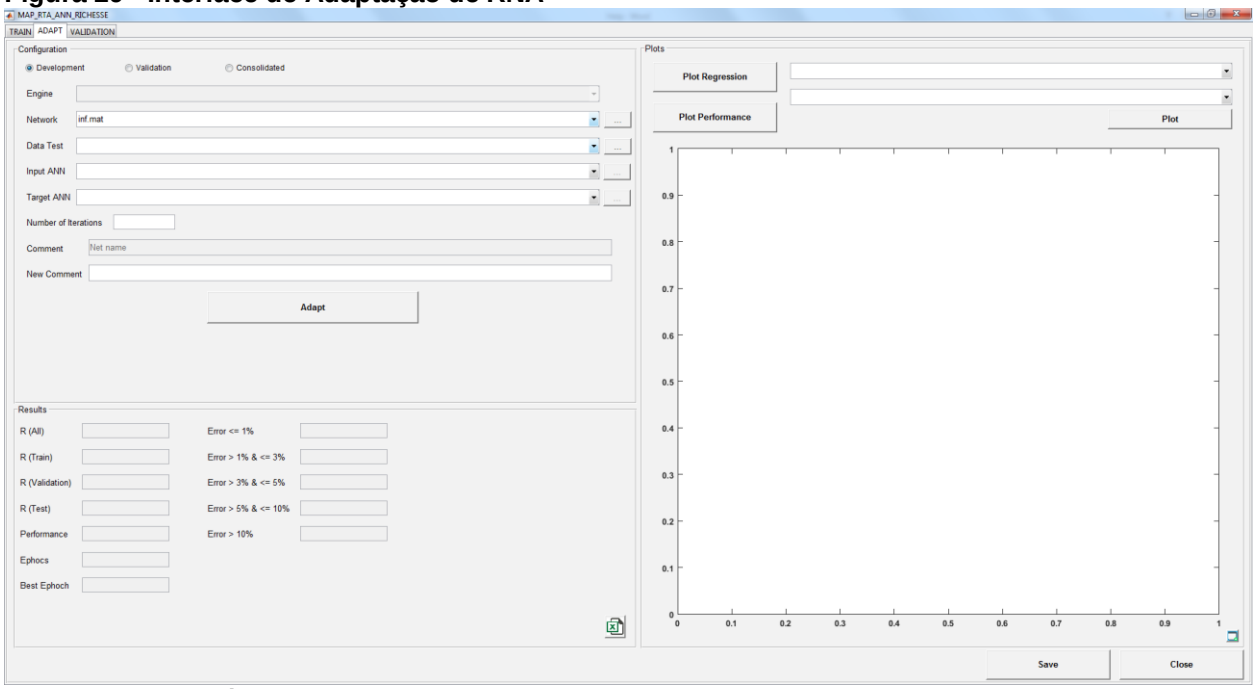
Figura 28 - Seção para visualização de gráficos



Fonte: Autoria própria.

A interface de adaptação da rede segue o padrão da interface de treinamento, a diferença está na seção *Configuration*, porém as seções *Results* e *Plots* são idênticas. A adaptação não substitui a RNA já treinada, é criada uma nova RNA com base no valor da rede já treinada, sendo assim a RNA antiga não é perdida.

Figura 29 - Interface de Adaptação de RNA

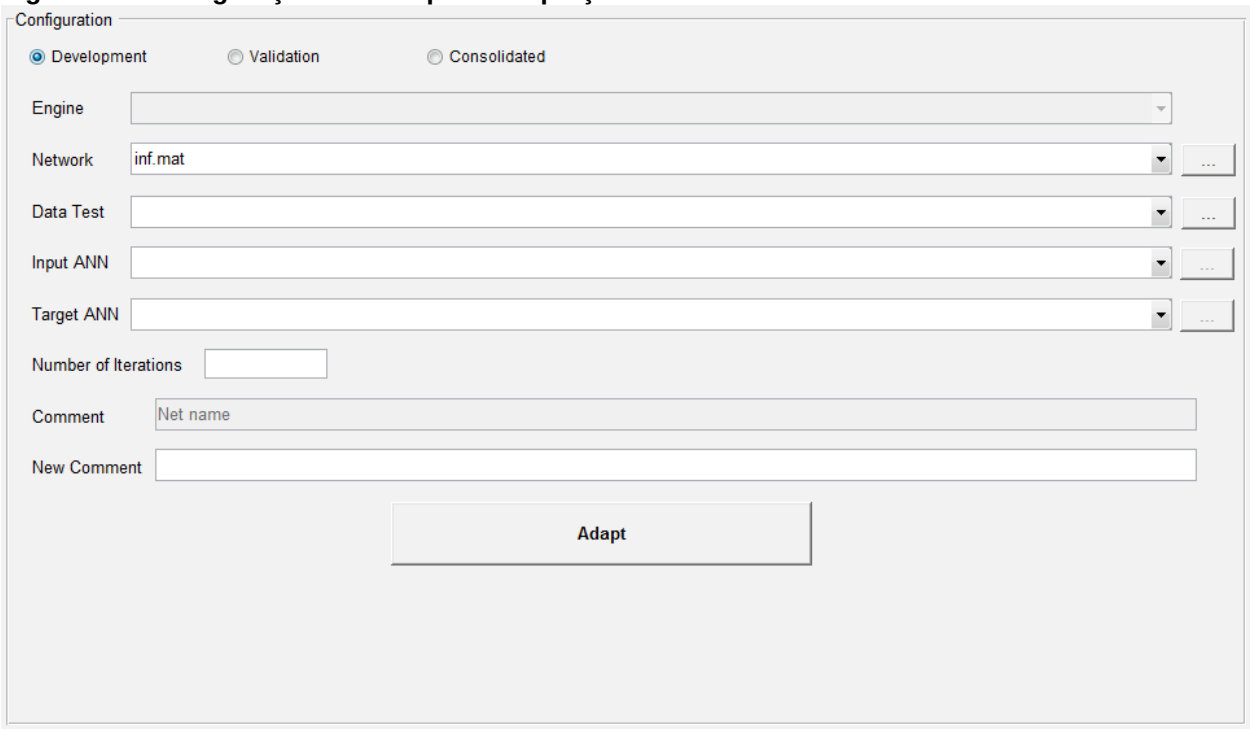


Fonte: Autoria própria

A primeira diferença é a opção de escolha entre *Development*, *Validation* e *Consolidated*. Quando uma RNA é treinada e salva, ela fica disponível na opção *Development*. Nesta opção a rede ainda está em desenvolvimento, logo não está vinculada a nenhum motor, sendo assim, o campo *Engine* fica desabilitado quando a opção *Development* está selecionada. Todas as redes em desenvolvimento serão listadas no campo *Network*. Também é possível carregar uma rede que não esteja listada, clicando no botão a direita do campo *Network*.

Quando a opção *Validation* ou *Consolidated* estão selecionadas, o campo *Engine* fica habilitado para a escolha dos motores e as redes listadas no campo *Network* são referentes ao motor que está selecionado.

Figura 30 - Configuração da RNA para Adaptação



Fonte: Autoria própria.

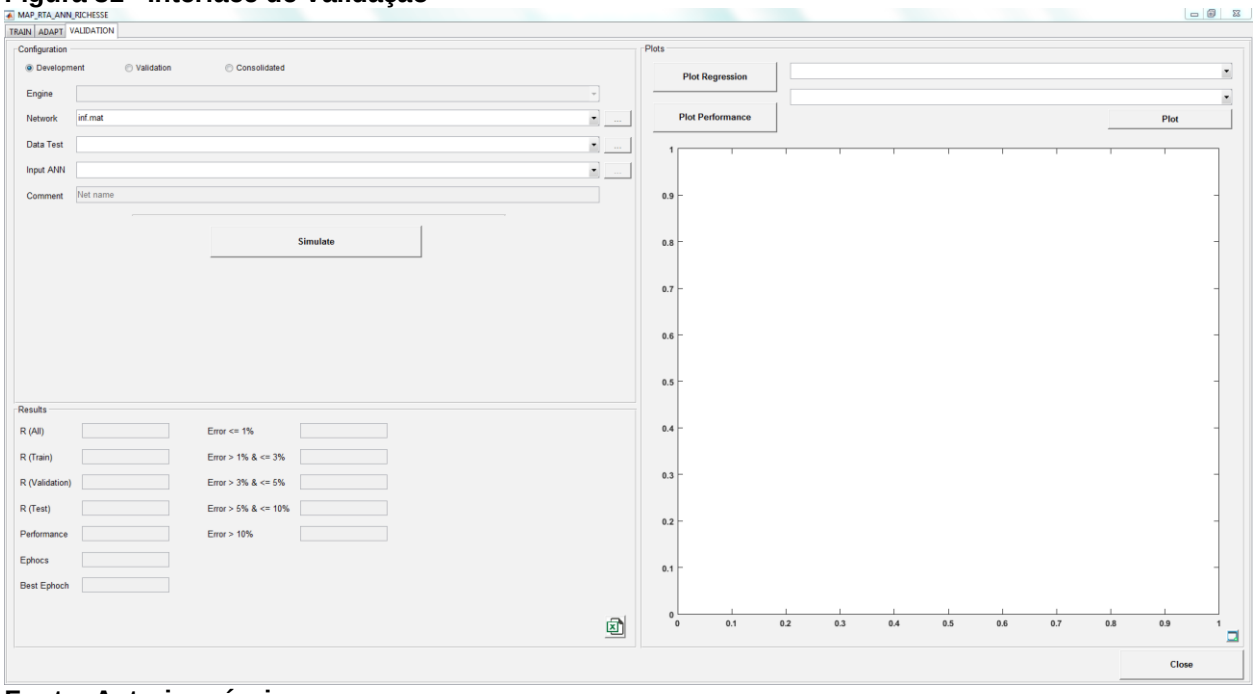
Os campos *Data Test*, *Input ANN* e *Target ANN* são idênticos aos do treinamento, sendo necessário carregar o ensaio, configurar os dados de entrada e a saída (objetivo).

O campo *Number of Iterations* refere-se ao número de iterações que serão realizadas para a adaptação da RNA, esse número é conforme a necessidade de cada RNA, sendo assim não vem com um valor pré-configurado. A adaptação tende a demorar mais que o treinamento, sendo assim, quanto maior o número de iterações, maior será o tempo para finalizar o processo.

O campo *Comment* apresenta o comentário que foi realizado na RNA e o campo *New Comment* recebe um novo comentário.

A interface de Validação segue o padrão da interface de Adaptação, a diferença entre as duas é que na Validação não há o campo *Target ANN*, pois a RNA irá simular o resultado a partir do seu aprendizado, sendo necessário somente os dados de entrada.

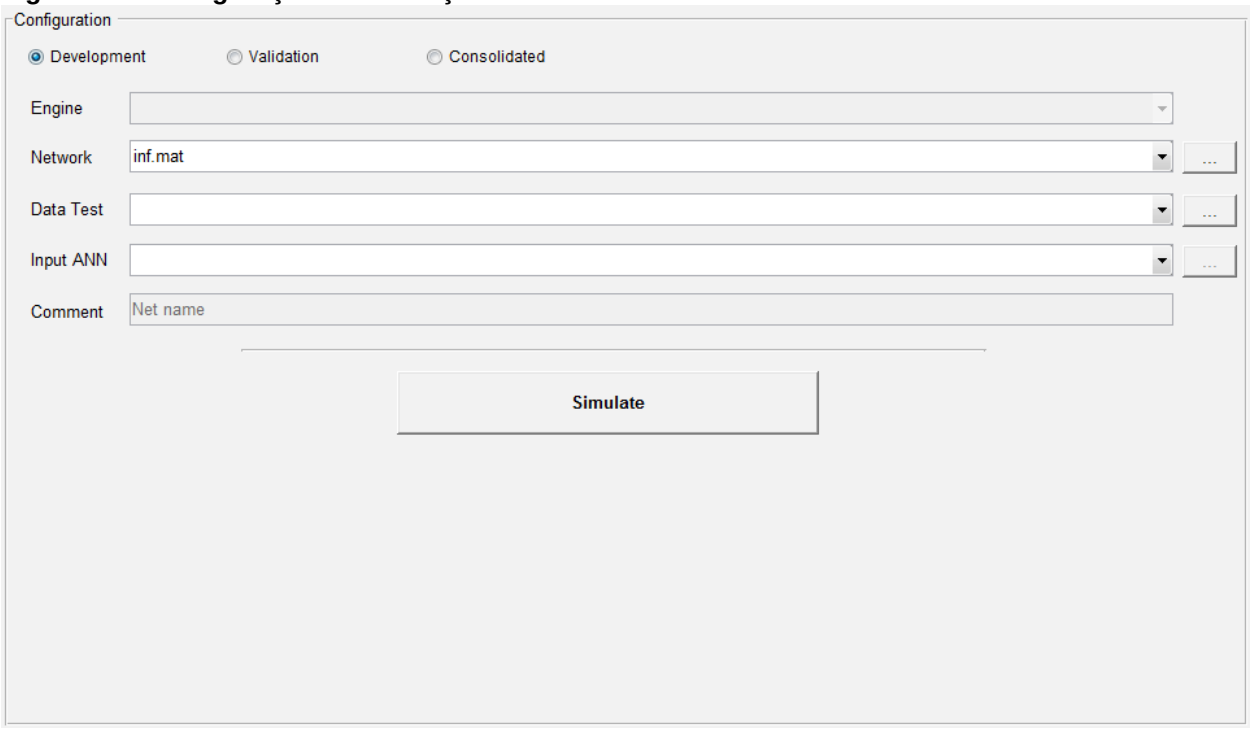
Figura 31 - Interface de Validação



Fonte: Autoria própria.

Assim como as interfaces de Treinamento e Adaptação, também é necessário carregar o ensaio e criar o conjunto de dados de entrada para a RNA. O processo de carregamento do ensaio e construção do conjunto é idêntico as outras duas interfaces.

Figura 32 - Configuração da Validação



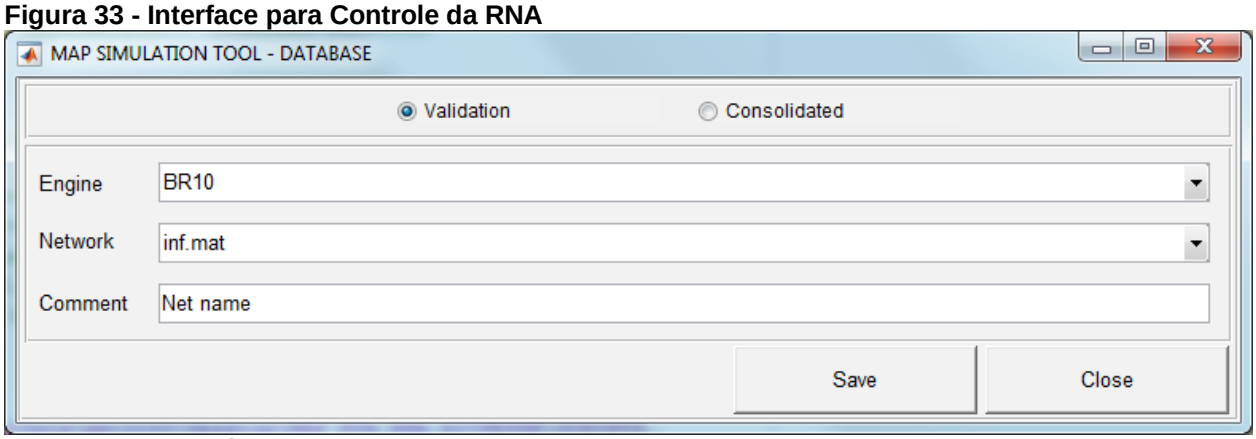
Fonte: Autoria própria.

A interface de gerenciamento das redes tem como finalidade promover uma rede que esteja em Desenvolvimento para Validação, e as que estão em Validação para Consolidação.

Quando a opção *Validation* está selecionada, serão listadas todas as redes que já foram treinadas e salvas na pasta de Desenvolvimento. Quando a opção *Consolidated* estiver selecionada, serão listadas todas as redes que estão em Validação, sendo assim, primeiro é necessário validar uma rede para posteriormente consolidar.

O campo *Engine* tem como objetivo escolher para qual motor a rede será salva. O campo *Network* lista todas as redes conforme a opção que está selecionada (*Validation* ou *Consolidated*). O campo comentário fica disponível para que algum comentário seja adicionado na rede que será salva, como por exemplo, "*RNA com taxa de erro menor que 10%*".

Figura 33 - Interface para Controle da RNA



Fonte: Autoria própria.

Promover uma rede para Validação e Consolidação é algo que deve ser feito com cuidado, sendo assim, será solicitado senha para que seja possível promover a rede e todas as operações realizadas são gravadas em um arquivo de log.